



DIGITAL DESIGN AND COMPUTER ORGANIZATION

Basic Structure of computers

T2: Chapter 1: 1.1-1.4

Department of Computer Science and Engineering

DIGITAL DESIGN AND COMPUTER

ORGANIZATION Computer Types T2:Ch1 1.1

Department of Computer Science and Engineering

Basic Structure of computers

Outline

- Computer Types
- Functional Units:
 - Input Unit,
 - Memory Unit,
 - ALU,
 - Output Unit,
 - Control Unit,
- Basic operational concepts

Basic Structure of computers

Introduction T2- section 1.1

- **What is Computer Organization ?**

Computer Organization: This field focuses on the functional units of a computer system and how they work together to execute instructions. It deals with the internal structure and operation of a computer, including its components like the CPU, memory, I/O devices, and control unit.

- **How is Computer Organization Different from Digital Design ?**

While computer organization deals with the high-level structure and function of a computer, digital design focuses on the low-level implementation of its components.

Basic Structure of computers

What is a computer ?

- Simply put, a computer is a sophisticated electronic calculating machine that:
 - Accepts **input** information,
 - **Processes** the information according to a list of internally stored instructions and
 - Produces the resulting **output** information.
- Functions performed by a computer are:
 - Accepting information to be processed as input.
 - Storing a list of instructions to process the information.
 - Processing the information according to the list of instructions.
 - Providing the results of the processing as output.

What are the functional units of a computer?

Basic Structure of computers

Types Of Computers

Classification #1:

- Micro Computers
- Mini Computers
- Mainframes
- Super Computers

Classification #2:

- Analog Computers
- Digital Computers
- Hybrid Computers

More General Classification:

- *General Purpose Computers*
- *Special Purpose Computers*

Types Of Computers

Workstations

- Workstations are *industry* – standard desktop computers with more computational power and high-resolution graphic display with variety of graphic input/output capability.

What is the use of a workstation ?

- Workstation PC's are usually employed in the applications of Computer Aided Design and Drafting (**CADD**). **Simulation** & Modeling, Interactive Graphic design, Multimedia and other engineering applications.

Types Of Computers

Enterprise Systems or Mainframes

- Enterprise systems or mainframes are like a family of computers with tremendous computing power beyond workstations.
- Their computational activities are distributed among one main system (**usually a server**) and number of child nodes or intelligent PC's with or without local computing power / processor (usually **client computers**) called dumb terminals.
- Mainframes are preferred at business data processing corporate offices. They are quite expensive with several hard disks, RAID's and backup storage units.

Types Of Computers

Super Computers

- One of the fastest computer's type currently available, *They are problem scalable.*
- Computational speed of a super computer is measured in terms of Floating Point Operations Per Second or **FLOPS**.
- Examples: Cray X/MP-14 , Param – 8000, Param - Padma



Basic Structure of computers

Functional Units T2:Ch1 1.2

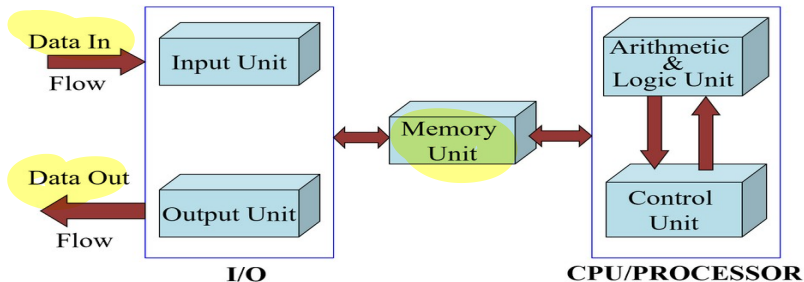
A computer in its simplest form comprises five functional units:

- 1) Input Unit
- 2) Output Unit
- 3) Memory Unit
- 4) Arithmetic & Logic Unit
- 5) Control Unit

Basic Structure of computers

Functional Units

The following figure depicts functional units of a computer:



Stored program concept-Serially fetching and executing machine code instructions stored in main memory by a processor that performs arithmetic and logical operations

Language translators- compilers

Information must be encoded -encodings- ASCII,EBCDIC

Functional Units of a Computer

Input Unit

- Computer accepts encoded information through input unit. The standard input device is a keyboard of a video monitor or terminal.
- Whenever a key is pressed, keyboard controller sends the scanned code of that letter, digit or symbol to CPU/Memory.
- Examples include Mouse, Joystick, Tablet or Digitizer, Scanner etc.



Functional Units of a Computer

Memory Unit

Memory unit stores the program **instructions**, **data** operands on and results of computations etc.

Memory has large semi-conductor storage cells.

- Words
- Word length
- Memory access time- time required to access one word
- RAM, cache, mainmemory(slowest and largest)
- Memory unit is classified as:
 - **1. Primary /Main Memory**
 - **2. Secondary /Auxiliary Memory**

Functional Units of a Computer

Memory Unit

Primary memory is the computer memory that is directly accessible by CPU. It is comprised of **DRAM** and provides the actual working space to the processor. It holds the data and instructions that the processor is currently working on.

- **Advantages of Primary Memory**

- a. **Speed:** Provides fast access to data and instructions.
- b. **Direct Access:** Allows the CPU to quickly read from and write to memory.

- **Disadvantages of Primary Memory**

- a. **Limited Size:** Is smaller compared to secondary memory

Functional Units of a Computer

Memory Unit

Secondary Memory / Mass Storage

The contents of the secondary memory first get transferred to the primary memory and then are accessed by the processor, **this is because the processor does not directly interact with the secondary memory.**

- **Advantages of Secondary Memory**
 - **Persistence:** Retains data even without power.
 - **Large Capacity:** Typically offers much more storage space than primary memory.
- **Disadvantages of Secondary Memory**
 - **Speed:** Slower access compared to primary memory.

Functional Units of a Computer

Memory Unit

1. Main memory is classified again as RAM and ROM.
 - **RAM** is termed as Read/Write memory or user memory that holds run time program instruction and data.
 - **ROM** holds system programs and firmware routines such as BIOS, POST, I/O Drivers that are essential to manage the hardware of a computer.



RAM



ROM

Functional Units of a Computer

Memory Unit

The key difference between RAM and ROM lies in their:

Purpose:

RAM: Temporary storage for data and program instructions, lost when power is turned off.

ROM: Permanent storage for firmware or system software.

Write capability:

RAM: Read-write (data can be written, modified, and erased).

ROM: Read-only (data cannot be modified after manufacture).

Volatility:

RAM: Volatile (loses data when power is turned off).

ROM: Non-volatile (retains data even without power).

Functional Units of a Computer

Memory Unit

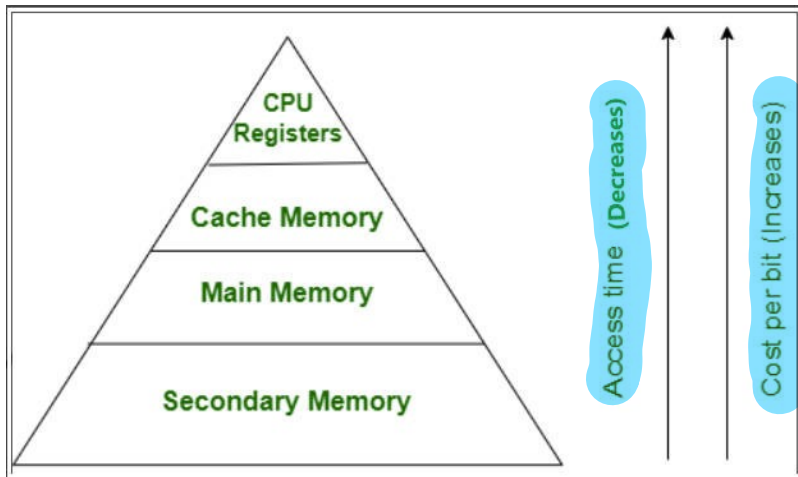
2. Secondary /Auxiliary Memory:

- While primary storage is essential, it is **volatile** in nature (i.e. its contents will be lost in the absence of power) and expensive too. Additional requirement of memory would be supplied as auxiliary memory at **cheaper cost**.
- Secondary memory are *magnetic memories* viz Floppy disk, Hard disk, Magnetic tape, CD-ROM etc.
- Secondary memories are **non volatile** in nature (contents will not be lost in the absence of power).



Functional Units of a Computer

Memory Hierarchy



What is an ALU?

- An ALU is a digital circuit that performs **arithmetic and logic operations** on binary numbers.
- ALUs are essential components of computer systems, as they enable the execution of various computational tasks.

How does an ALU work?

- ALUs **process binary data using logic gates**, which perform operations like AND, OR, NOT, and XOR. The combination of these gates allows ALUs to execute complex arithmetic and logic operations.

Functional Units of a Computer

Functions of an ALU

- Operations are **executed** in the Arithmetic and Logic Unit (ALU).
 - Arithmetic operations such as addition, subtraction.
 - Logic operations such as comparison of numbers.
- In order **to execute** an instruction, **operands** need to be brought into the ALU **from the memory**.
 - Operands are stored **in general purpose registers** available in the ALU.
 - Access times of general purpose registers are faster than the cache.
- **Results** of the operations are **stored back** in the memory or retained in the processor for immediate use.

Functional Units of a Computer

Output Unit

- Computer **returns** the computed results, error messages, etc., via **output unit**.
- The standard output device is a video monitor, LCD/TFT monitor.
- Other output devices are *printers*, *plotters* to take a paper copy of the results, programs, graphs called *print out* or a hardcopy
- Printers types: Dot Matrix printer, Inkjet printer, Laser printer. . .

Functional Units of a Computer

Control Unit

- **Coordinates activities:** The control unit oversees the operations of all components within the CPU.
- **Issues timing signals:** It sends signals like **MEMR** (Memory Read), **MEMW** (Memory Write), **IOR** (Input Output Read), and **IOW** (Input Output Write) to control data transfers.
- **Governs data transfers:** These timing signals determine **when specific operations should occur**.
- **Interprets instructions:** The control unit decodes instructions to determine the required actions.

Functional Units of a Computer

Control Unit

Operation of a computer can be summarized as:

- **Accepts** information from the input units (**Input unit**).
- **Stores** the information (**Memory**).
- **Processes** the information (**ALU**).
- Provides **processed results** through the output units (**Output unit**).
- **Operations** of Input unit, Memory, ALU and Output unit are **coordinated** by **Control unit**.
 - **Instructions control “what”** operations take place (e.g. data transfer, processing).
 - **Control unit** generates timing signals which determines **“when”** a particular operation takes place.

Main conceptual events/operations of a computer

1. A set of instructions which perform a given task, called a **program** must reside in the main memory of computer during its execution.
2. The **CPU fetches those instructions sequentially** one-by-one from the main memory, decodes them and perform the specified operation on associated data operands in ALU.
3. **Processed data** i.e. useful information will be displayed on an output unit.
4. All activities pertaining to processing and data movement inside the computer machine are governed by **control unit**.

Functional Units of a Computer

Information Handled by a Computer:

Instructions/machine instructions

- Govern the transfer of information within a computer as well as between the computer and its I/O devices
- Specify the arithmetic and logic operations to be performed
- List of instructions is called a Program(in memory)

Data (numbers or encoded chars)

- Used as operands by the instructions. Eg(Binary, BCD, ASCII,EBCDIC)

Functional Units of a Computer

Information in a computer -- Instructions

Instructions specify commands to:

- Transfer information **within** a **computer** (e.g., from memory to ALU)
- Transfer of information between the **computer** and **I/O** devices (e.g., from keyboard to computer, or computer to printer)
- Perform arithmetic and logic operations (e.g., Add two numbers, Perform a logical AND).

A sequence of instructions to perform a task is called a **program**, which is stored in the memory.

Processor fetches instructions that make up a program from the memory and performs the operations stated in those instructions.

What do the instructions operate upon?

Functional Units of a Computer

MCQ

Which is NOT a characteristic of secondary memory?

- a) Non-volatile storage
- b) Cheaper per bit than primary memory
- c) Directly accessible by the CPU
- d) Provides large storage capacity

Which of the following best describes the function of the Control Unit (CU)?

- a) Performs logical comparisons of data.
- b) Generates timing and control signals to coordinate computer operations.
- c) Stores frequently accessed data and instructions.
- d) Acts as a permanent repository for system programs.

Functional Units of a Computer

MCQ

Which is NOT a characteristic of secondary memory?

- a) Non-volatile storage
- b) Cheaper per bit than primary memory
- c) Directly accessible by the CPU
- d) Provides large storage capacity

Ans: c) Directly accessible by the CPU

Which of the following best describes the function of the Control Unit (CU)?

- a) Performs logical comparisons of data.
- b) Generates timing and control signals to coordinate computer operations.
- c) Stores frequently accessed data and instructions.
- d) Acts as a permanent repository for system programs.

Answer: b) Generates timing and control signals to coordinate computer operations

Which of the following statements is correct regarding **primary vs secondary memory**?

- a) Primary memory is slower but larger than secondary memory.
- b) Secondary memory is volatile, whereas primary memory is non-volatile.
- c) Primary memory is directly accessible by the CPU, while secondary memory is not.
- d) Both memories are equally fast but differ in size.

Which of the following statements is correct regarding **primary vs secondary memory**?

- a) Primary memory is slower but larger than secondary memory.
- b) Secondary memory is volatile, whereas primary memory is non-volatile.
- c) Primary memory is directly accessible by the CPU, while secondary memory is not.
- d) Both memories are equally fast but differ in size.

Answer: c) Primary memory is directly accessible by the CPU, while secondary memory is not



THANK YOU

Team DDCO

Department of Computer Science



DIGITAL DESIGN AND COMPUTER ORGANIZATION

Basic Operational Structure of computers

T2: Chapter 1: 1.1-1.4

Department of Computer Science and Engineering

Functional Units of a Computer

Basic Operational Concepts T2:Ch1 1.3

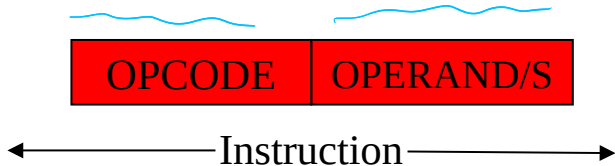
Review

- Activity in a computer is governed by **instructions**.
- To perform a task, an appropriate program consisting of a list of instructions is stored in the **memory**.
- **Individual instructions** are brought from the **memory into the processor**, which executes the specified operations.
- Data to be used as **operands** are also stored in the **memory**.

Functional Units of a Computer

Basic Operational Concepts

An Instruction consists of two parts: An Operation code and operand/s.



ADD MLOC, R0



The diagram shows an assembly instruction "ADD MLOC, R0" in blue text. A black curved arrow originates from the "ADD" opcode and points to the "R0" operand, which is underlined with a blue checkmark.

Functional Units of a Computer

Basic Operational Concepts

Execution steps: To add two operands

Step 1: **Fetch** the **instruction** from main memory into the processor

Step 2: **Fetch** the **operand** at location MLOC (memory location) from main memory into the processor

Step 3: **Add** the memory **operand** (i.e. fetched contents of MLOC()) to the contents of **register R0**

Step 4: **Store** the resulting **sum in R0** itself.

The original contents of R0 is overwritten.

Functional Units of a Computer

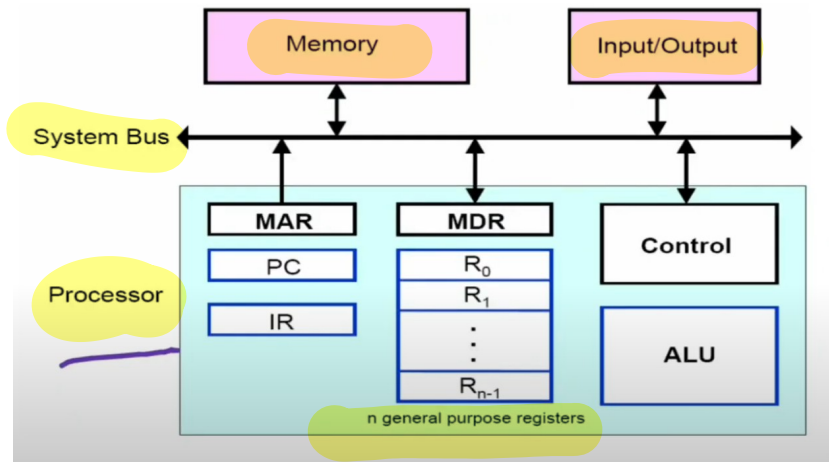
Basic Operational Concepts

- Prev example combines a memory access operation with an ALU operation
- This can be realized as
 - Load LOCA,R1**
 - Add R1,R0**
- Transfers b/w the memory and the cpu are started by sending the address of the location to memory unit and issuing the appropriate control signals

Functional Units of a Computer

Basic Operational Concepts

Processor and Main Memory Interaction



MAR → memory address register
PC → Program counter
IR → Instruction register

Program Counter (PC)

The Program Counter is a register that holds the memory address of the next instruction to be executed. It guides the processor through a program by keeping track of its place in the instruction sequence. 

- **During the fetch cycle:** The address stored in the PC is transferred to the Memory Address Register (MAR).
- **After an instruction is fetched:** The PC is automatically incremented to point to the address of the next instruction in the program.
- **Handling non-sequential instructions:** When the CPU executes a jump, branch, or subroutine call, the PC's value is modified to point to a new, non-sequential memory address. 

Memory Address Register (MAR)

The Memory Address Register is a register that stores the memory address of an instruction or data that the CPU needs to access. It acts as a bridge between the CPU and main memory (RAM). 

- **Fetching an instruction:** The address from the PC is moved into the MAR. The MAR then places this address on the address bus to retrieve the instruction from memory.
- **Accessing data:** When an instruction requires data from a specific memory location, that address is loaded into the MAR to perform a read or write operation. 

Instruction Register (IR)

The Instruction Register holds the instruction that is currently being executed or decoded. 

- **During the fetch cycle:** Once an instruction has been fetched from memory and is temporarily held in the Memory Data Register (MDR), it is transferred to the IR.
- **During the decode cycle:** The CPU's Control Unit decodes the instruction in the IR to determine the operation to be performed and the operands (data) involved. It then generates the necessary control signals to execute the instruction. 

Functional Units of a Computer

Registers

What is a Register?

Definition: A register is a **small, fast storage** location within the CPU used to temporarily hold data and instructions during processing.

Key Characteristics:

- **Fast Access:** Registers are the fastest form of memory, as they are located directly within the CPU.
- **Limited Size:** Registers usually have a very small storage capacity, typically 32-bit or 64-bit, depending on the CPU architecture.
- **Temporary Storage:** Used to store data temporarily for fast retrieval during computations.

Functional Units of a Computer

Types of Registers

- **Instruction Register (IR)**
 - **Purpose:** Temporarily holds the instruction currently being executed by the processor.
 - **Function:** Decodes the instruction, allowing the processor to understand and execute the required operation.
- **Program Counter (PC)**
 - **Purpose:** Tracks the address of the next instruction to be executed in the program sequence.
 - **Function:** Automatically increments to point to the next instruction address after each execution.

Functional Units of a Computer

Types of Registers

- **General-Purpose Registers ($R0 - Rn-1$)**
 - **Purpose:** Temporarily store operands for quick access during instruction execution, supporting various operations like arithmetic and data transfer.
 - **Function:** Hold and manipulate data during calculations, minimizing memory access for faster CPU performance.
- **Memory Address Register (MAR)**
 - **Purpose:** Holds the memory address of the location that the CPU is about to read from or write to.
 - **Function:** Communicates directly with the memory bus for fetching or storing data.

Functional Units of a Computer

General Purpose Register

		7	0	Addr.	
General Purpose Working Registers		R0		0x00	
		R1		0x01	
		R2		0x02	
		...			
		R13		0x0D	
		R14		0x0E	
		R15		0x0F	
		R16		0x10	
		R17		0x11	
		...			
		R26		0x1A	X-register Low Byte
		R27		0x1B	X-register High Byte
		R28		0x1C	Y-register Low Byte
		R29		0x1D	Y-register High Byte
		R30		0x1E	Z-register Low Byte
		R31		0x1F	Z-register High Byte

Functional Units of a Computer

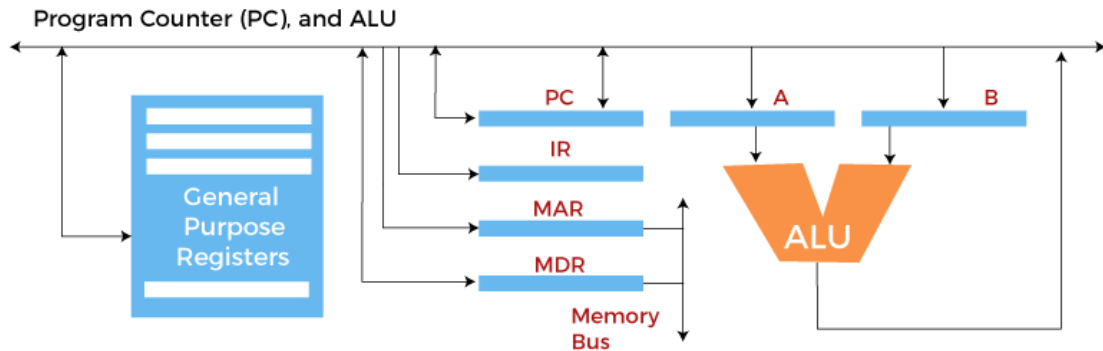
Types of Registers

- **Memory Data Register (MDR)**
 - **Purpose:** Contains the actual data being transferred to or from the addressed memory location.
 - **Function:** Acts as a buffer between the CPU and memory.

Functional Units of a Computer

Basic Operational Concepts

Operational Steps Associated with Processor & Main Memory Interaction



One-bus datapath

Functional Units of a Computer

Basic Operational Concepts

How is an instruction executed ?

A fetch-decode cycle is performed for each instruction in a program.

This means that every time the CPU needs to execute a new instruction, it goes through this cycle:

1. **Fetch:** The CPU retrieves the next instruction from memory using the program counter (PC).
2. **Decode:** The CPU interprets the instruction to determine what operation it needs to perform and what data it needs to use.
3. **Execute:** The CPU carries out the instruction, which might involve performing calculations, manipulating data, or controlling other parts of the computer.

Functional Units of a Computer

Basic Operational Concepts

How is the next instruction executed ?

After the execution of an instruction, the PC is typically incremented to point to the next instruction in the program sequence, and the cycle repeats. This continuous process allows the CPU to execute a program's instructions one by one until the program is finished.

Let us now look at this process in detail.

Functional Units of a Computer

Basic Operational Concepts

1. **Programs reside in memory through input devices:**

Programs are loaded into the computer's memory from input devices like keyboards or storage devices (e.g., hard drives, SSDs). This step typically happens before the program starts executing.

2. **PC is set to point to the first instruction:**

The Program Counter (PC) holds the address of the next instruction to be executed. At the start, it is initialized to point to the first instruction in the program.

3. **The contents of PC are transferred to MAR:**

The Memory Address Register (MAR) holds the address of the memory location that is being accessed. The address from the PC is copied to MAR.

Functional Units of a Computer

Basic Operational Concepts

4. **A Read signal is sent to the memory:**

A signal is sent to the memory to indicate that data should be read from the address specified by the MAR.

5. **The first instruction is read out and loaded into MDR:**

The instruction located at the address in MAR is read from memory and placed into the Memory Data Register (MDR).

6. **The contents of MDR are transferred to IR:**

The Instruction Register (IR) holds the current instruction to be executed. The data in MDR (which is the fetched instruction) is transferred to IR.

Functional Units of a Computer

Basic Operational Concepts

7. **The instruction is ready to be decoded and executed:**

The instruction in IR is decoded to understand what action needs to be performed.

8. If the instruction involves an operation to be performed by the ALU:

- If the instruction requires arithmetic or logical operations, the necessary operands must be obtained.
- **Get operands:**
- **General-purpose register:** If the operands are in general-purpose registers, these registers are accessed directly.

Functional Units of a Computer

Basic Operational Concepts

8.

- **Memory:** If the operands are in memory, the following steps are performed:
 - The address of the operand is transferred to MAR.
 - A Read signal is sent to memory to fetch the operand.
 - The operand is read into MDR and then transferred to the ALU.

9. Perform operation in ALU:

The Arithmetic Logic Unit (ALU) performs the operation specified by the instruction using the operands it received.

Functional Units of a Computer

Basic Operational Concepts

10. Store the result back:

- **To general-purpose register:** If the result needs to be stored in a register, it is written to the appropriate general-purpose register.
- **To memory:** If the result needs to be stored in memory:
 - The address for storing the result is transferred to MAR.
 - The result is placed in MDR.
 - A Write signal is sent to memory to store the result.

11. During the execution of the current instruction, PC is incremented to the next instruction

Functional Units of a Computer

Interrupt

- **What is an interrupt ?**
 - An interrupt is a request from an I/O device for service by the processor.
- **Why is an interrupt necessary?**
 - Normal execution of programs may be **interrupted** if some device requires **urgent** servicing.
 - To deal with the situation immediately, the normal execution of the current program must be interrupted.

Functional Units of a Computer

Procedure of interrupt operation

1. The I/O device raises an **interrupt signal**.
2. The processor provides the requested service by executing an appropriate **interrupt-service routine**.
3. The **state of the processor is first saved** before servicing the interrupt
4. Normally, the contents of the PC, the general registers, and some control information are stored in memory.
5. When the interrupt-service routine is completed, the **state of the processor is restored** so that the interrupted program may continue.

Functional Units of a Computer

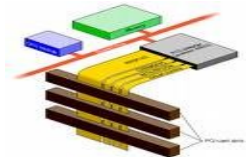
Bus Structures T2:Ch1 1.4

What is Bus ?

- A bus is a **collection of wires** that connects different parts of a computer in an organized manner for the purpose of communicating information such as memory address, I/O address, Data, Control bits etc.

Why is a bus required ?

- With the bus structure, speed of operation can be achieved since collection of wires in a **bus permits for transferring all bits of information at a time i.e., in parallel.** One byte or one full word at a time simultaneously on different wires.

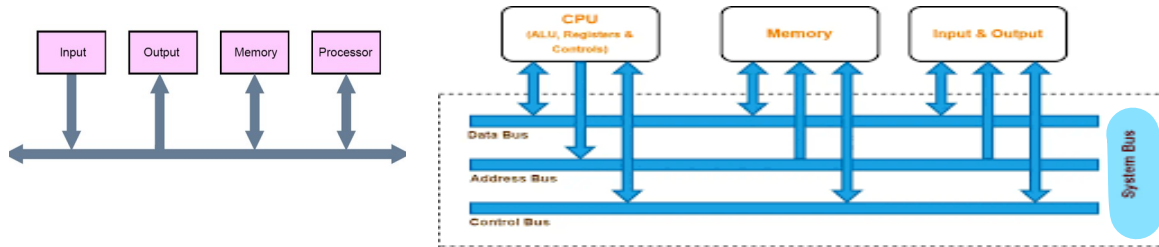


Functional Units of a Computer

Bus Structures

A group of lines that serves a connecting path for several devices is called a **bus**

- In addition to the lines that carry the **data**, the bus must have lines for **address** and **control** purposes
- The simplest way to interconnect functional units is to use a **single bus**, as shown below



Functional Units of a Computer

Bus Structures

Single-bus

- Simplest way of interconnecting functional units.
- All units are connected to this bus.
- Can be used for only one transfer at a time, thereby only 2 units can actively use the bus at a time.
- **Low Cost and flexibility for attaching peripheral devices.**

Functional Units of a Computer

Bus Structures

Drawbacks of the Single Bus Structure

- The devices connected to a bus **vary** widely in their **speed** of operation
 - Some devices are relatively **slow**, such as printer and keyboard
 - Some devices are considerably **fast**, such as optical disks
 - Memory and processor units operate are the fastest parts of a computer

Solutions

- Efficient transfer mechanism is needed to cope with this timing problem
 - A common approach is to include **buffer registers**.
 - An another approach is to use **two-bus structure** and an additional transfer mechanism(**One bus can be used to fetch instruction other can be used to fetch data, required for execution**).

Functional Units of a Computer

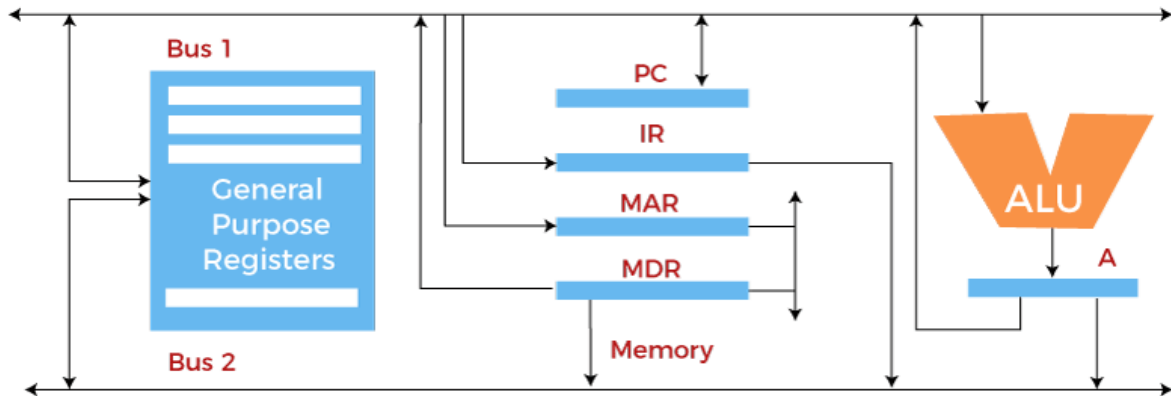
Buffer Register

Working :

- The approach is to include **buffer registers with the devices to hold the information during transfers** which smoothes out the timing difference between the cpu, memory and i/o devices
- A buffer register holds data temporarily. Buffer register compensate the timing differences among processor, memory and slow speed peripherals like Printer, Key-board, magnetic tape etc., during the transfers over a common communication path i.e., single bus.

Functional Units of a Computer

Two Bus Architecture



Two Bus Datapath

Functional Units of a Computer

Two Bus Architecture

Two-Bus Structure and an additional transfer mechanism (**One bus can be used to fetch instruction other can be used to fetch data, required for execution**).

Working :

- The approach is to include buffer registers with the devices to hold the information during transfers which smoothes out the timing difference between the cpu, memory and i/o devices
- A buffer register holds data temporarily. Buffer register compensate the timing differences among processor, memory and slow speed peripherals like Printer, Key-board, magnetic tape etc., during the transfers over a common communication path i.e., single bus.

Functional Units of a Computer

MCQ

Which of the following registers acts as a buffer between CPU and main memory?

- a) Program Counter (PC)
- b) Memory Address Register (MAR)
- c) Memory Data Register (MDR)
- d) Instruction Register (IR)

Which of the following registers acts as a buffer between CPU and main memory?

- a) Program Counter (PC)
- b) Memory Address Register (MAR)
- c) Memory Data Register (MDR)
- d) Instruction Register (IR)

Answer: c) Memory Data Register (MDR)

Explanation: MDR holds the actual data being transferred to/from memory, acting as a buffer.

The Program Counter (PC) of a processor contains 0x200. The instruction at 0x200 is a branch instruction to address 0x400. Assuming the branch is taken, what will be the content of the PC after the instruction is executed?

- a) 0x200
- b) 0x201
- c) 0x400
- d) 0x401

The Program Counter (PC) of a processor contains 0x200. The instruction at 0x200 is a branch instruction to address 0x400. Assuming the branch is taken, what will be the content of the PC after the instruction is executed?

- a) 0x200
- b) 0x201
- c) 0x400
- d) 0x401

Answer: c) 0x400

Explanation: PC holds the address of the next instruction. For a branch instruction, if the branch is taken, the PC is loaded with the branch target (0x400).

6. An **interrupt** in a computer system is best described as:
- a) A request from an I/O device for immediate processor service
 - b) A program error caused by division by zero
 - c) A trap used to handle illegal instructions
 - d) A buffer overflow condition

An **interrupt** in a computer system is best described as:

- a) A request from an I/O device for immediate processor service
- b) A program error caused by division by zero
- c) A trap used to handle illegal instructions
- d) A buffer overflow condition

Answer: a) A request from an I/O device for immediate processor service

Explanation: Interrupts are signals raised by I/O devices to get urgent CPU service.

Which of the following is a major drawback of the **single-bus structure** in computer organization?

- a) High cost of connecting devices
- b) Only one transfer at a time is possible
- c) Difficult to attach peripheral devices
- d) Requires multiple buffer registers

Which of the following is a major drawback of the **single-bus structure** in computer organization?

- a) High cost of connecting devices
- b) Only one transfer at a time is possible
- c) Difficult to attach peripheral devices
- d) Requires multiple buffer registers

Answer: b) Only one transfer at a time is possible

Explanation: A single bus allows only two devices to communicate at a time, creating bottlenecks.



THANK YOU

Team DDCO

Department of Computer Science



DIGITAL DESIGN AND COMPUTER ORGANIZATION

Number, Arithmetic Operations, And Characters Memory Locations And Address

T2: Chapter 2: 2.1-2.2

Department of Computer Science and Engineering

Number, Arithmetic Operations, And Characters

Outline

1. Number Representation
2. Addition of Positive Numbers
3. Addition And Subtractions of Signed Numbers
4. Overflow In Integer Arithmetic
5. Characters

DIGITAL DESIGN AND COMPUTER ORGANIZATION

1. Number Representation T2:Ch2 2.1

Department of Computer Science and Engineering

Number, Arithmetic Operations, And Characters

Introduction(T2- section 2.1)

- Programming languages are classified into three categories: Machine Level Language (MLL), Assembly Level Language (ALL), and High Level Language (HLL)
- ALL- readable representation of MLL
- Computers are built using logic circuits that operate on information represented by two- valued electrical signals . We label the two values as 0 and 1
- we define the amount of information represented by such a signal as a bit of information, where **bit** stands for **binary digit**.
- The most natural way to represent a number in a computer system is by a string of bits, called a **binary number**.
- **A text character can also be represented by a string of bits called a character code.**

Number Representation-signed and unsigned

Signed Integer

Consider an n-bit vector

$$B = b_{n-1} b_{n-2} \dots b_1 b_0$$

where $b_i = 0$ or 1 for $0 \leq i \leq n - 1$.

This vector can represent unsigned integer values V in the range 0 to $2^n - 1$, where

$$V(B) = b_{n-1} \times 2^{n-1} + \dots + b_1 \times 2^1 + b_0 \times 2^0$$

Number Representation-signed and unsigned

Signed Integer

- 3 major representations:

Sign and magnitude

One's complement

Two's complement

- In all three systems, the **leftmost bit is 0 for positive and 1 for negative**
- **Positive numbers have identical representations in all systems**, but negative values have different representations

Example:

4-bit machine word

16 different values can be represented

Roughly half are positive, half are negative

Number Representation

Binary Signed Integer Representation

B	Values represented		
$b_3 b_2 b_1 b_0$	Sign and magnitude	1's complement	2's complement
0 1 1 1	+ 7	+ 7	+ 7
0 1 1 0	+ 6	+ 6	+ 6
0 1 0 1	+ 5	+ 5	+ 5
0 1 0 0	+ 4	+ 4	+ 4
0 0 1 1	+ 3	+ 3	+ 3
0 0 1 0	+ 2	+ 2	+ 2
0 0 0 1	+ 1	+ 1	+ 1
0 0 0 0	+ 0	+ 0	+ 0
1 0 0 0	- 0	- 7	- 8
1 0 0 1	- 1	- 6	- 7
1 0 1 0	- 2	- 5	- 6
1 0 1 1	- 3	- 4	- 5
1 1 0 0	- 4	- 3	- 4
1 1 0 1	- 5	- 2	- 3
1 1 1 0	- 6	- 1	- 2
1 1 1 1	- 7	- 0	- 1

- **SIGN & MAGNITUDE SYSTEM:** Negative value is obtained by changing the sign bit (MSB)

Range: $-(2^{n-1}) - 1$ to $+2^{n-1} - 1$

- **SIGNED 1'S COMPLEMENT:** Negative number is obtained by complementing each bit of the corresponding positive number i.e $(2^n - 1) - N$

Range: $-(2^{n-1}) - 1$ to $+2^{n-1} - 1$

- **SIGNED 2'S COMPLEMENT:** Negative number is obtained by taking 2's complement of positive number i.e $2^n - N$
- Range: $-(2^{n-1})$ to $+(2^{n-1} - 1)$

Addition of Positive Number

T2:Ch2 2.1

0		1		0		1
+ 0		+ 0		+ 1		+ 1
-----		-----		-----		-----
0		1		1		1 0
-----		-----		-----		-----

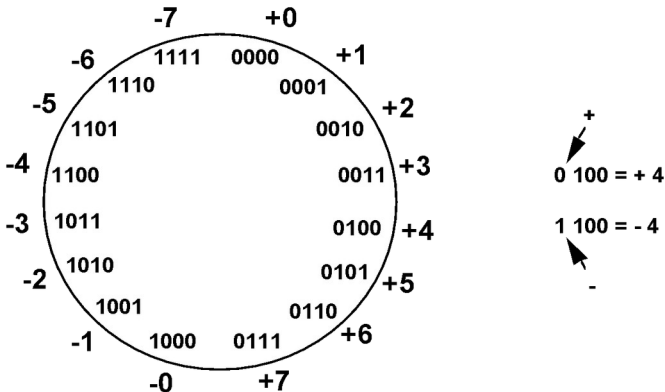
Addition of 1-bit numbers

Carry-out

Addition And Subtraction of Signed Numbers

Sign and Magnitude Representation

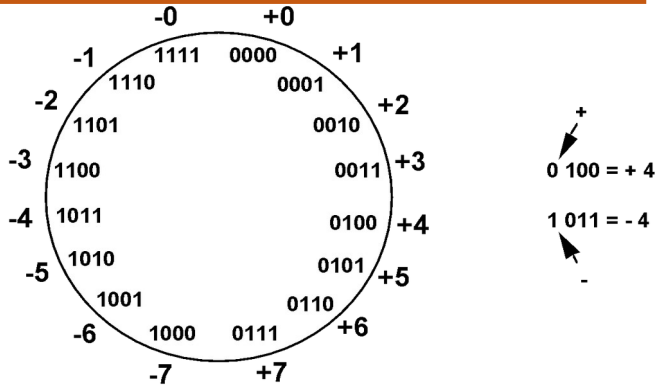
T2:Ch2 2.1



High order bit is sign: 0 = positive (or zero), 1 = negative
 Three low order bits is the magnitude: 0 (000) thru 7 (111)
 Range = $-(2^{n-1})-1$ to $+2^{n-1} -1$
 Two representations for 0

Addition And Subtraction of Signed Numbers

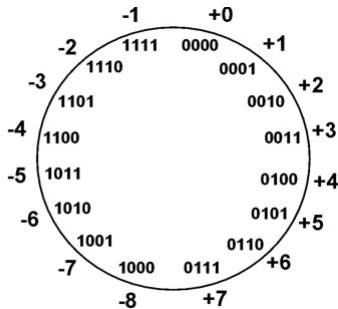
One's Complement Representation



- In 1's complement -ve value is obtained by complementing each bit of the corresponding +ve value
- The operation of forming 1's complementing is equivalent to subtracting that no. from $2^n - 1$
- Still two representations of 0! This causes some problems

Addition And Subtraction of Signed Numbers

Two's Complement Representation



Modulo 16 for 2's complement representation

- The operation of forming 2's complementing is equivalent to subtracting that no. from 2^n
- 2's complement of a no. can be obtained by adding 1 to the 1's complement of that no.
- Only one representation for 0
- One more negative number than positive number

Addition And Subtraction of Signed Numbers

Importat Points

Rules for Addition and Subtraction of n -bit Signed Numbers (2's Complement System)

1.Addition:

To add two numbers, add their n -bit representations, **ignoring the carry-out signal** from the **most significant bit (MSB)** position. The sum will be the algebraically correct value in the 2's-complement representation as long as the answer is in the range: -2^{n-1} *through* $+2^{n-1} - 1$

2.Subtraction:

To subtract two numbers X and Y (i.e., to perform $X - Y$), form the 2's-complement of Y and then add it to X , as in rule 1. Again, the result will be the algebraically correct value in the 2's-complement representation system if the answer is in the range:

$$-2^{n-1} \text{ through } +2^{n-1} - 1$$

Addition And Subtraction of Signed Numbers

- For signed arithmetic operations 2's complement format is used
- For signed arithmetic operation sign and magnitude and 1's complement format is not suitable
- To represent signed number in 2's complement using larger number of bits repeat the sign as many times needed which is called **sign extension**

Addition And Subtraction of Signed Numbers

Addition using 2's Complement

$$\begin{array}{r} 0111 \\ + 1101 \\ \hline \end{array}$$
$$\begin{array}{r} 10100 \\ \hline \end{array}$$

The carry out should be ignored

Example of addition of n – bit signed numbers using 2's complement representation

+2 0010
 +3 + 0011

+5 0101

-4 1100
 -3 + 1101

-7 ①1001

+4 0100
 -6 +1010

-2 1110

-6 1010
 +3 + 0011

-3 1101

Example of subtraction of n - bit signed numbers using 2's complement representation

$$\begin{array}{r}
 +4 \quad \quad \quad 0100 \\
 - +3 \quad \quad - 0011 \\
 \hline
 +1 \\
 \hline
 \end{array}$$

$$\begin{array}{r}
 +5 \quad \quad \quad 0101 \\
 - -2 \quad \quad - 1110 \\
 \hline
 +7 \\
 \hline
 \end{array}$$

$$\begin{array}{r}
 \quad \quad \quad 0100 \\
 + 1101 \\
 \hline
 \textcircled{1} \quad 0001 \\
 \hline
 \end{array}$$

$$\begin{array}{r}
 \quad \quad \quad 0101 \\
 + 0010 \\
 \hline
 0111 \\
 \hline
 \end{array}$$

Example of subtraction of n – bit signed numbers using 2's complement representation

$$\begin{array}{r} -3 \\ - -6 \\ \hline \\ \hline +3 \\ \hline \end{array}$$

$$\begin{array}{r} 1101 \\ - 1010 \\ \hline \end{array}$$

①

$$\begin{array}{r} 1101 \\ +0110 \\ \hline \\ \hline 0011 \\ \hline \end{array}$$

$$\begin{array}{r} -4 \\ - +2 \\ \hline \\ \hline -6 \\ \hline \end{array}$$

$$\begin{array}{r} 1100 \\ - 0010 \\ \hline \end{array}$$

①

$$\begin{array}{r} 1100 \\ + 1110 \\ \hline \\ \hline 1010 \\ \hline \end{array}$$

Example of addition and subtraction of n – bit signed numbers using 2's complement representation leading to overflow

$$\begin{array}{r} +3 \\ - -6 \\ \hline \\ +9 \\ \hline \end{array}$$

$$\begin{array}{r} 0011 \\ - 1010 \\ \hline \end{array} \rightarrow$$

$$\begin{array}{r} 0011 \\ + 0110 \\ \hline 1001 \rightarrow -7 \\ \hline \end{array}$$

$$\begin{array}{r} -4 \\ + -5 \\ \hline \\ -9 \\ \hline \end{array}$$

$$\begin{array}{r} 1100 \\ + 1011 \\ \hline 1\ 0111 \rightarrow +7 \\ \hline \end{array}$$

Summary

- It might seem that the 1's-complement representation would be just as good as the 2's-complement system. However, although complementation is easy, the result obtained after an addition operation is not always correct. The carry-out, c_n , cannot be ignored.
- If $c_n = 0$, the result obtained is correct.
- If $c_n = 1$, then a 1 must be added to the result to make it correct.
- The need for this correction cycle, which is conditional on the carry-out from the add operation, means that addition and subtraction cannot be implemented as conveniently in the 1's-complement system as in the 2's-complement system.

- When result of arithmetic operation is outside representable range, arithmetic overflow has occurred.
- Overflow will not occur when two numbers having opposite signs added
- **Overflow can occur only if two numbers having same signs are added**
- The **carry out signal** from the sign bit position **is not a sufficient indicator** of overflow when adding signed numbers
- A simple way to detect the overflow is to examine the signs of the two numbers X and Y and the sign of the result. **When both the operands X and Y have the same sign, an over flow occurs when the sign of S is not the same as the signs of X and Y**

- In addition to numbers, computers must be able to handle nonnumeric text information consisting of characters.
- Characters can be letters of the alphabet, decimal digits, punctuation marks, and so on.
- They are represented by codes that are usually eight bits long. One of the most widely used such codes is the American Standards Committee on Information Interchange (ASCII) code

Table E.2 The 7-bit ASCII code

Bit positions	Bit positions 654							
3210	000	001	010	011	100	101	110	111
0000	NUL	DLE	SPACE	0	@	P	'	p
0001	SOH	DC1	!	1	A	Q	a	q
0010	STX	DC2	"	2	B	R	b	r
0011	ETX	DC3	#	3	C	S	c	s
0100	EOT	DC4	\$	4	D	T	d	t
0101	ENQ	NAK	%	5	E	U	e	u
0110	ACK	SYN	&	6	F	V	f	v
0111	BEL	ETB	'	7	G	W	g	w
1000	BS	CAN	(	8	H	X	h	x
1001	HT	EM	)	9	I	Y	i	y
1010	LF	SUB	*	:	J	Z	j	z
1011	VT	ESC	+	;	K	[	k	{
1100	FF	FS	,	<	L	/	l	
1101	CR	GS	-	=	M	]	m	}
1110	SO	RS	.	>	N	^	n	~
1111	SI	US	/	?	O	_	o	DEL

In sign-and-magnitude representation using 4 bits, the result of $(+5) + (-5)$ is:

- a) 0000 (unique zero)
- b) 1000 (alternate zero)
- c) Either 0000 or 1000 (two representations of zero)
- d) 1111

In sign-and-magnitude representation using 4 bits, the result of $(+5) + (-5)$ is:

- a) 0000 (unique zero)
- b) 1000 (alternate zero)
- c) Either 0000 or 1000 (two representations of zero)
- d) 1111

Answer: c) Either 0000 or 1000 (two representations of zero)

Which of the following statements about overflow in signed integer arithmetic is correct?

- a) Overflow can occur when adding two numbers of opposite signs.
- b) Overflow occurs when carry-out from MSB is 1.
- c) Overflow occurs when adding two numbers of same sign and the result has opposite sign.
- d) Overflow occurs when sign extension is applied.

Which of the following statements about overflow in signed integer arithmetic is correct?

- a) Overflow can occur when adding two numbers of opposite signs.
- b) Overflow occurs when carry-out from MSB is 1.
- c) Overflow occurs when adding two numbers of same sign and the result has opposite sign.
- d) Overflow occurs when sign extension is applied.

Answer: c) Overflow occurs when adding two numbers of same sign and the result has opposite sign.

If numbers $X = 0111$ and $Y = 1101$ are added in 2's complement 4-bit representation, what is the result?

- a) 0100 with overflow
- b) 0010 with no overflow
- c) 1110 with overflow
- d) 1000 with no overflow

If numbers $X = 0111$ and $Y = 1101$ are added in 2's complement 4-bit representation, what is the result?

- a) 0100 with overflow
- b) 0100 with no overflow
- c) 1110 with overflow
- d) 1000 with no overflow

Answer: b) 0100 with no overflow



THANK YOU

Team DDCO

Department of Computer Science



DIGITAL DESIGN AND COMPUTER ORGANIZATION

Memory Locations And Addresses

T2: Chapter 2: 2.2

Department of Computer Science and Engineering

Memory Locations And Addresses

Introduction(T2- section 2.2)

- Main memory is a collection of semiconductor storage cells,
- Each Cell can store one bit of information say a “1” or a “0” bit.
- Memory stores binary information in groups of bits called **words**.
- Memory word is an addressable location to store a number, characters as data or an instruction in binary.
- The length of a memory word can be specified as n-bits per word called the **word length**.
- Modern computers have word length of the order 16-bits to 64-bits.
- 8 bits= 1 byte

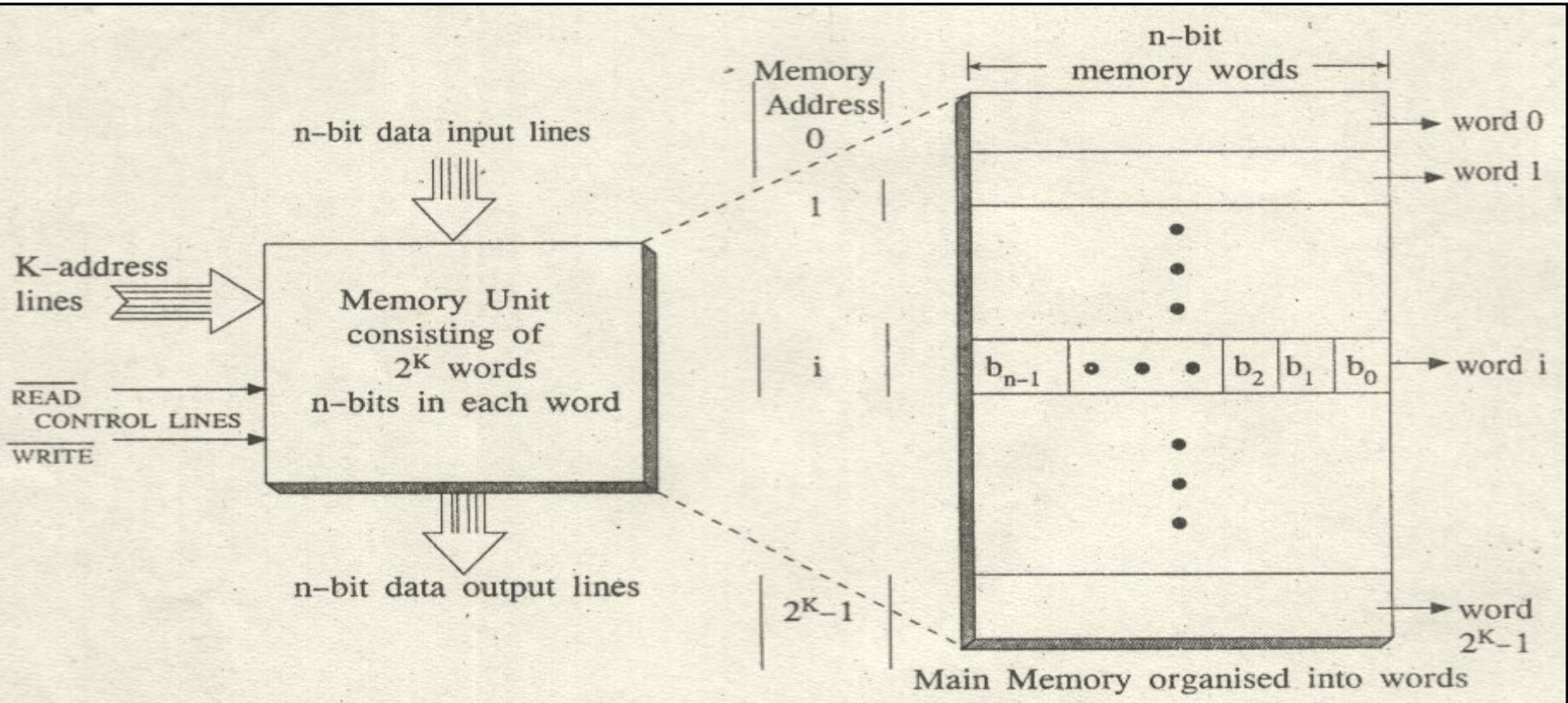
Memory Locations And Addresses

Memory Addresses

- Each word of information to be stored or retrieved from memory requires an identifying name called its **Address**.
- Special input lines called **Address lines** (wires) select each and every word in memory distinctly.
- For which each word in memory is assigned an identification number, called as address starting from 0 and continues 1,2,3,.....,up to 2^k where k is the number of address lines.
- The selection of a specific memory word is achieved by applying the K -bit binary address to the address lines.
- The address range from 0 to $2^k - 1$ constitute **address space** comprising 2^k memory words.

Memory Locations And Addresses

Memory Organization and Addressing



Memory Locations, Addresses, and Operation

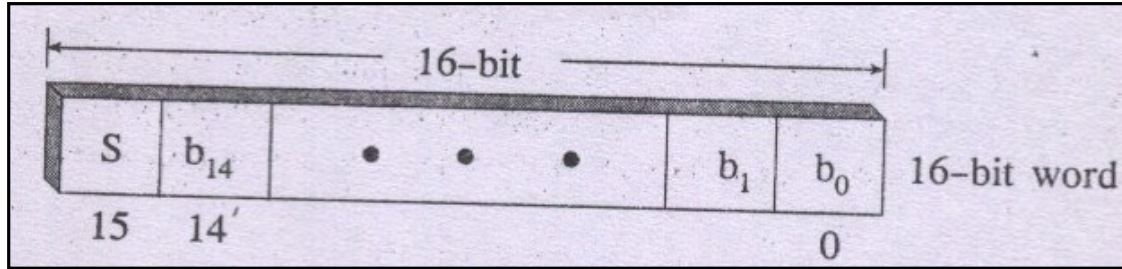
- It is impractical to assign distinct addresses to **individual bit** locations in the memory.
- The most practical assignment is to have successive addresses refer to successive byte locations in the memory - **byte-addressable memory**.
- **Byte locations have addresses 0, 1, 2, ... If word length is 32 bits, they successive words are located at addresses 0, 4, 8,...**
- **1 Tera= 2^{40}**
- **1Giga= 2^{30}**
- **1 Mega= 2^{20}**
- **1 Kilo= 2^{10}**
- **2^{24} can also be called 16MB**

Memory Locations, Addresses, and Operation

Encoding of Information

Numbers

- The length of the operand depends on the specific data type. 16 - bit
- 16 - bit memory word pattern** to hold integer numbers.



Bit 15 indicate sign bit → If it is 0, number is +ve

If it is 1, number is -ve

Binary number 0000 0010 0001 1101 is interpreted as a decimal +541

Memory Locations, Addresses, and Operation

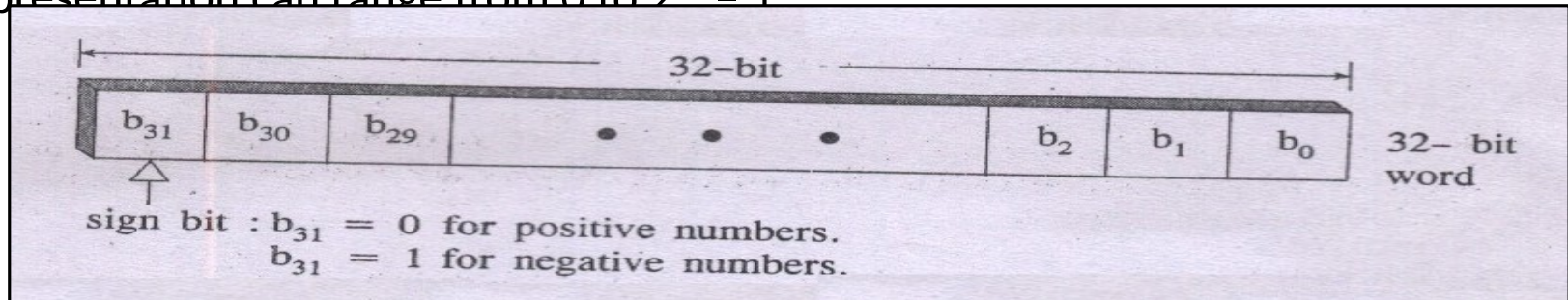
Encoding of Information

➤ **32-bit memory word pattern** to hold integer numbers

➤ Magnitude of the number can be computed using **binary positional weight notation** as:

$$= b_{30} \times 2^{30} + b_{29} \times 2^{29} + \dots + b_1 \times 2^1 + b_0 \times 2^0$$

Magnitude represented in 32-bit memory words using signed-magnitude representation can range from 0 to $2^{31} - 1$

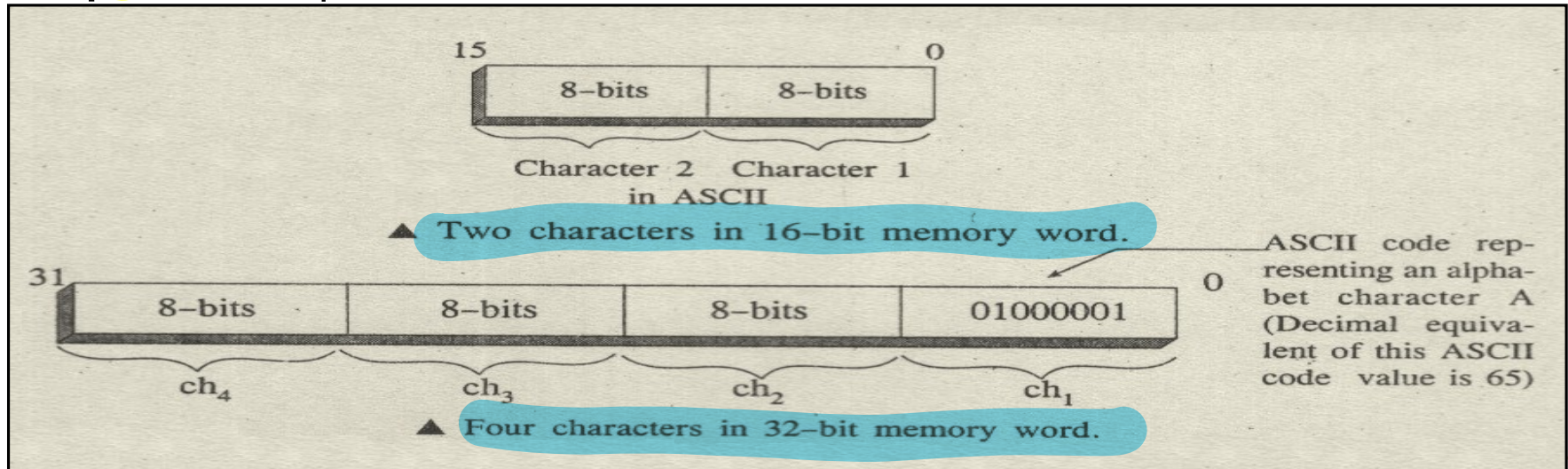


Memory Locations, Addresses, and Operation

Characters

Characters

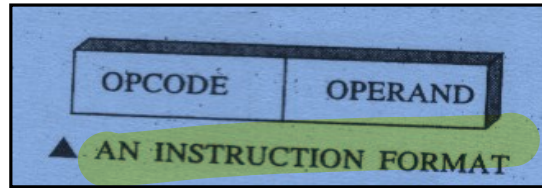
- A memory word can also store character information such as alphabets, decimal digits, symbols like +, -, \$, ?, punctuation marks like ;, ' " etc., Such characters can be represented using a ~~7~~⁸-bit ASCII code. **Memory Word Bit pattern** to represent characters



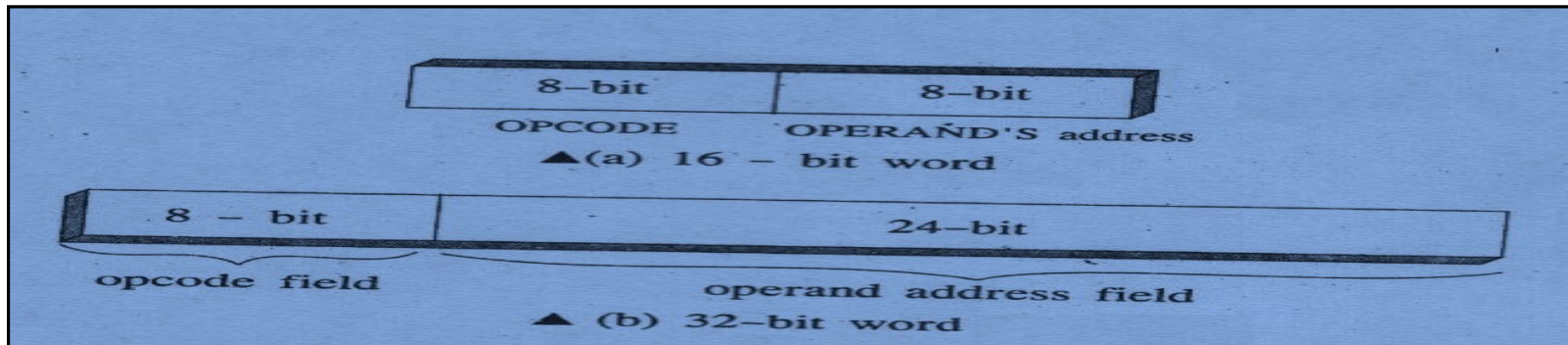
Memory Locations, Addresses, and Operation Instructions

Instructions

- Program instructions are also stored in main memory words. An instruction has two parts : **operation code (opcode)** and **operand field**:



- A machine instruction format in 16-bit and 32-bit memory words:



Memory Locations, Addresses, and Operation

Instructions

- The magnitude of the number in 16-bit memory word is computed as
- $$= b_{14} \times 2^{14} + b_{13} \times 2^{13} + \dots + b_1 \times 2^1 + b_0 \times 2^0$$

For a number in signed – magnitude representation,
where in **bit 15(b_{15})** specify a sign bit, a 0 or 1.

- Now Binary number **0000 0010 0001 1101** has the equivalent magnitude in decimal as:

$$\begin{aligned} &= 0 \times 2^{14} + 0 \times 2^{13} + 0 \times 2^{12} + 0 \times 2^{11} + 0 \times 2^{10} + 1 \times 2^9 + 0 \times 2^8 + 0 \times 2^7 \\ &\quad + 0 \times 2^6 + 0 \times 2^5 + 1 \times 2^4 + 1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 \end{aligned}$$

$$= 512 + 16 + 8 + 4 + 1 = +541$$

Memory Locations And Address

Outline

1. Byte Addressability
2. Big-Endian And Little-Endian Assignments
3. Word Alignment
4. Accessing Numbers, Characters, And Character Strings

- If the smallest addressable unit in a computer is a word, then such computers are said to be **word addressable computer**.
- A computer in which each byte stored can be addressed individually is called as **byte addressable computer**.
- If the word length of the given computer machine is **32 -bit** words, then sequence of memory words can be located subsequently at addresses 0 (word 0), 4 (word 1), 8 (word 2)...
- Each such word contains 4 bytes.

DIGITAL DESIGN AND COMPUTER ORGANIZATION

2. Big-Endian & Little-Endian Assignments T2:Ch2 2.2

Department of Computer Science and Engineering

Big-Endian And Little-Endian Assignment

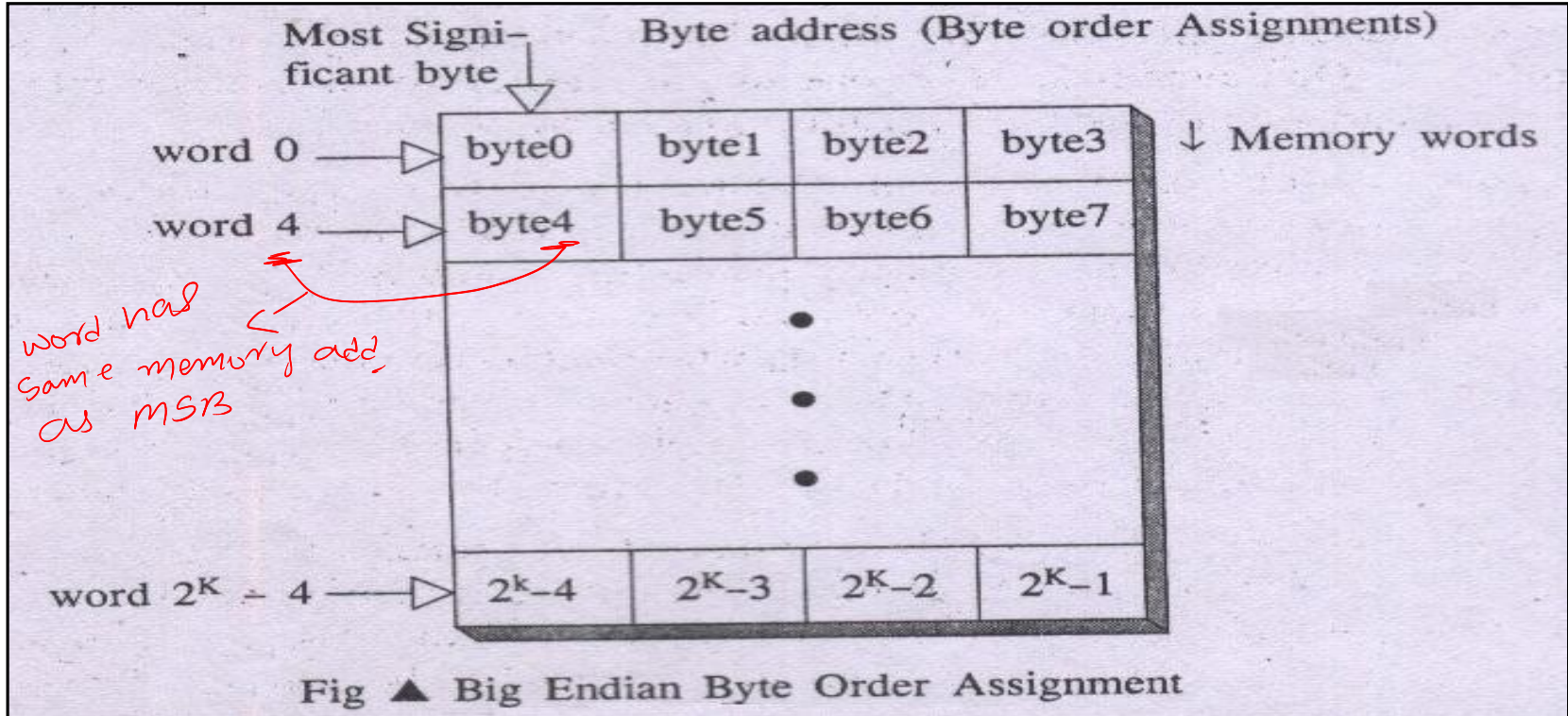
Two kinds of representations in assigning bytes in a memory word are:

- (1) Big-endian assignment
- (2) Little-endian assignment

➤ **Big - endian assignment:** Here the bytes are assigned a number starting with most significant byte (left most byte) and successively next bytes in that word. Also the word is given the same memory address as its most significant byte.

➤ Big - Endian scheme of address assignment is used in 68000(CISC processor) processor and Power PC (RISC processor) processors.

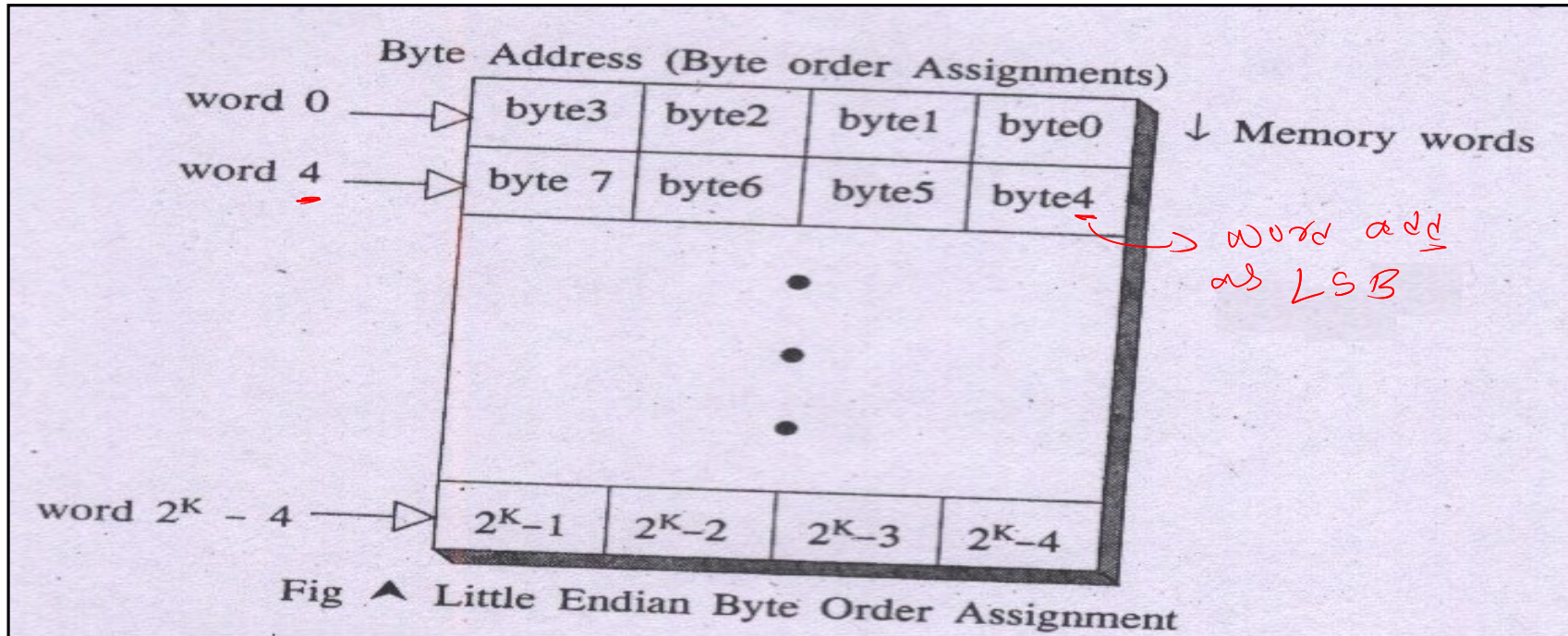
Big-Endian Assignment



Little-Endian Assignment

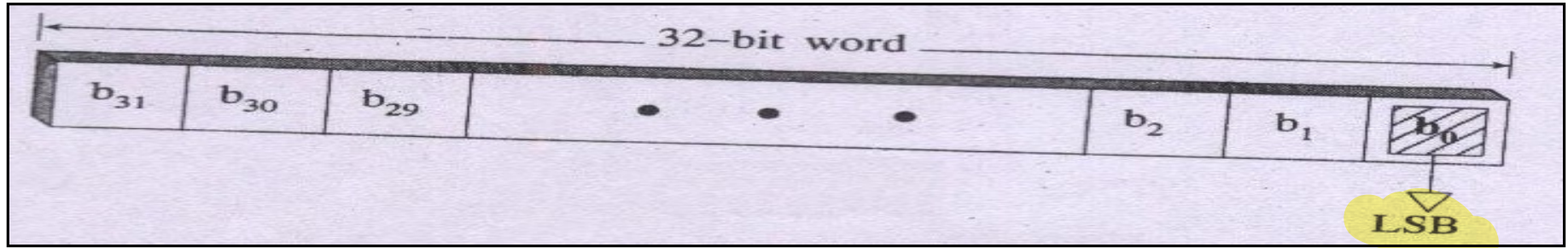
In this scheme byte 0 appears as the right most byte of word 0.

Low order bytes represents least significant bytes or right most bytes.

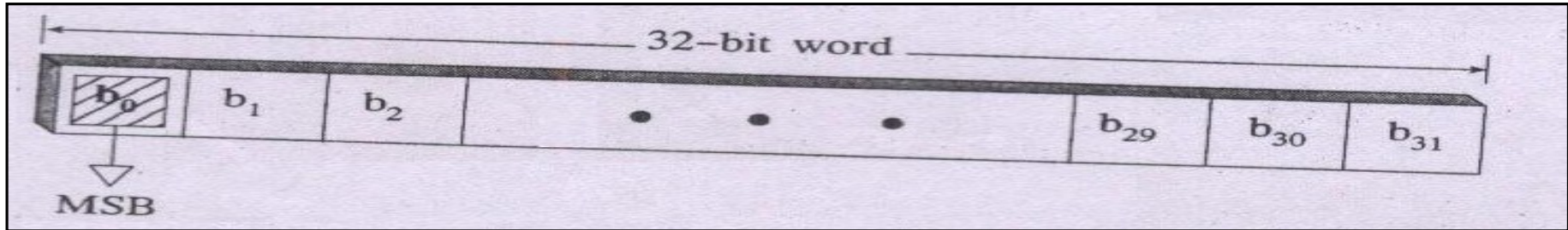


Conventions for numbering bits in a memory word

For example 68000 processor uses a convention in which bit 0 (**b0**) is treated as least significant bit (**LSB**) of a word.

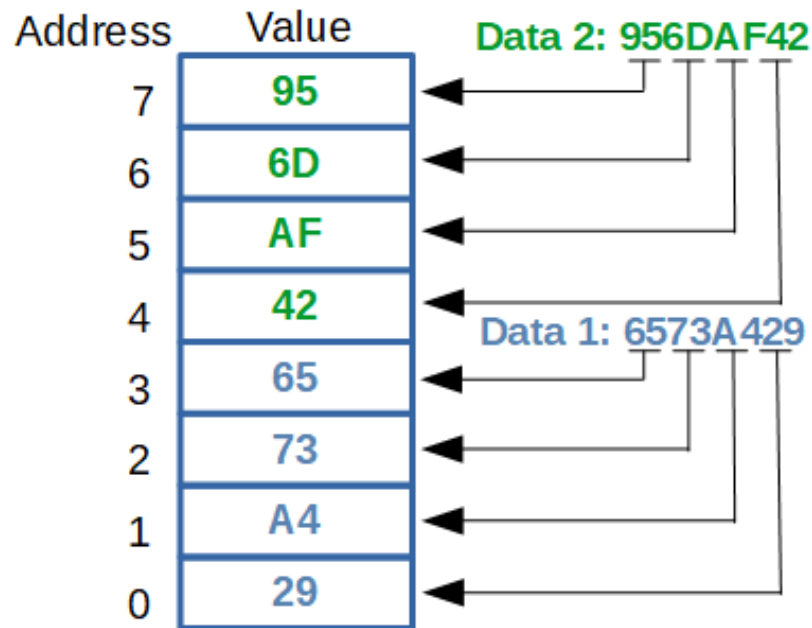


➤ Using the other convention, Power PC for example treats bit 0 (**b0**) as the most significant bit (**MSB**) of a memory word.



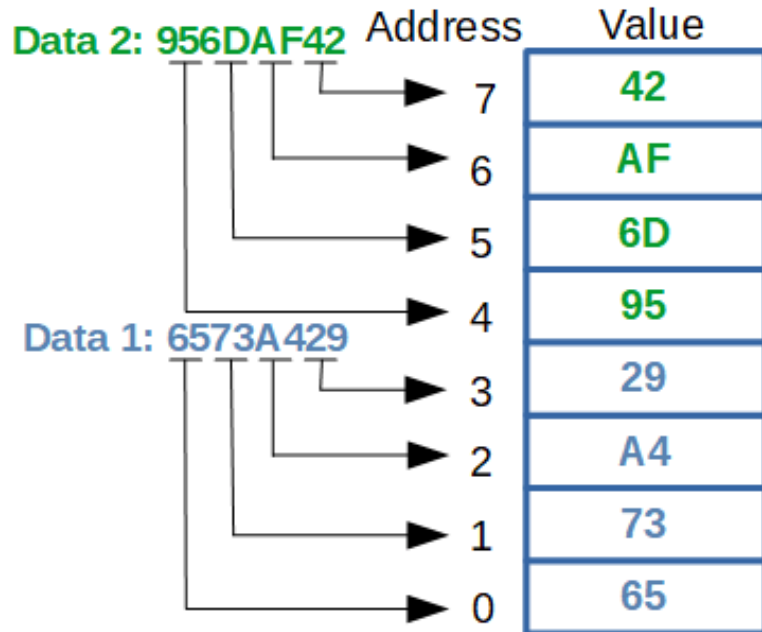
Important

Memory



Little-endian format

Memory



Big-endian format

DIGITAL DESIGN AND COMPUTER ORGANIZATION

3. Word Alignment T2:Ch2 2.2

Department of Computer Science and Engineering

Word Alignment

It refers to grouping of bytes in sequence in a given memory unit.

➤ For a computer with 32-bit word length

word 0 at address 0 (comprising bytes 0,1,2,3),

word 1 at address 4 (comprising bytes 4,5,6,7)

word 2 at address 8 (comprising bytes 8,9,10,11) ...

➤ Similarly for a computer with 16-bit word length:

word 0 at address 0 (comprising bytes 0,1),

word 1 at address 2 (comprising bytes 2,3)

word 2 at address 4 (comprising bytes 4,5) ...

➤ Similarly for a computer with 64-bit word length:

word 0 at address 0 (comprising bytes 0,1,2,3,4,5,6,7),

word 1 at address 8 (comprising bytes 8,9,10,11,12,13,14,15)

word 2 at address 16 (comprising bytes 16,17,18,19,20,21,22,23) ...

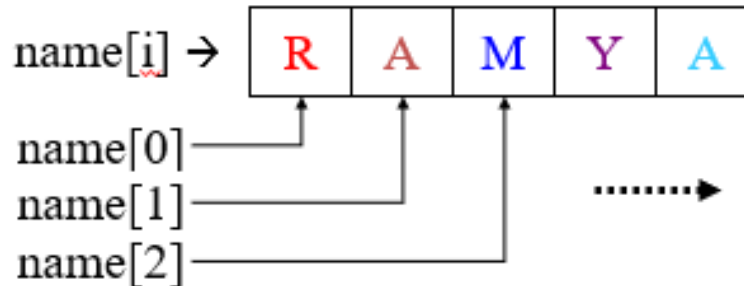
DIGITAL DESIGN AND COMPUTER ORGANIZATION

4. Numbers, Character and Character Strings T2:Ch2 2.2

Department of Computer Science and Engineering

Numbers, Character and Character Strings

- Numbers can be accessed from memory unit by specifying its **word address** or multiple byte address in memory.
- Character constants occupy *single byte* in memory. Therefore they can be accessed specifying their **byte address**.
- For example a character string **RAMYA** in an array of char can be accessed at byte addresses in memory say bytes 0,1,2,3 & 4



1. A system has 1K words of 16-bit memory. How many address lines are required to address the memory?

- a) 8 b) 10 c) 16 d) 32

2. Consider a computer with 32-bit word length using **Big-Endian** format. If a 32-bit integer is stored at address 1000, which byte holds the Most Significant Byte (MSB)?

- a) Address 1000
b) Address 1001
c) Address 1002
d) Address 1003

1. A system has 1K words of 16-bit memory. How many address lines are required to address the memory?

- a) 8 b) 10 c) 16 d) 32

Answer: b) 10

(Reason: $2^{10} = 1024$, 1K words, independent of word size.)

2. Consider a computer with 32-bit word length using **Big-Endian** format. If a 32-bit integer is stored at address 1000, which byte holds the Most Significant Byte (MSB)?

- a) Address 1000
b) Address 1001
c) Address 1002
d) Address 1003

Answer: a) Address 1000

(Big-endian → MSB stored at lowest memory address.)

A memory unit has 16 address lines. If the memory is byte-addressable, what is the maximum memory capacity?

- a) 64 KB
- b) 128 KB
- c) 256 KB
- d) 512 KB

A computer uses 32-bit word length. In a byte-addressable memory, what are the addresses of the first four words?

- a) 0, 1, 2, 3
- b) 0, 4, 8, 12
- c) 0, 2, 4, 6
- d) 0, 8, 16, 24

A memory unit has 16 address lines. If the memory is byte-addressable, what is the maximum memory capacity?

- a) 64 KB
- b) 128 KB
- c) 256 KB
- d) 512 KB

Answer:

(Reason: $2^{16} = 65,536$ addresses $\rightarrow$ 64K bytes = 64 KB.)

A computer uses 32-bit word length. In a byte-addressable memory, what are the addresses of the first four words?

- a) 0, 1, 2, 3
- b) 0, 4, 8, 12
- c) 0, 2, 4, 6
- d) 0, 8, 16, 24

Answer: b) 0, 4, 8, 12 (Reason: Each word = 4 bytes; hence addresses increase by 4 for successive words.)

Think about it

A computer system uses a **32-bit word length** with a **byte-addressable memory**. The system supports both **Big-Endian** and **Little-Endian** formats. The system has **20 address lines**.

- Calculate the total size of the memory in bytes and words.
- If an integer 0x12345678 is stored at starting address 1000, show how the bytes will be stored in **Big-Endian** and **Little-Endian** format.
- Demonstrate how **word alignment** affects storage by showing the addresses of the first four words.
- If a character string "GATE" is stored beginning at address 2000, show how it is represented in memory.
- Briefly explain why byte-addressable memory and endianness conventions are important for **system compatibility**.



THANK YOU

Team DDCO

Department of Computer Science



DIGITAL DESIGN AND COMPUTER ORGANIZATION

Addressing Modes T2: Chapter 2: 2.5

Department of Computer Science and Engineering

Introduction(T2- section 2.5)

- Situations like **program looping**, **branching** demand convenient ways of locating memory words or memory operands.
- This convenience is being offered by different instructions (of a particular instruction set as defined by the **Microprocessor**) via various **Addressing Modes** they support.
- Suppose a given memory operand address is **too large to fit into a given instruction format** (i.e., in operand field), then it is the addressing mode (say **Register Indirect Mode**) that can resolve this problem.

Addressing Modes

Introduction

What is Addressing Mode . . . ?

The different ways in which the location of an operand is specified in an instruction are referred to as *addressing modes*

Addressing Modes

Introduction

What is Addressing Mode ... ?



- The addressing mode specifies a rule or method for interpreting or modifying the address field of the instruction before the operand is actually accessed for manipulation.
- It is concerned with the different ways in which the location of an operand can be specified in a given instruction.
- Various schemes for specifying addresses of operands in an instruction have been introduced. Such schemes are collectively known as Addressing Modes.

Addressing Modes

Introduction

Advantages of Addressing modes :

- To be able to reference large number of memory location in main memory or for some system in **virtual memory**.
- To **reduce number of bits** in the address field of an instruction
- To extend programming convenience to the user / programmer by providing such facilities as **indexing of set of data values, counters for loop control, pointers to memory locations**

Addressing Modes

Outline

1. Immediate mode
2. Absolute (Direct) mode
3. Register mode
4. Indirect mode
5. Indexed mode
6. Relative mode
7. Auto increment mode
8. Auto decrement mode

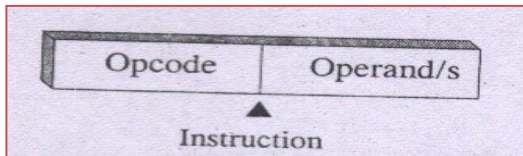
DIGITAL DESIGN AND COMPUTER ORGANIZATION

1. Immediate mode T2:Ch2 2.5

Department of Computer Science and Engineering

Immediate mode

- Here the operand is specified in the instruction itself. An instruction that follows immediate mode has an **operand field** rather than an address field.



For example:

Move 50_{immediate}, R0

	R2
	R1
50	R0

- A common convention says, a symbol # has to precede the value of an immediate operand

Move #50, R0

Immediate mode

Example

Write snippet of code for $A=B+6$

Move B,R1

Add #6,R1

Move R1,A

DIGITAL DESIGN AND COMPUTER ORGANIZATION

2. Absolute (Direct) mode

T2:Ch2 2.5

Department of Computer Science and Engineering

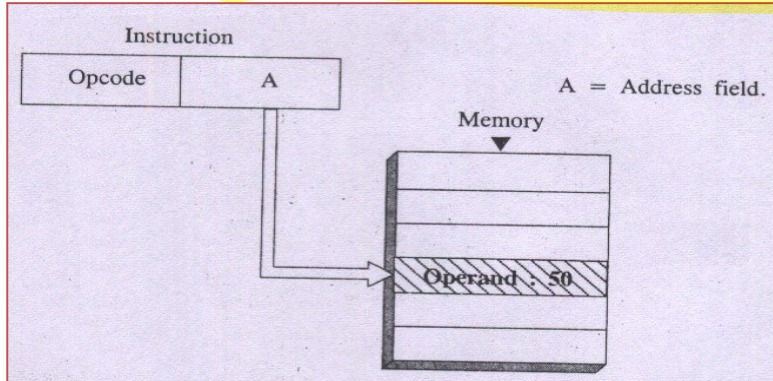
We accessed an operand by specifying the name of the register or the address of the memory location where the operand is located. The precise definitions of these two modes are:

Addressing Modes

- **Register Mode** – The operand is the contents of a processor register; the name (address) of the register is given in the instruction.
- **Absolute Mode** – The operand is in a memory location; the address of this location is given explicitly in the instruction. (In some assembly languages, this mode is called *Direct*.)

Absolute (Direct) mode

➤ Here **operand resides in Memory** and its address is given explicitly in the address field of an instruction. This scheme need only one memory reference in addition to instruction fetch cycle and no further calculation is required to compute operand address.



Absolute (Direct) mode

- Direct addressing scheme is **simple to use** and easy to implement without the requirement for additional hardware.
- However it offers only **limited memory address space**.

➤ Examples

Move LOC,**R0**

Load **P**

Store **S**

DIGITAL DESIGN AND COMPUTER ORGANIZATION

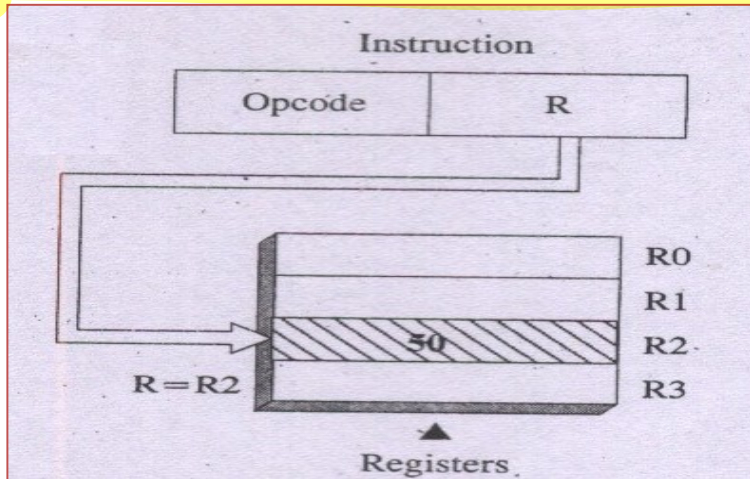
3. Register mode T2:Ch2 2.5

Department of Computer Science and Engineering

Register Addressing mode

➤ In this scheme, the operand is the contents of a register appearing in the instruction i.e $A=R$

➤ example
Add R1,R2



Register Addressing mode

Advantages of this scheme :

- ✓ No memory reference
- ✓ A few bit address to indicate register location
- ✓ Speedy execution since register is inside the processor
& has low access time.

Disadvantages of this scheme :

- ✓ Limited address space as number of registers are less
in many of the processors.

DIGITAL DESIGN AND COMPUTER ORGANIZATION

4. Indirect mode

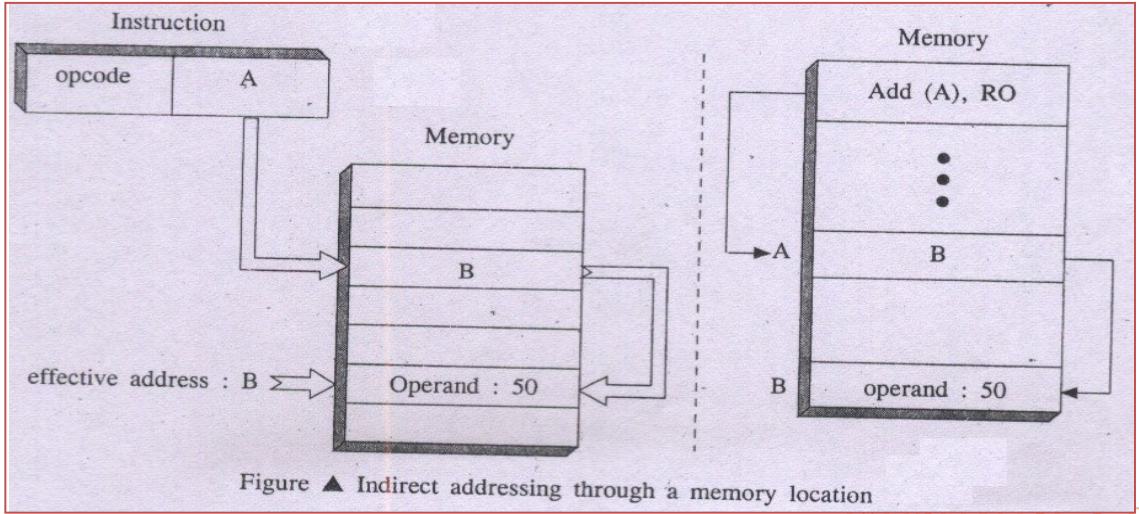
T2:Ch2 2.5

Department of Computer Science and Engineering

Indirect Addressing mode

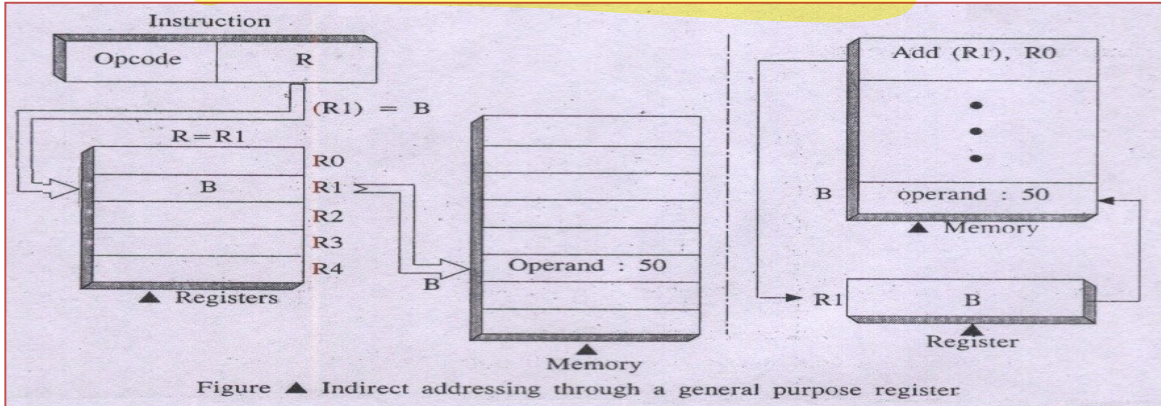
- The effective address of the operand is the contents of a register or memory location whose address appears in the instruction
- By referring to this address (EA), the required operand can be fetched from memory.
- Example: **Add (A), R0** i.e. $EA = (A)$ i.e. contents of **A** is **B**
Here **B** is effective address of desired operand in memory

Indirect Addressing mode



Register Indirect Addressing mode

- Here, instruction specifies a register in the CPU whose contents give the effective address of the operand in Memory.
- For example **Add (R1), R0** i.e. $EA = (R1)$ i.e. contents of **R1** is **B**



Indirect Addressing mode-

Program to add list of n numbers

Address	Contents
	Move N, R1
	Move #NUM, R2
	Clear R0
LOOP	Add (R2), R0
	Add #4, R2
	Decrement R1
	Branch > 0 LOOP
	Move R0, SUM

Indirect Addressing mode

The advantage of Register Indirect Addressing:

- It uses one less memory reference (memory read operation)
- Address field of the instruction uses a fewer bits to specify a register
- Register indirect addressing can be specified with

Effective Address $EA = (R)$ i.e. B

Advantages of Indirect addressing:

a wider address range to refer to a large number of memory locations.

Disadvantage of Indirect addressing:

2 or more memory references (memory read operations) required to fetch the desired operand in memory.

Indirect Addressing mode

The Register or memory location that contains address of operand is called pointer

Consider C language snippet:

`A=*B`

equivalent :

`Move B,R1`

`Move(R1),A`

Using indirect addressing:

`Move (B),A`

move the value inside R1 into A. NOT move R1 into itself (reference)

DIGITAL DESIGN AND COMPUTER ORGANIZATION

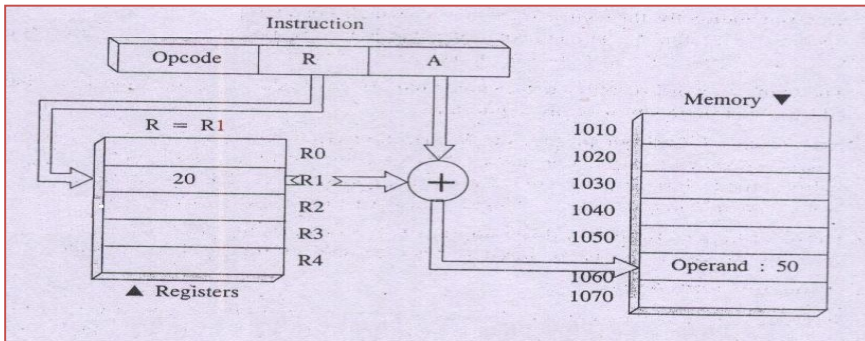
5. Indexed mode T2:Ch2 2.5

Department of Computer Science and Engineering

Indexed Addressing mode

- Here, effective address of the operand is generated by adding a constant value to the contents of a register.
- This constant value may be either an *offset value* called *displacement* or *beginning address* of data/operand array in main memory (*Base*).
- Indexed addressing mode is symbolically represented as $X(R)$
 - 👉 Here X denote a constant and R is name of the register involved in **Indexing**.
- Effective address EA of the operand is given as
$$EA = X + [R]$$
 - 👉 Contents of index register are not changed during the process of address generation.

Indexed Addressing mode



$A = 1040 = X$; constant X corresponds to a memory location

$(R) = 20$ i.e. **Index** or **offset** value

$EA = A + (R) = 1040 + 20$

$EA = 1060$ at which you will find desired operand **50**

Indexed Addressing mode

You can use indexed addressing mode in two ways

- I) A constant value X defines here beginning address of operand in memory and index register Ri contains offset value (Displacement)

👉 For example 1040 in the following instruction:

Add 1040 (R1), R2

X=1040 R1=20 offset

- II) A constant value X defines an offset and index register Ri contains beginning address of operand in memory.

👉 For example 20 in the following instruction:

Add 20(R1), R2

X=20 R1=1040

Indexed Addressing mode

- Index addressing is fast and is excellent for manipulating data structures such as **arrays** as all you need to do is set up a base address then use the index in your code to access individual elements.
- Another advantage of indexed addressing is that if the **array is re-located in memory at any point then only the base address needs to be changed**. The code making use of the index can remain exactly the same.

Indexed Addressing mode

Example

Example: Add 20(R1),R2

Add 2000(R3),R4

Assume that a list of scores of the student beginning at location LIST as shown in the following diagram

A 4 word memory comprises a record that stores relevant info for each student

Compute the sum of all scores obtained on each of the test and store these sums in memory locations SUM1,SUM2,SUM3

Indexed Addressing mode

N 5000

LIST 5004

LIST+4 5008

LIST+8 5012

LIST+12 5016

LIST+16 5020

n
Student ID
Test1
Test2
Test3
Student ID
Test1
Test2
Test3
computer organization_ unit-1
.
.

Indexed Addressing mode

	Move		#LIST, R0
	Clear		R1
	Clear		R2
	Clear		R3
	Move		N, R4
LOOP	Add		4(R0), R1
	Add		8(R0), R2
	Add		12(R0), R3
	Add		#16, R0
	Decrement	R4	
	Branch>0	LOOP	
	Move		R1, SUM1
	Move		R2, SUM2
	Move		R3, SUM3

Indexed Addressing mode

☐ *Advantages of Index mode*

is the flexibility it offers to access relative memory locations

☐ *Disadvantages of Index mode*

- * Is the complexity of computing effective address.
- * **The instruction requires to have two address fields at least one of which is an explicit number.**

Indexed Addressing mode

- Several variations of this basic form provide a very efficient access to memory operands in practical programming situations.
- For example, a second register may be used to contain the offset X, in which case we can write the Index mode as

(Ri, Rj)

The effective address is the sum of the contents of registers Ri and Rj.

- The second register is usually called the **base register**.
- This form of indexed addressing provides more flexibility in accessing operands, **because both components of the effective address can be changed.**

Indexed Addressing mode

Another version of the Index mode uses two registers plus a constant, which can be denoted as

$$X(R_i, R_j)$$

In this case, the effective address is the sum of the constant X and the contents of registers R_i and R_j .

This added flexibility is useful in accessing multiple components inside each item in a record, where the beginning of an item is specified by the (R_i, R_j) part of the addressing mode.

DIGITAL DESIGN AND COMPUTER ORGANIZATION

6. Relative mode T2:Ch2 2.5

Department of Computer Science and Engineering

Relative mode

- This scheme supplies the relative position of the memory operand to be located.
- It's like index mode only but program counter register PC substitutes for base address contents
- Commonly used to specify the target address in branch instruction
- Relative Mode specify Effective Address by a notation:

$X(PC)$

- Effective address is $EA = [PC] + X$

- **Branch > 0 loop** Here jump value / displacement $\pm X$

DIGITAL DESIGN AND COMPUTER ORGANIZATION

7. Auto increment mode T2:Ch2 2.5

Department of Computer Science and Engineering

Auto increment mode

- The EA of the operand is the contents of a register specified in the instruction
- After accessing the operand, the contents of this register are incremented **automatically to point to the next operand** in contiguous memory locations.
- Notation for Auto Increment Mode: $(R_i) +$
- For example: **Add (R2) +, R0**
- Use of Auto Increment mode instruction eliminates the use of explicit increment instruction

Auto increment mode

Implicitly, the increment amount is 1 when the mode is given in this form. But in a byte-addressable memory, this mode would only be useful in accessing successive bytes of some list.

To access successive words in a byte-addressable memory with a 32-bit word length, the increment must be 4. Computers that have the Autoincrement mode automatically increment the contents of the register by a value that corresponds to the size of the accessed operand.

Thus, the increment is 1 for byte-sized operands, 2 for 16-bit operands, and 4 for 32-bit operands. Since the size of the operand is usually specified as part of the operation code of an instruction, it is sufficient to indicate the Autoincrement mode as $(Ri)+$.

Auto increment mode

Program to add n numbers using autoincrement

```
    MOVE N,R1
    MOVE #NUM1,R2
    CLEAR R0
LOOP  ADD (R2)+,R0
      DECREMENT R1
      BRANCH >0 LOOP
      MOVE R0,SUM
```

DIGITAL DESIGN AND COMPUTER ORGANIZATION

8. Auto decrement mode T2:Ch2 2.5

Department of Computer Science and Engineering

Auto decrement mode

- Here, content of a register specified in the instruction is
- Decrement to denote effective address of next operand in
- Successive memory location.
- Notation for Auto Decrement Mode: $-(R_i)$
- For instance
 - **Add $-(R2), R0$**
- It allows accessing of operands in decreasing order of memory address..

Auto decrement mode

The actions performed by the Autoincrement and Autodecrement addressing modes can obviously be achieved by using two instructions, one to access the operand and the other to increment or decrement the register that contains the operand address. Combining the two operations in one instruction reduces the number of instructions

Addressing Mode

The different ways in which the location of an operand is specified in an instruction are referred to as addressing modes.

Name	Assembler syntax	Addressing function
Immediate	#Value	Operand = Value
Register	R i	EA = R i
Absolute (Direct)	LOC	EA = LOC
Indirect	(R i) (LOC)	EA = [R i] EA = [LOC]
Index	X(R i)	EA = [R i] + X
Base with index	(R i, R j)	EA = [R i] + [R j]
Base with index and offset	X(R i, R j)	EA = [R i] + [R j] + X
Relative	X(PC)	EA = [PC] + X
Autoincrement	(R i)+	EA = [R i] ; Increment R i
Autodecrement	-(R i)	Decrement R i ; EA = [R i]



THANK YOU

Team DDCO

Department of Computer Science



DIGITAL DESIGN AND COMPUTER ORGANIZATION

Memory Operations Instructions And Instruction Sequencing

T2: Chapter 2: 2.3-2.4

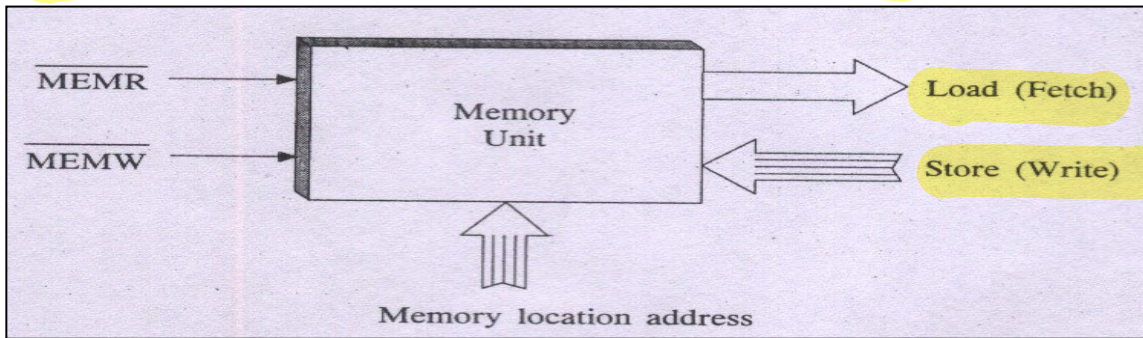
Department of Computer Science and Engineering

Memory Operations

Introduction(T2 -section 2.3)

While executing a program, two distinct operations associated with Processor-Memory interaction are:

- **Load** operation / Memory **Read** - (Instruction fetch / Operand fetch)
- **Store** operation / Memory **Write** - (Write computed results)



Memory Operations

Load (Read Or Fetch)

- Transfers a copy of the contents of a specific memory location to the processor
- Processor sends address of the desired location to memory and requests that its contents be read (Issues read signal)
- Memory reads the data and sends it to the processor
- Memory contents remains unchanged

Memory Operations

Store Or Write

- Transfers an item of info from the CPU to a specific memory location, destroying the former contents of that location
- Processor sends the address of the desired location to the memory, together with the data to be written
- Issues write signal
- Sends the data via **data bus** and write into the selected particular memory location

Instructions And Instruction Sequencing

Introduction(T2 – section 2.4)

- An **instruction** is a command to the processor to perform a given task on data operands.
- A **program** is a set of instructions that specify **operations**, **operands** & the **sequence** by which processing has to occur.
- Thus operation of a computer is controlled by **stored program**
- In general a computer must support 4 categories of operations:
 1. **Data Movement** : To conduct I/O transfers
 2. **Data Storage** : Across Memory and processor registers.
 3. **Data Processing** : Arithmetic and logical operations
 4. **Program sequencing and control** : Test and Branch.

Register Transfer Notation

- Data operands & Instructions are situated in main memory locations & processor registers.
- Like **mnemonics** for Opcodes, Symbolic names or label identifiers are used to name or identify such locations.
- For instances **memory location names** include **M, LOC, A, B, C, SUM, N1, VAR1, VAR2, NUM1**, etc.,
- Similarly **processor register names** include **R0, R1, R2, R3, R4, R5**, etc., and **registers** of I/O subsystem may be designated as **DATAIN, DATAOUT** etc.

Register Transfer Notation

- While depicting operations, the contents of a memory location or a register are denoted by placing the corresponding name in a pair of square brackets.

For instance :

$R0 \leftarrow [M]$

Suggests to copy contents of memory location M to register $R0$

- The corresponding instruction is

MOVE $M, R0,$

Memory
Location

Register

- Similarly : MOVE $R1, SUM$

Suggests : $SUM \leftarrow [R1]$

Register

Memory Location

- The instruction : MOVE B, LOC Means : $LOC \leftarrow [B]$

Register Transfer Notation

- Similar operations that involve **Registers only** are:

R1 $\leftarrow$ [**R2**] **MOVE R2, R1**

- **R1** $\leftarrow$ [**R1**] + [**R3**] **ADD R3, R1**

Suggest to add contents of register R1 and register R3 and rewrite the R1 contents to store the sum.

- All such notations with leftward arrow ($\leftarrow$) assignment operations involving register locations give rise to what is known as **Register Transfer Notation**

DIGITAL DESIGN AND COMPUTER ORGANIZATION

2. Assembly Language Notation T2:Ch2 2.4

Department of Computer Science and Engineering

Assembly Language Notation

- An assignment statement in a High Level Language Program:

$S = P + Q$

- Here **P**, **Q** & **S** are variables.

- **Variable name** refers to *symbolic address of memory location* from which data operand can be read or written.

$S \leftarrow [P] + [Q]$

- This operation suggests contents of two memory locations **P** and **Q** are fetched from memory and transferred into processor where sum of two numbers is performed. The resulting sum is then sent back to memory and stored in memory location **S**.

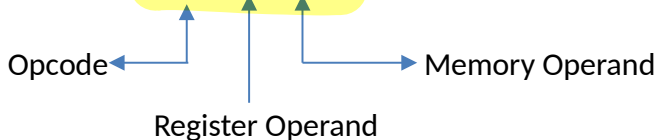
- An instruction to this effect in Assembly Language Notation:

Add P, Q, S

Assembly Language Notation

- Consider $R1 \leftarrow [R1] + [R3]$
- Equivalent Assembly Language instruction is **Add R3, R1**.
- Here the operation Code Mnemonic is **Add**
- Register Locations **R1, R3** represent *operand fields*

- Similarly, in the instruction **ADD R1, SUM**



- i.e. $SUM \leftarrow [SUM] + [R1]$

DIGITAL DESIGN AND COMPUTER ORGANIZATION

3. Basic Instruction Types T2:Ch2 2.4

Department of Computer Science and Engineering

Basic Instruction Types

- ☐ Three - Address Instruction
- ☐ Two - Address Instruction
- ☐ One - Address Instruction
- ☐ Zero - Address Instruction

➤ Three - Address Instruction

Suppose we would like to use a single machine instruction to perform the following addition operation:

$$S \leftarrow [P] + [Q]$$

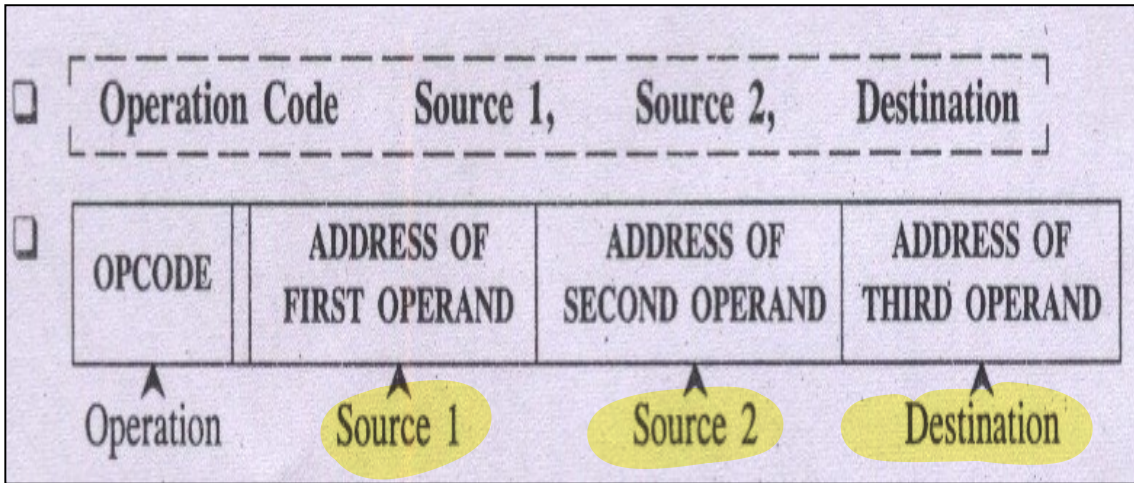
We will have to use a **three - address instruction** to carry out addition and to store the sum in a third variable **S**.

➤ Then three address instruction to perform addition is:

Add P, Q, S

Basic Instruction Types

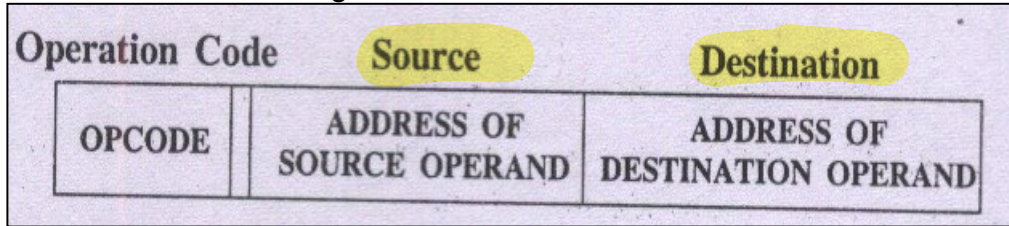
General form of three – address instruction



Basic Instruction Types

Two – Address Instruction

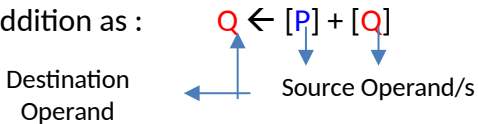
- Two – address instruction general format:



- Example of two – address instruction

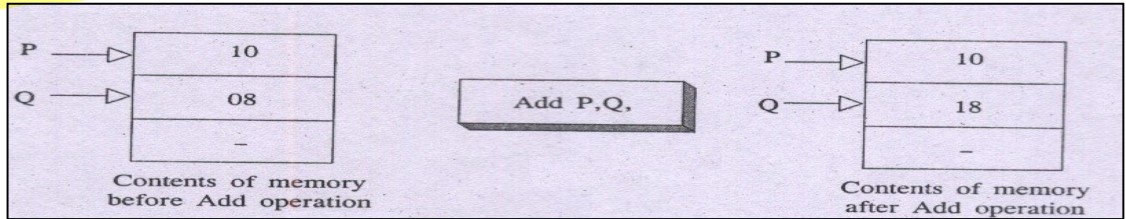
Add P, Q

- To perform operation of addition as :



Basic Instruction Types

- Here one of the operand i.e. destination operand Q acts as source as well as destination.



➤ To perform $S \leftarrow [P] + [Q]$

➤ Use two - address instructions sequence:

```
MOVE Q, S
ADD  P, S
```


Basic Instruction Types

One-Address Instruction

- One -address instruction format specify a single operand along with operation code.
- The requirement of second operand is fulfilled by an implicit processor register known as **Accumulator (AC)**.
- Example of one-address instruction:
 - It Specify:
 - **Load P** Suggest $AC \leftarrow [P]$ (Fetch P into AC)
 - **Store S** Indicate $S \leftarrow [AC]$ (Write AC into S)
 - To perform $S \leftarrow [P] + [Q]$

ADD Q

$AC \leftarrow [AC] + [Q]$

(Fetch P into AC)

(Write AC into S)

Basic Instruction Types

One-Address Instruction

- One -address instruction format specify a single operand along with operation code.
- The requirement of second operand is fulfilled by an implicit processor register known as **Accumulator (AC)**.

- Example of one-address instruction:

ADD Q

- It Specify:

$AC \leftarrow [AC] + [Q]$

- **Load P** Suggest $AC \leftarrow [P]$ (Fetch P into AC)

- **Store S** Indicate $S \leftarrow [AC]$ (Write AC into S)

- To perform $S \leftarrow [P] + [Q]$

- Use one - address instructions : **Load P;** $AC \leftarrow [P]$

Add Q; $AC \leftarrow [AC] + [Q]$

Store S; $S \leftarrow [AC]$

Basic Instruction Types

Zero – Address Instruction

- Stack – organized computer make use of a **special memory structure called push down stack to store operands**. In such computer machines it is possible to use instructions that contain only operation codes and **no explicit operands**.
- The name **Zero – address** specifies the absence of an address field of operands in machine instructions.
- Example of Zero – address instruction: **ADD**
- To perform operation of addition as **$TOS \leftarrow (P + Q)$**

Basic Instruction Types

Mov A,Ri is equivalent
Load A,Ri

and

Move Ri,A is equivalent to
Store Ri,A

Basic Instruction Types

➤ If processor supports ALU operations where one data may be in memory and other in register then the instruction sequence is for

ADD A,B,C :

In processors where arithmetic operations are allowed only on operands that are in processor registers, the $C = A + B$ task can be performed by the instruction sequence

Move A,Ri

Move B,Rj

Add Ri,Rj

Move Rj,C

In processors where one operand may be in the memory but the other must be in a register, an instruction sequence for the required task would be

Move A,Ri

Add B,Ri

Move Ri,C

DIGITAL DESIGN AND COMPUTER ORGANIZATION

4. Instruction Execution And Straight-Line Sequencing

T2:Ch2 2.4

Department of Computer Science and Engineering

Instruction Execution And Straight-Line Sequencing

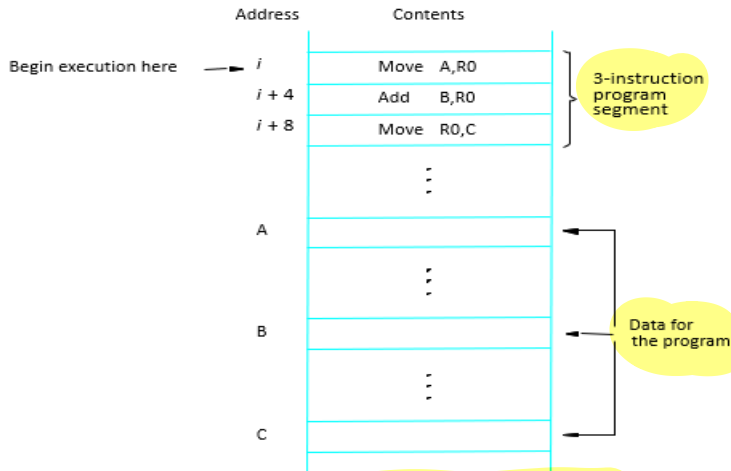


Figure 2.8. A program for $C \leftarrow [A] + [B]$.

Assumptions:

- One memory operand per instruction
- 32-bit word length
- Memory is byte addressable
- Full memory address can be directly specified in a single-word instruction

Two-phase procedure

- Instruction fetch
- Instruction execute

Instruction Execution And Straight-Line Sequencing

- **PC – Program counter:** hold the address of the next instruction to be executed
- **Straight line sequencing:** If fetching and executing of instructions is carried out one by one from successive addresses of memory, it is called straight line sequencing.
- Major two phase of instruction execution
 - ❖ **Instruction fetch phase:** Instruction is fetched from memory and is placed in instruction register IR
 - ❖ **Instruction execute phase:** Contents of IR is decoded and processor carries out the operation either by reading data from memory or registers.

Instruction Execution And Straight-Line Sequencing

i	Move	NUM1,R0
$i + 4$	Add	NUM2,R0
$i + 8$	Add	NUM3,R0
		⋮
$i + 4n - 4$	Add	NUM n ,R0
$i + 4n$	Move	R0,SUM
		⋮
SUM		
NUM1		
NUM2		
		⋮
NUM n		

Figure 2.9. A straight-line program for adding n numbers.

DIGITAL DESIGN AND COMPUTER ORGANIZATION

5. Branching T2:Ch2 2.4

Department of Computer Science and Engineering

Branching

- Branch instructions are those which change the normal sequence of execution.
- Sequence can be changed either conditionally or unconditionally.
- Accordingly we have **conditional** branch instructions and **unconditional branch instruction**.
- Conditional branch instruction changes the sequence only when certain conditions are met.
- Unconditional branch instruction changes the sequence of execution irrespective of the condition of the results.

Branching

- Consider branch instruction in a program:

Branch > 0 Loop

- Program execution continues in a straight line sequence until encountering of **Branch > 0 Loop** instruction
- Execution results in a jump to location of **Loop**.
- That is a branch instruction loads a new value in PC - the memory location address (loop) at which program control is to be transferred to start/resume execution.
- As a result processor is prohibited from fetching and executing next instruction in straight - line sequence.

Branching

Branch target

Conditional branch

Program loop

LOOP

SUM

N

NUM1

NUM2

NUM n

Move N,R1

Clear R0

Determine address of
"Next" number and add
"Next" number to R0

Decrement R1

Branch>0 LOOP

Move R0,SUM

•
•
•

•
•
•

Figure 2.10. Using a loop to add n numbers.

Check whether number Positive or negative

Move a,R0

Branch >0 Pos

Move R0,R2

Pos:

Move R0,R1

DIGITAL DESIGN AND COMPUTER ORGANIZATION

6. Condition Codes

T2:Ch2 2.4

Department of Computer Science and Engineering

Condition Codes

- The data conditions or status, after an arithmetic or logical operation are indicated by setting or clearing the flip - flops called *flags* or *condition codes*.
- Each flip-flop holding a data condition code is a one -bit storage cell (logic circuit) that can be set to a 1 or reset to 0 value.
- These condition codes are set/reset as a result of arithmetic and logical operations in the ALU.
- Condition code bits or flag bits are accommodated in a groups of 4 - bit, 8 bit or 16 - bit flag register or a status register in CPU.

Condition Codes

- Results of various instructions are stored for subsequent use by conditional instructions
- This is done by recording the required info in individual bits, called as condition code flags – grouped together in special processor register called as **condition code register or status register**

Condition Codes

➤ CONDITIONAL CODE FLAGS:

N – Negative	1 if results are Negative 0 if results are Positive
Z – Zero	1 if results are Zero 0 if results are Non zero
V – Overflow	1 if arithmetic overflow occurs 0 no overflow occurs
C – Carry	1 if carry from MSB bit 0 if there is no carry from MSB bit

Branch>0. Branch
is taken when
neither N or Z=1

Condition Codes

➤ **Pentium processor** makes use of following condition codes:

C (carry flag)

P (parity flag)

A (Auxiliary carry flag)

Z (zero flag)

S (sign flag)

O (over flow flag)

Condition Codes

List the steps needed to execute the machine instruction Add LOC, R0 in terms of transfers between memory and processor and some simple control commands. Assume that the instruction itself is stored in the memory at location INSTR and that this address is initially in register PC.

Condition Codes

- Transfer the contents of register PC to register MAR
- Issue a Read command to memory, and then wait until it has transferred the requested word into register MDR
- Transfer the instruction from MDR into IR and decode it
- Transfer the address LOCA from IR to MAR
- Issue a Read command and wait until MDR is loaded
- Transfer contents of MDR to the ALU
- Transfer contents of R0 to the ALU
- Perform addition of the two operands in the ALU and transfer result into R0
- Transfer contents of PC to ALU
- Add 1 to operand in ALU and transfer incremented address to PC

Condition Codes

Repeat the problem for machine instruction
Add R1,R2,R3

Condition Codes

- Transfer the contents of register PC to register MAR
- Issue a Read command to memory, and then wait until it has transferred the requested word into register MDR
- Transfer the instruction from MDR into IR and decode it
- Transfer contents of R1 and R2 to the ALU
- Perform addition of two operands in the ALU and transfer answer into R3
- Transfer contents of PC to ALU
- Add 1 to operand in ALU and transfer incremented address to PC

Condition Codes

Give a short sequence of machine instructions for the task: “Add the contents of memory location A to those of location B, and place the answer in location C.”

Instructions Load LOC, R_i
and Store R_i , LOC

are the only instructions available to transfer data between the memory and general purpose register R_i . Do not destroy the contents of either location A or B.

Condition Codes

Load A,R0
Load B,R1
Add R0,R1
Store R1,C

Condition Codes

Suppose that move and add instructions are available in the format
Move/Add loc1,loc2

Loci can be either memory or reg. Is it possible to use fewer instructions?
Give the seq

Condition Codes

Move B,C

Add A,C

DIGITAL DESIGN AND COMPUTER ORGANIZATION

7. Generating Memory Addresses T2:Ch2 2.4

Department of Computer Science and Engineering

Generating Memory Addresses

- The instruction block at **LOOP** needs to add a different number from a list during each iteration.
- The **Add** instruction must refer to a different memory address during each pass through the loop.
- The memory operand address cannot be directly given in a single **Add** instruction because it would require modification on each pass.
- A possible solution is to use a processor register, such as **Ri**, to hold the memory address.
- **Ri** can be initialized with the address **NUM1** before the loop and incremented by 4 after each iteration to point to the next memory address.
- This situation highlights the need for flexible ways to specify operand addresses.
- These methods are known as **addressing modes**, which vary in implementation but share common underlying concepts across different computers.

Think about it

- Write assembly code in three two and one address format for SUB A,B,C
- Write assembly code in three two and one address format for $A=B+C-D$

A processor executes instructions in **straight-line sequencing** unless:

- a) A Load instruction is encountered
- b) A Store instruction is encountered
- c) A Branch instruction is encountered
- d) A Condition Code is set

Which condition flag is set to 1 if an arithmetic operation results in a value too large for the number of bits available?

- a) Zero (Z)
- b) Carry (C)
- c) Overflow (V)
- d) Sign (N)

A processor executes instructions in **straight-line sequencing** unless:

- a) A Load instruction is encountered
- b) A Store instruction is encountered
- c) A Branch instruction is encountered
- d) A Condition Code is set

Answer: c) A Branch instruction is encountered

Explanation: Branch instructions alter the normal program counter (PC) sequence

Which condition flag is set to 1 if an arithmetic operation results in a value too large for the number of bits available?

- a) Zero (Z)
- b) Carry (C)
- c) Overflow (V)
- d) Sign (N)

Answer: c) Overflow (V)

Explanation: Overflow flag is set when the signed result exceeds representable range

- Which one of the following instruction formats uses the **accumulator (AC)** implicitly as one operand?
 - Zero-address instruction
 - One-address instruction
 - Two-address instruction
 - Three-address instruction

- Consider the sequence of instructions:

Load P

Add Q

Store S

This corresponds to which instruction format?

- Three-address
- Two-address
- One-address
- Zero-address

- Which one of the following instruction formats uses the **accumulator (AC)** implicitly as one operand?
- a) Zero-address instruction b) One-address instruction c) Two-address instruction d) Three-address instruction

Answer: b) One-address instruction

Explanation: In one-address instructions, one operand is explicitly given, while the accumulator (AC) is the implicit second operand

- Consider the sequence of instructions:

Load P

Add Q

Store S

This corresponds to which instruction format?

- a) Three-address
b) Two-address
c) One-address
d) Zero-address

Answer: b) One-address instruction



THANK YOU

Team DDCO
Department of Computer Science



DIGITAL DESIGN AND COMPUTER ORGANIZATION

Addressing Modes T2: Chapter 2: 2.5

Department of Computer Science and Engineering

Introduction(T2- section 2.5)

- Situations like **program looping**, **branching** demand convenient ways of locating memory words or memory operands.
- This convenience is being offered by different instructions (of a particular instruction set as defined by the **Microprocessor**) via various **Addressing Modes** they support.
- Suppose a given memory operand address is **too large to fit into a given instruction format** (i.e., in operand field), then it is the addressing mode (say **Register Indirect Mode**) that can resolve this problem.

Addressing Modes

Introduction

What is Addressing Mode . . . ?

The different ways in which the location of an operand is specified in an instruction are referred to as *addressing modes*

Addressing Modes

Introduction

What is Addressing Mode . . . ?



- The addressing mode specifies *a rule or method for interpreting or modifying the address field of the instruction before the operand is actually accessed for manipulation.*
- *It is concerned with the different ways in which the location of an operand can be specified in a given instruction.*
- Various schemes for specifying *addresses of operands* in an instruction have been introduced. Such schemes are collectively known as *Addressing Modes.*

Addressing Modes

Introduction

Advantages of Addressing modes :

- To be able to reference large number of memory location in main memory or for some system in *virtual memory*.
- To *reduce number of bits* in the address field of an instruction
- To extend programming convenience to the user / programmer by providing such facilities as *indexing of set of data values, counters for loop control, pointers to memory locations*

Addressing Modes

Outline

1. Immediate mode
2. Absolute (Direct) mode
3. Register mode
4. Indirect mode
5. Indexed mode
6. Relative mode
7. Auto increment mode
8. Auto decrement mode

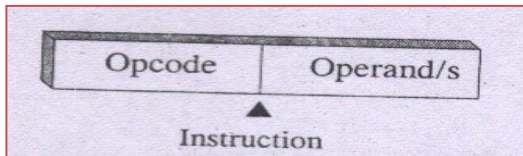
DIGITAL DESIGN AND COMPUTER ORGANIZATION

1. Immediate mode T2:Ch2 2.5

Department of Computer Science and Engineering

Immediate mode

- Here the operand is specified in the instruction itself. An instruction that follows immediate mode has an **operand field** rather than an address field.



For example:

Move 50_{immediate}, R0

	R2
	R1
50	R0

- A common convention says, a symbol # has to precede the value of an immediate operand

Move #50, R0

Immediate mode

Example

Write snippet of code for $A=B+6$

Move B,R1

Add #6,R1

Move R1,A

DIGITAL DESIGN AND COMPUTER ORGANIZATION

2. Absolute (Direct) mode

T2:Ch2 2.5

Department of Computer Science and Engineering

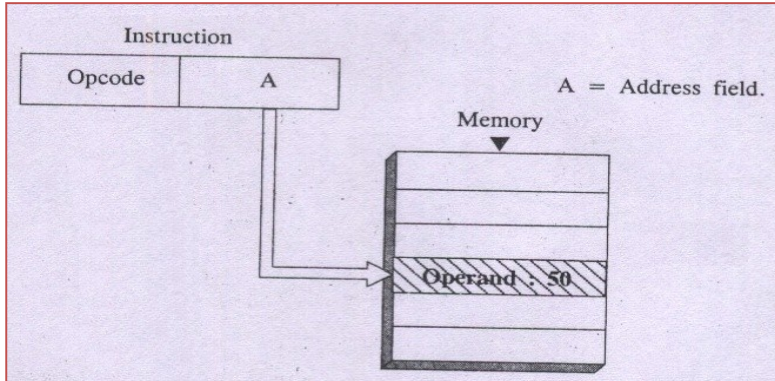
We accessed an operand by specifying the name of the register or the address of the memory location where the operand is located. The precise definitions of these two modes are:

Addressing Modes

- **Register Mode** – The operand is the contents of a processor register; the name (address) of the register is given in the instruction.
- **Absolute Mode** – The operand is in a memory location; the address of this location is given explicitly in the instruction. (In some assembly languages, this mode is called *Direct*.)

Absolute (Direct) mode

➤ Here **operand resides in Memory** and its address is given explicitly in the address field of an instruction. This scheme need only one memory reference in addition to instruction fetch cycle and no further calculation is required to compute operand address.



Absolute (Direct) mode

- Direct addressing scheme is **simple to use** and easy to implement without the requirement for additional hardware.
- However it offers only **limited memory address space**.

➤ Examples

Move LOC,**R0**

Load **P**

Store **S**

DIGITAL DESIGN AND COMPUTER ORGANIZATION

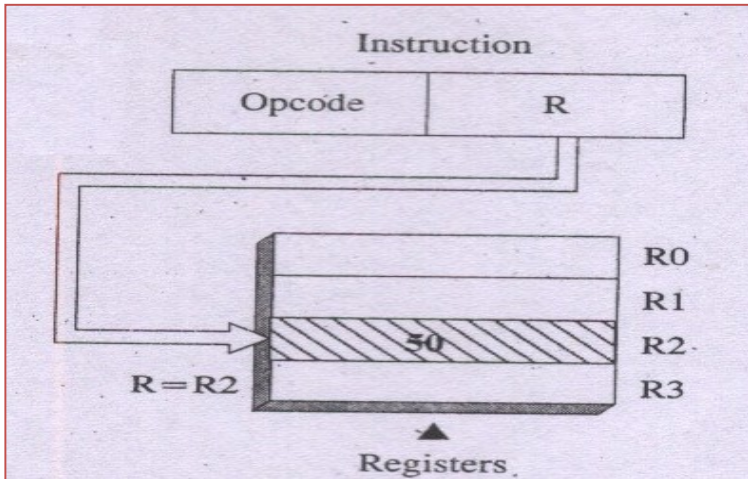
3. Register mode T2:Ch2 2.5

Department of Computer Science and Engineering

Register Addressing mode

➤ In this scheme, the operand is the contents of a register appearing in the instruction i.e $A=R$

➤ example
Add R1,R2



Register Addressing mode

Advantages of this scheme :

- ✓ No memory reference
- ✓ A few bit address to indicate register location
- ✓ Speedy execution since register is inside the processor
& has low access time.

Disadvantages of this scheme :

- ✓ Limited address space as number of registers are less
in many of the processors.

DIGITAL DESIGN AND COMPUTER ORGANIZATION

4. Indirect mode

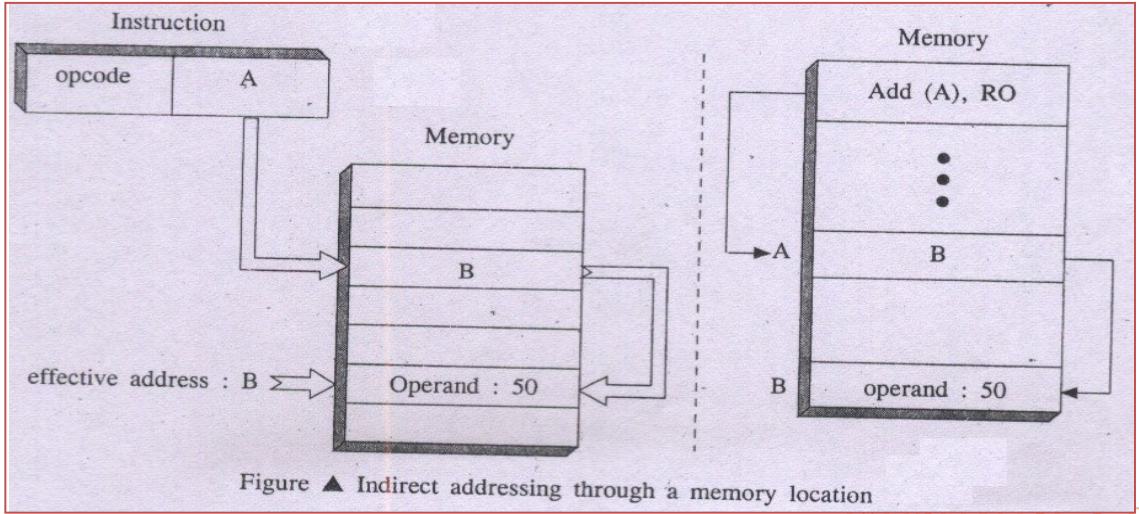
T2:Ch2 2.5

Department of Computer Science and Engineering

Indirect Addressing mode

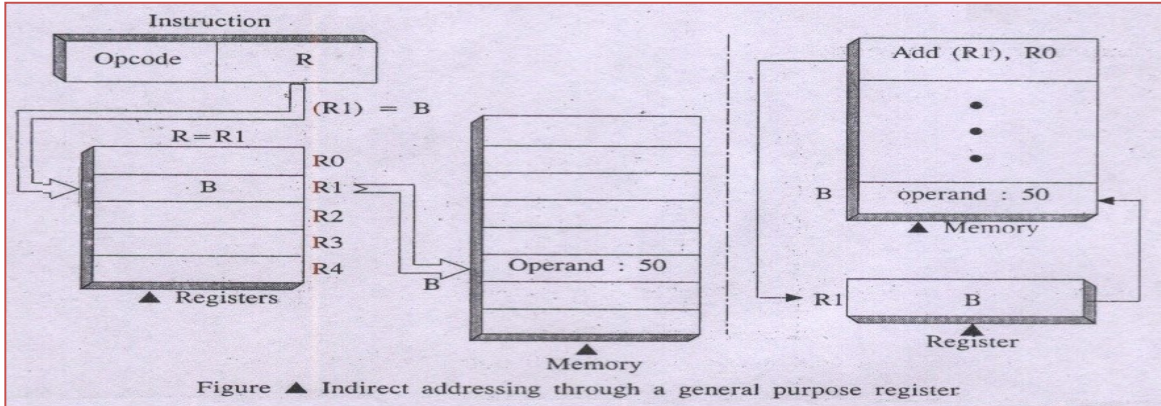
- The effective address of the operand is the contents of a register or memory location whose address appears in the instruction
- By referring to this address (EA), the required operand can be fetched from memory.
- Example: **Add (A), R0** i.e. $EA = (A)$ i.e. contents of **A** is **B**
Here **B** is effective address of desired operand in memory

Indirect Addressing mode



Register Indirect Addressing mode

- Here, instruction specifies a register in the CPU whose contents give the effective address of the operand in Memory.
- For example **Add (R1), R0** i.e. $EA = (R1)$ i.e. contents of **R1** is **B**



Indirect Addressing mode-

Program to add list of n numbers

Address	Contents
	Move N, R1
	Move #NUM, R2
	Clear R0
LOOP	Add (R2), R0
	Add #4, R2
	Decrement R1
	Branch > 0 LOOP
	Move R0, SUM

Indirect Addressing mode

The advantage of Register Indirect Addressing:

It uses one less memory reference (memory read operation)-You don't have to fetch both the operand and its address from memory separately.

➤ Address field of the instruction uses a fewer bits to specify a register -example, a register code might use 3 bits (for 8 registers) instead of 16 bits for a memory address.

➤ Register indirect addressing can be specified with

Effective Address $EA = (R)$ i.e. B

Advantages of Indirect addressing:

a wider address range to refer to a large number of memory locations.

Disadvantage of Indirect addressing:

2 or more memory references (memory read operations) required to fetch the desired operand in memory.

Indirect Addressing mode

The Register or memory location that contains address of operand is called pointer

Consider C language snippet:

`A=*B`

equivalent :

`Move B,R1`

`Move(R1),A`

Using indirect addressing:

`Move (B),A`

DIGITAL DESIGN AND COMPUTER ORGANIZATION

5. Indexed mode T2:Ch2 2.5

Department of Computer Science and Engineering

Indexed Addressing mode

- Here, effective address of the operand is generated by adding a constant value to the contents of a register.
- This constant value may be either an *offset value* called *displacement* or *beginning address* of data/operand array in main memory (**Base**).
- Indexed addressing mode is symbolically represented as **X(R)**

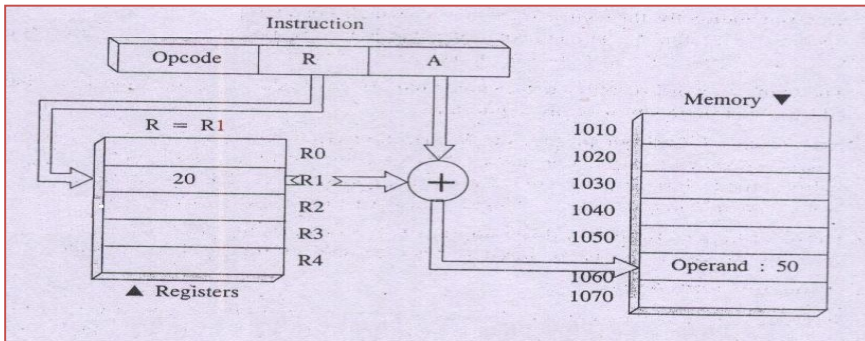
👉 Here **X** denote a constant and **R** is name of the register involved in **Indexing**.

- Effective address EA of the operand is given as

$$EA = X + [R]$$

👉 Contents of index register are not changed during the process of address generation.

Indexed Addressing mode



A = 1040 = X; constant X corresponds to a memory location

(R) = 20 i.e. Index or offset value

EA = A + (R) = 1040 + 20

EA = 1060 at which you will find desired operand 50

Indexed Addressing mode

You can use indexed addressing mode in two ways

- I) A constant value X defines here beginning address of operand in memory and index register Ri contains offset value (Displacement)

👉 For example 1040 in the following instruction:

Add 1040 (R1), R2

X=1040 R1=20 offset

- II) A constant value X defines an offset and index register Ri contains beginning address of operand in memory.

👉 For example 20 in the following instruction:

Add 20(R1), R2

X=20 R1=1040

Indexed Addressing mode

- Index addressing is fast and is excellent for manipulating data structures such as **arrays** as all you need to do is set up a base address then use the index in your code to access individual elements.
- Another advantage of indexed addressing is that if the **array is re-located in memory at any point then only the base address needs to be changed**. The code making use of the index can remain exactly the same.

Indexed Addressing mode

Example

Example: Add 20(R1),R2

Add 2000(R3),R4

Assume that a list of scores of the student beginning at location LIST as shown in the following diagram

A 4 word memory comprises a record that stores relevant info for each student

Compute the sum of all scores obtained on each of the test and store these sums in memory locations SUM1,SUM2,SUM3

Indexed Addressing mode

N 5000

LIST 5004

LIST+4 5008

LIST+8 5012

LIST+12 5016

LIST+16 5020

n
Student ID
Test1
Test2
Test3
Student ID
Test1
Test2
Test3
computer organization_ unit-1
.
.

Indexed Addressing mode

	Move		#LIST, R0
	Clear		R1
	Clear		R2
	Clear		R3
	Move		N, R4
LOOP	Add		4(R0), R1
	Add		8(R0), R2
	Add		12(R0), R3
	Add		#16, R0
	Decrement	R4	
	Branch>0	LOOP	
	Move		R1, SUM1
	Move		R2, SUM2
	Move		R3, SUM3

Indexed Addressing mode

☐ *Advantages of Index mode*

is the flexibility it offers to access relative memory locations

☐ *Disadvantages of Index mode*

- * Is the complexity of computing effective address.
- * **The instruction requires to have two address fields at least one of which is an explicit number.**

Indexed Addressing mode

- Several variations of this basic form provide a very efficient access to memory operands in practical programming situations.
- For example, a second register may be used to contain the offset X, in which case we can write the Index mode as

(Ri, Rj)

The effective address is the sum of the contents of registers Ri and Rj.

- The second register is usually called the **base register**.
- This form of indexed addressing provides more flexibility in accessing operands, **because both components of the effective address can be changed.**

Indexed Addressing mode

Another version of the Index mode uses two registers plus a constant, which can be denoted as

$$X(R_i, R_j)$$

In this case, the effective address is the sum of the constant X and the contents of registers R_i and R_j .

This added flexibility is useful in accessing multiple components inside each item in a record, where the beginning of an item is specified by the (R_i, R_j) part of the addressing mode.

DIGITAL DESIGN AND COMPUTER ORGANIZATION

6. Relative mode T2:Ch2 2.5

Department of Computer Science and Engineering

Relative mode

- This scheme supplies the relative position of the memory operand to be located.
- It's like index mode only but program counter register PC substitutes for base address contents
- Commonly used to specify the target address in branch instruction
- Relative Mode specify Effective Address by a notation:

$X(PC)$

- Effective address is $EA = [PC] + X$

- **Branch > 0 loop** Here jump value / displacement $\pm X$

DIGITAL DESIGN AND COMPUTER ORGANIZATION

7. Auto increment mode T2:Ch2 2.5

Department of Computer Science and Engineering

Auto increment mode

- The EA of the operand is the contents of a register specified in the instruction
- After accessing the operand, the contents of this register are incremented **automatically to point to the next operand** in contiguous memory locations.
- Notation for Auto Increment Mode: $(R_i) +$
- For example: **Add (R2) +, R0**
- Use of Auto Increment mode instruction eliminates the use of explicit increment instruction

Auto increment mode

Implicitly, the increment amount is 1 when the mode is given in this form. But in a byte-addressable memory, this mode would only be useful in accessing successive bytes of some list.

To access successive words in a byte-addressable memory with a 32-bit word length, the increment must be 4. Computers that have the Autoincrement mode automatically increment the contents of the register by a value that corresponds to the size of the accessed operand.

Thus, the increment is 1 for byte-sized operands, 2 for 16-bit operands, and 4 for 32-bit operands. Since the size of the operand is usually specified as part of the operation code of an instruction, it is sufficient to indicate the Autoincrement mode as $(Ri)+$.

Auto increment mode

Program to add n numbers using autoincrement

```
    MOVE N,R1
    MOVE #NUM1,R2
    CLEAR R0
LOOP  ADD (R2)+,R0
      DECREMENT R1
      BRANCH >0 LOOP
      MOVE R0,SUM
```

DIGITAL DESIGN AND COMPUTER ORGANIZATION

8. Auto decrement mode T2:Ch2 2.5

Department of Computer Science and Engineering

Auto decrement mode

- Here, content of a register specified in the instruction is
- Decrement to denote effective address of next operand in
- Successive memory location.
- Notation for Auto Decrement Mode: - (R_i)
- For instance

Add

- ($R2$), $R0$
- It allows accessing of operands in decreasing order of memory address..

Auto decrement mode

The actions performed by the Autoincrement and Autodecrement addressing modes can obviously be achieved by using two instructions, one to access the operand and the other to increment or decrement the register that contains the operand address. Combining the two operations in one instruction reduces the number of instructions

Addressing Mode

The different ways in which the location of an operand is specified in an instruction are referred to as addressing modes.

Name	Assem bler syn tax	Addressing function
Immediate	#Value	Op erand = Value
Register	R i	EA = R i
Absolute (Direct)	LOC	EA = LOC
Indirect	(R i)	EA = [R i]
	(LOC)	EA = [LOC]
Index	X(R i)	EA = [R i] + X
Base with index	(R i,R j)	EA = [R i] + [R j]
Base with index and offset	X(R i,R j)	EA = [R i] + [R j] + X
Relative	X(PC)	EA = [PC] + X
Autoincrement	(R i)+	EA = [R i] ; Increment R i
Autodecrement	-(R i)	Decremen t R i ; EA = [R i]

In **Register Addressing Mode**, which of the following is correct?

- a) Operand is always stored in memory
- b) Operand is always a constant
- c) Operand is stored in CPU register specified in instruction
- d) Operand is fetched using Program Counter (PC)

Which addressing mode is commonly used to implement **looping with counters**?

- a) Immediate
- b) Indexed
- c) Register Indirect
- d) Relative

In **Register Addressing Mode**, which of the following is correct?

- a) Operand is always stored in memory
- b) Operand is always a constant
- c) Operand is stored in CPU register specified in instruction
- d) Operand is fetched using Program Counter (PC)

Answer: c) Operand is stored in CPU register specified in instruction

Which addressing mode is commonly used to implement **looping with counters**?

- a) Immediate
- b) Indexed
- c) Register Indirect
- d) Relative

Answer: b) Indexed

Which of the following addressing modes **does NOT** require memory access for operand fetch?

- a) Register Addressing
- b) Indirect Addressing
- c) Direct Addressing
- d) Indexed Addressing

Which of the following addressing modes requires **two memory references** (excluding instruction fetch) to access an operand?

- a) Immediate
- b) Register Direct
- c) Indirect
- d) Indexed

Which of the following addressing modes **does NOT** require memory **access** for operand fetch?

- a) Register Addressing
- b) Indirect Addressing
- c) Direct Addressing
- d) Indexed Addressing

Answer: a) Register Addressing

Which of the following addressing modes requires **two memory references** (excluding instruction fetch) to access an operand?

- a) Immediate
- b) Register Direct
- c) Indirect
- d) Indexed

Answer: c) Indirect



THANK YOU

Team DDCO

Department of Computer Science



DIGITAL DESIGN AND COMPUTER ORGANIZATION

Assembly Language

Department of Computer Science and Engineering

DIGITAL DESIGN AND COMPUTER ORGANIZATION

Assembly Language T2:Ch2 2.6

Department of Computer Science and Engineering

Basic Structure of computers

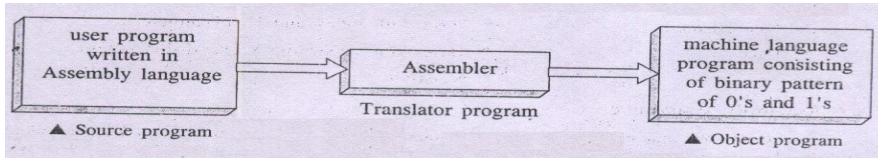
Outline

- **Assembly Languages**
 - **Assembly Language Syntax**
 - **Assembler Directives**
 - **Assembly and Execution of Programs**
 - **Number Notation**
- **I/O Operations**

Machine Instructions and Programs

Assembly Language

- A complete set of such **mnemonics**, **symbolic names** for register & memory locations and a list of rules for their use forms a programming language called an *Assembly Language*.
- The basic unit of assembly language program is a *line of code*.
- Here every line of code has symbolic code called **mnemonic opcode** and symbolic name to represent address of memory location / register as the **operand field**.
- The translator program is called an **Assembler**.



Machine Instructions and Programs

Assembly Language(T2 –section 2.6)

- Here, symbolic codes are used to represent binary pattern of machine instructions. These symbolic codes are called as *mnemonics*.
- **Mnemonics** are abbreviations that represent operation code(OP Code) of an instruction in a compact and meaningful symbolic form.
- For instance mnemonics for few operation codes include:

INC / INR- Increment

ADDI - To add immediate operand

ADD - To add

LOAD - To load operand from memory

STORE- To store operand to memory

MOVE - To transfer data from one location to another location/Register.

Machine Instructions and Programs

Assembly Language Syntax

- For example, assembly language syntax suggests that a
- move instruction to appear as follows:
 - **MOVE R0, SUM**
- The opcode mnemonic **MOVE** is followed by **at least one blank space**.
- The source operand is in register R0 (register operand).
- The destination operand is in the memory location **SUM**
- For instance symbolic name of operand memory location **SUM** suggest **absolute mode or direct addressing mode** is used in the instruction.

Machine Instructions and Programs

Assembly Language Syntax

- Sign **#** usually denotes an **immediate operand** in the instruction using **Immediate Addressing mode**
- For example: **ADD #10, R2** or **ADDI 10, R2**
- **MOVE #20, R3** or **MOVEI 10, R2**
- **Indirect addressing** is usually specified by placing **parentheses** around the name or symbol
For example **ADD (A), R0**

Format of Assembly Language Statements

LABEL	OPCODE	OPERAND(S)	; COMMENTS
-------	--------	------------	------------

- Assembler Directives are the **assembler commands** to the assembler concerning the program being assembled. These commands are neither translated into machine opcode nor assigned any memory location in the object program.
- Thus an assembly language program is said to be a complete one and acceptable to an assembler for translation and for further assignment of memory locations – if it is associated with necessary assembler directives.
- Examples of Assembler Directives:
 - **S EQU 150**
 - The assembler command **EQU** directs the assembler during translation that the symbolic name **S** must be replaced by value **150**

Machine Instructions and Programs

Assembly Language

➤ Memory arrangement for the program

```
100    Move N,R1
104    Move #NUM1,R2
108    Clear R0
112    Add (R2), R0
116    Add #4, R2
120    Decrement R1
124    Branch>0 LOOP
128    Move R0, SUM
```

Machine Instructions and Programs

Memory arrangement for the program

SUM	200	
N	204	100
NUM1	208	
NUM2	212	
		.
		.
NUMn	604	.
	608	

The machine instructions and the required data items are to be found after the program is loaded for execution. If the assembler is to produce an object program according to this arrangement, it has to know

- How to interpret the names
- Where to place the instructions in the memory
- Where to place the data operands in the memory

	100	Move	N,R1
	104	Move	#NUM1,R2
	108	Clear	R0
LOOP	112	Add	(R2),R0
	116	Add	#4,R2
	120	Decrement	R1
	124	Branch>0	LOOP
	128	Move	R0,SUM
	132		
			⋮
SUM	200		
N	204		100
NUM1	208		
NUM2	212		
			⋮
NUMn	604		

Figure 2.17 Memory arrangement for the program in Figure 2.12.

Machine Instructions and Programs

ALL representation for the program

SUM	EQU	200
	ORIGIN	204
N	DATAWORD	100
NUM1	RESERVE	400
	ORIGIN	100
START	MOVE	N,R1
	MOVE	#NUM1,R2
	CLR R0	
LOOP	ADD	(R2),R0
	ADD	#4,R2
	DEC	R1
	BGTZ	LOOP
	MOVE	R0,SUM
	RETURN	
	END	START

Machine Instructions and Programs

Assembler Directives

- **ORIGIN 204**
- Instruct assembler to place data block at main memory locations starting from 204
 - **N DATAWORD 100**
- Inform the assembler that value of N i.e. data value 100 is to be placed in the memory location 204.
 - **ORIGIN 100**
- The second ORIGIN directive states that assembler directive must load machine instructions of the object program in the main memory starting from location 100.

- **NUM1 RESERVE 400**

This directive declares that a memory block of 400 bytes is to be reserved for data, and that the name NUM1 is to be associated with address 208

Machine Instructions and Programs

Assembler Directives

RETURN

- This is an assembler directive that identifies the point at which execution of the program should be terminated.
- It causes the assembler to insert an appropriate machine instruction that returns control to the operating system of the computer.

END START

The last statement in the source program is the assembler directive END, which tells the assembler **that this is the end of the source program text.**

The END directive includes the label START, **which is the address of the location at which execution of the program is to begin.**

Machine Instructions and Programs

Assembler Directives

Label	Operation	Operand(s)	Comment
-------	-----------	------------	---------

These four *fields* are separated by an appropriate delimiter, typically one or more blank characters. The Label is an optional name associated with the memory address where the machine language instruction produced from the statement will be loaded. Labels may also be associated with addresses of data items. In Figure 2.18 there are five labels: SUM, N, NUM1, START, and LOOP.

The Operation field contains the OP-code mnemonic of the desired instruction or assembler directive. The Operand field contains addressing information for accessing one or more operands, depending on the type of instruction. The Comment field is ignored by the assembler program. It is used for documentation purposes to make the program easier to understand.

How does an assembler directive differ from a regular instruction?

- Unlike regular instructions, which are translated into machine code and executed by the CPU, **assembler directives are not converted into machine opcodes and are not assigned any memory location in the object program.** They are essentially instructions to the assembler itself, rather than to the computer's processor.

Machine Instructions and Programs

ASSEMBLY AND EXECUTION OF PROGRAMS

What steps are involved in the assembly and execution of programs, and why might these steps be crucial for efficient program operation?

Machine Instructions and Programs

ASSEMBLY AND EXECUTION OF PROGRAMS

- A source program written in an **assembly language** must be assembled into a **machine language** object program before it can be executed -performed by **assembler**.
- As the assembler scans through a source programs, it **keeps track of all names and the numerical values** that correspond to them in a **symbol table**.

Machine Instructions and Programs

ASSEMBLY AND EXECUTION OF PROGRAMS

- When a name appears a second time, it is replaced with its value from the table .
- A problem arises when a name appears as an operand before it is given a value. For example, this happens if a forward branch is required.
- A simple solution to this problem is to have the assembler scan through the source program twice.
- During the **first pass, it creates a complete symbol table**. At the end of this pass, all names will have been assigned numerical values.
- The assembler **then goes through the source program a second time and substitutes values for all names from the symbol table**. Such an assembler is called a **two-pass assembler**.

Machine Instructions and Programs

ASSEMBLY AND EXECUTION OF PROGRAMS

- The assembler stores the object program on a magnetic disk.
- The object program must be loaded into the memory of the computer before it is executed.
- This task is performed by an utility program called a **loader**.
- **Assembler can detect syntax errors**

Machine Instructions and Programs

ASSEMBLY AND EXECUTION OF PROGRAMS

- The loader must know the length of the program and the address in the memory where it will be stored.
- The assembler usually places this information in a header preceding the object code.
- Having loaded the object code, the loader starts execution of the object program by branching to the first instruction to be executed.
- The address of this instruction has been included in the assembly language program as the operand of the END assembler directive. The assembler includes this address in the header that precedes the object code on the disk.
- Errors are found in program using debugger program

Machine Instructions and Programs

NUMBER NOTATION

ADD #93, R1

or

ADD #%01011101, R1

or

ADD #\$5D, R1

DIGITAL DESIGN AND COMPUTER ORGANIZATION

I/O Operations

Department of Computer Science and Engineering

Machine Instructions and Programs

BASIC INPUT/OUTPUT OPERATIONS (T2 section 2.7)



How is the interaction between the central processing unit (CPU) and input/output (I/O) devices is managed in terms of data transfer and control?

- **Program Controlled I/O** refers to a method of input/output (I/O) operation in which the CPU is directly responsible for managing the transfer of data between peripherals (like keyboards, displays, or sensors) and memory.
- Processor and I/O requires a synchronization mechanism because of the difference in speed.

A solution to this problem is as follows: On output, the processor sends the first character and then waits for a signal from the display that the character has been received. It then sends the second character, and so on. Input is sent from the keyboard in a similar way; the processor waits for a signal indicating that a character key has been struck and that its code is available in some buffer register associated with the keyboard. Then the processor proceeds to read that code.

Machine Instructions and Programs

BASIC INPUT/OUTPUT OPERATIONS

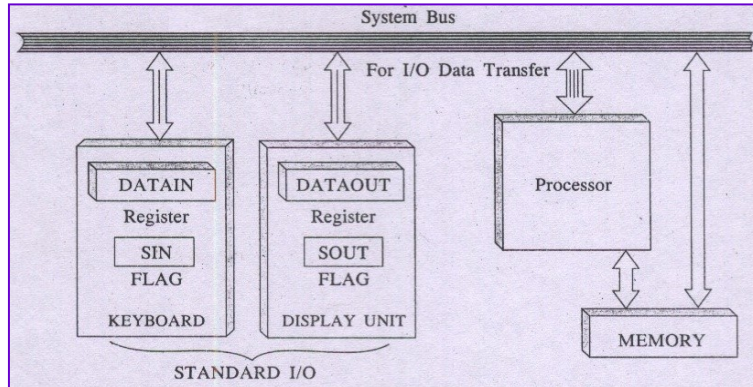
Mechanism of I/O transfer between processor and keyboard & video monitor:

- A character key when pressed from the keyboard its **scan code** is sent to an **8-bit buffer register DATAIN** in the keyboard.
- Processor is informed about a valid character data presence in **DATAIN** register by setting a synchronization **flag SIN to 1**.
- **I/O driver program** continuously monitors contents of **SIN** flag, & when **SIN** is set to **1**, it reads the contents of **DATAIN**.
- Thus **character stored in DATAIN register is transferred to processor** over a system bus and **SIN** content is automatically reset to **0**.

Machine Instructions and Programs

ASSEMBLY AND EXECUTION OF PROGRAMS

Mechanism of I/O transfer between Processor and Keyboard & Video monitor



- A similar set of events take place while **transferring a character data from processor to the display screen.**
- Here a **DATAOUT** register that holds a character's code to be displayed when synchronization control flag **SOUT** is set to **1**. **When SOUT equals 1, the display device is ready to receive a character from processor.**
- The transfer of a character to **DATAOUT** resets **SOUT** to **0**.
- I/O driver program instructions control the status of **SOUT** flag.
- 👉 The buffer registers **DATAIN**, **DATAOUT**, and control flags **SIN**, **SOUT** in this hardware setup forms parts of a connectivity circuits commonly known as **device interface** or **interface hardware**.

Machine Instructions and Programs

ASSEMBLY AND EXECUTION OF PROGRAMS

- I/O driver program instructions for I/O data transfer

READWAIT : Branch to READWAIT if SIN = 0
Input from DATAIN to R0 (if SIN = 1)

WRITEWAIT Branch to WRITEWAIT if SOUT = 0
Output from R0 to DATAOUT (if SOUT = 1)

- **SOUT** is set to **1** when display terminal is free to display next character.
- The wait loop is executed repeatedly until the control flag **SOUT** is set to **1** by the display terminal.
- Initial state of SIN is 0 and SOUT is 1

- Memory mapped i/o – **some memory addresses refer to peripheral device buffer registers, such as DATAIN, DATAOUT**
- Data can be transferred b/w the cpu and these registers using **same set of instructions and same status flags**
- Ex: MoveByte DATAIN, R0
- MoveByte R0, DATAOUT
- **Status flags SIN and SOUT can be included device status register**
- Assume that the bit b_3 of the status registers INSTATUS, OUTSTATUS corresponds to SIN and SOUT

Machine Instructions and Programs

ASSEMBLY AND EXECUTION OF PROGRAMS

Then read and write operation can be implemented as:

```
READWAIT    TestBit   #3,INSTATUS  
             Branch=0 READWAIT  
             MoveByte DATAIN,R0
```

```
WRITEWAIT   TestBit   #3,OUTSTATUS  
                     Branch=0 WRITEWAIT  
                     MoveByte R0,DATAOUT
```

Machine Instructions and Programs

ASSEMBLY AND EXECUTION OF PROGRAMS

	Move	#LOC,R0	Initialize pointer register R0 to point to the address of the first location in memory where the characters are to be stored.
READ	TestBit	#3,INSTATUS	Wait for a character to be entered in the keyboard buffer DATAIN.
	Branch=0	READ	
	MoveByte	DATAIN,(R0)	Transfer the character from DATAIN into the memory (this clears SIN to 0).
ECHO	TestBit	#3,OUTSTATUS	Wait for the display to become ready.
	Branch=0	ECHO	
	MoveByte	(R0),DATAOUT	Move the character just read to the display buffer register (this clears SOUT to 0).
	Compare	#CR,(R0)+	Check if the character just read is CR (carriage return). If it is not CR, then
	Branch≠0	READ	branch back and read another character. Also, increment the pointer to store the next character.

Program to read a line of chars and to display it

In Program Controlled I/O, which synchronization flag is set to 1 when a key is pressed on the keyboard?

- a) SOUT
- b) DATAIN
- c) SIN
- d) DATAOUT

The END directive in an Assembly program is used to:

- a) Terminate program execution
- b) Mark end of source program text
- c) Generate an unconditional jump
- d) Indicate the return address

In Program Controlled I/O, which synchronization flag is set to 1 when a key is pressed on the keyboard?

- a) SOUT
- b) DATAIN
- c) SIN
- d) DATAOUT

Answer: c) SIN

The END directive in an Assembly program is used to:

- a) Terminate program execution
- b) Mark end of source program text
- c) Generate an unconditional jump
- d) Indicate the return address

Answer: b) Mark end of source program text

Which one of the following is an **assembler directive** and not a machine instruction?

- a) MOVE
- b) ADD
- c) ORIGIN
- d) LOAD

Answer: c) ORIGIN

Which one of the following is an **assembler directive** and not a machine instruction?

- a) MOVE
- b) ADD
- c) ORIGIN
- d) LOAD

A two-pass assembler is required mainly because:

- a) It must scan the program twice to detect all errors
- b) Forward references to labels cannot be resolved in a single pass
- c) Directives must be placed before instructions
- d) Loader requires two passes for relocation

Which one of the following is an **assembler directive** and not a machine instruction?

- a) MOVE
- b) ADD
- c) ORIGIN
- d) LOAD

Ans: Origin

A two-pass assembler is required mainly because:

- a) It must scan the program twice to detect all errors
- b) Forward references to labels cannot be resolved in a single pass
- c) Directives must be placed before instructions
- d) Loader requires two passes for relocation

Answer: b) Forward references to labels cannot be resolved in a single pass



THANK YOU

Team DDCO
Department of Computer Science



DIGITAL DESIGN AND COMPUTER ORGANIZATION

Machine Instructions and Programs

Department of Computer Science and Engineering

DIGITAL DESIGN AND COMPUTER ORGANIZATION

Input/Output Organisation T2:Ch4 4.1,4.2 (except 4.2.4)

Department of Computer Science and Engineering

Basic Structure of computers

Outline

- **Input/Output Organisation**
 - **Accessing I/O Devices**
- **Interrupts**
 - **Interrupt Hardware**
 - **Enabling and Disabling Interrupts**
 - **Handling Multiple Devices**
 - **Exceptions**

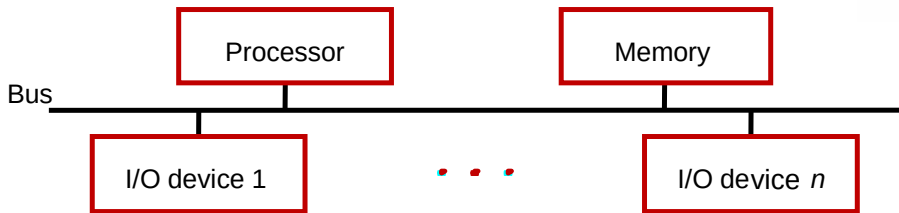
DIGITAL DESIGN AND COMPUTER

ORGANIZATION Accessing I/O Devices T2:Ch4 4.1

Department of Computer Science and Engineering

Machine Instructions and Programs

Accessing I/O Devices(Single bus arrangement)



- **Multiple I/O** devices may be connected to the **processor** and the **memory** via a bus.
- Bus consists of three sets of lines to carry **address, data and control signals**.
- Each I/O device is assigned an **unique address**.
- To access an I/O device, **the processor places the address on the address lines**.
- The device recognizes the address, and **responds to the control signals**.

Machine Instructions and Programs

Accessing I/O Devices

- ❑ Programmed I/O is a technique where the **CPU is directly responsible** for transferring data between I/O devices and memory.
- ❑ Memory-Mapped I/O Devices are accessed like memory. CPU uses MOV, STORE, LOAD instructions. Both I/O devices and memory share same address space
- ❑ I/O-Mapped I/O Devices are accessed with special I/O instructions (IN, OUT).

Machine Instructions and Programs

Accessing I/O Devices

- **I/O devices and the memory may share the same address space:**
 - Memory-mapped I/O.
 - Any machine instruction that can access memory can be used to transfer data to or from an I/O device.
 - Ex: Move DATAIN,R 1 – **reads the data from DATAIN and puts to processor register R0**
 Move R2,DATAOUT
 - Simpler software.
 - **DATAIN- Address of input buffer**
 - **DATAOUT_ output data buffer of printer/monitor/any output device**

Machine Instructions and Programs

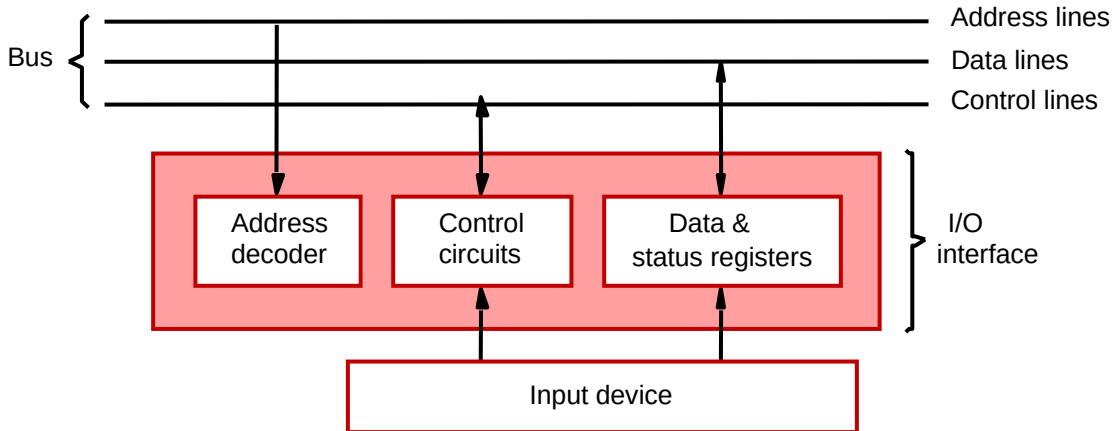
Accessing I/O Devices



- I/O devices and the memory may have different address spaces(Intel architectures)
- **Special instructions** to transfer data to and from I/O devices. (IN and OUT)
 - I/O devices may have to deal with **fewer address lines**.
 - I/O address lines need not be physically separate from memory address lines.
 - In fact, address lines may be shared between I/O devices and memory, with a **control signal** to indicate whether it is a memory address or an I/O address.

Machine Instructions and Programs

Accessing I/O Devices



Machine Instructions and Programs

Accessing I/O Devices

- I/O device is connected to the bus using an I/O interface circuit which has:
 - Address decoder, control circuit, and data and status registers.
- **Address decoder** decodes the address placed on the address lines thus enabling the device to recognize its address.
- **Data register** holds the data being transferred to or from the processor.
- **Status register** holds information necessary for the operation of the I/O device.
- Data and status registers are connected to the **data lines, and have unique addresses.**
- The Address decoder, the data & status register and the control circuitry required to coordinate I/O transfer constitute the **Device's interface circuit**

Machine Instructions and Programs

Accessing I/O Devices

- Program-controlled I/O:
- A status flag i.e SIN and SOUT is included in the interface circuit as part of the status register.
- This flag is set to 1 when a char is entered at the keyboard and set to 0 when the char is read by the processor .
- Same for SOUT.

Machine Instructions and Programs

Accessing I/O Devices

- To review the concept, let us consider an example of I/O operations involving a keyboard and a display device.
 - The 4 regs involved in the data transfer operations are : DATAIN, DATAOUT, STATUS, CONTROL
 - STATUS reg contains two control flags SIN and SOUT and two flags KIRQ, DIRQ which are used in conjunction with interrupts

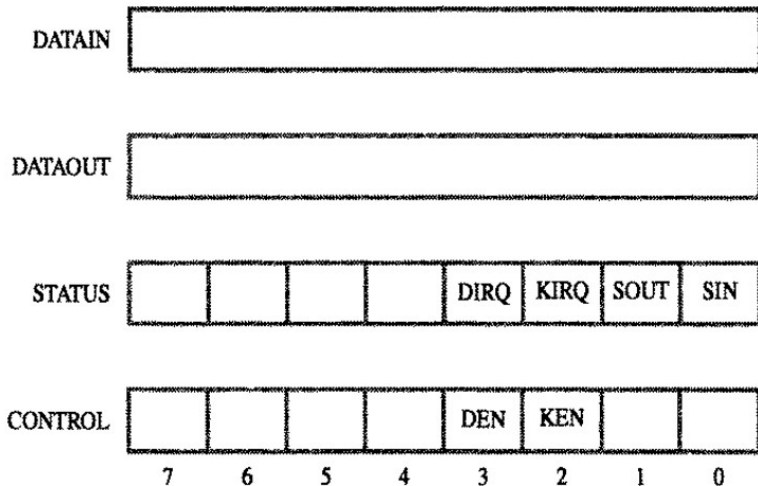


Figure 4.3 Registers in keyboard and display interfaces.

	Move	#LINE,R0	Initialize memory pointer.
WAITK	TestBit	#0,STATUS	Test SIN.
	Branch=0	WAITK	Wait for character to be entered.
	Move	DATAIN,R1	Read character.
WAITD	TestBit	#1,STATUS	Test SOUT.
	Branch=0	WAITD	Wait for display to become ready.
	Move	R1,DATAOUT	Send character to display.
	Move	R1,(R0)+	Store character and advance pointer.
	Compare	#\$0D,R1	Check if Carriage Return.
	Branch≠0	WAITK	If not, get another character.
	Move	#\$0A,DATAOUT	Otherwise, send Line Feed.
	Call	PROCESS	Call a subroutine to process the the input line.

Figure 4.4 A program that reads one line from the keyboard, stores it in memory buffer, and echoes it back to the display.

Machine Instructions and Programs

Accessing I/O Devices

- Recall that the rate of transfer to and from I/O devices is slower than the speed of the processor. This creates the need for mechanisms to **synchronize data transfers between them**.
- Program-controlled I/O:**
 - Processor **repeatedly monitors a status flag** to achieve the necessary synchronization.
 - Processor **polls** the I/O device.

Machine Instructions and Programs

Accessing I/O Devices

- Two other mechanisms used for synchronizing data transfers between the processor and memory:
 - **Interrupts.**
 - **Direct Memory Access.**

DIGITAL DESIGN AND COMPUTER

ORGANIZATION Interrupts

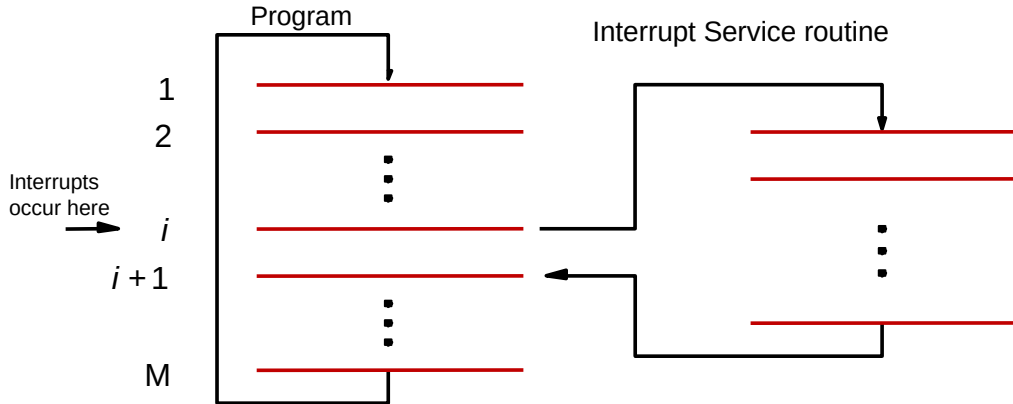
Department of Computer Science and Engineering

- In **program-controlled I/O**, when the processor continuously monitors the status of the device, it does **not perform any useful tasks**.
- An alternate approach would be for the *I/O device to alert the processor when it becomes ready*.
 - Do so by sending a hardware signal called an interrupt to the processor.
 - At least one of the bus control lines, called an **interrupt-request line** is dedicated for this purpose.
- Processor can perform other **useful tasks while it is waiting for the device to be ready**.

- Whenever the printer becomes ready, it alerts the processor by sending an interrupt-request signal. In response, the processor interrupts execution of the COMPUTE routine and transfers control to the PRINT routine.
- The PRINT routine sends the second line to the printer and is again suspended. Then the interrupted COMPUTE routine resumes execution at the point of interruption. This process continues until all n lines have been printed and the PRINT routine ends.
- The PRINT routine will be restarted whenever the next set of n lines is available for printing. If COMPUTE takes longer to generate n lines than the time required to print them, the processor will be performing useful computations all the time.

Machine Instructions and Programs

Interrupts



- Processor is executing the instruction located at address i when an interrupt occurs.
- Routine executed in response to an interrupt request is called the interrupt-service routine.
- When an interrupt occurs, control must be transferred to the interrupt service routine.

- But before transferring control, **the current contents of the PC ($i+1$), must be saved in a known location.**
- This will enable the return-from-interrupt instruction to resume execution at $i+1$.
- **Return address, or the contents of the PC are usually stored on the processor stack.**
- **Interrupt request and interrupt acknowledgement**

Machine Instructions and Programs

Comparison of Interrupts and Subroutine

Treatment of an interrupt-service routine is very similar to that of a subroutine.

However there are significant differences:

- A subroutine performs a task that is required by the calling program.
- Interrupt-service routine may not have anything in common with the program it interrupts.
- Interrupt-service routine and the program that it interrupts may belong to different users.

Interrupt latency

- Saving and restoring information can be done automatically by the processor
- Saving and restoring registers involves memory transfers:
 - Increases the total execution time.
 - Increases the delay between the time an interrupt request is received, and the start of execution of the interrupt-service routine. This delay is called interrupt latency.
- In order to reduce the interrupt latency, most processors save only the minimal amount of information:
 - This minimal amount of information includes Program Counter and processor status registers.
- Any additional information that must be saved, must be saved explicitly by the program instructions at the beginning of the interrupt service routine.

Machine Instructions and Programs

Interrupt Hardware

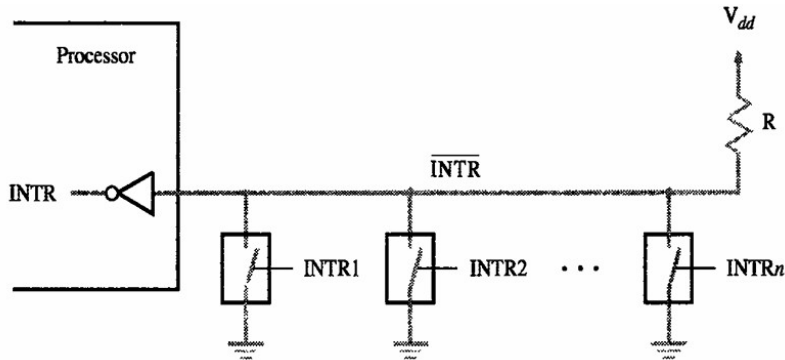


Figure 4.6 An equivalent circuit for an open-drain bus used to implement a common interrupt-request line.

Machine Instructions and Programs

Interrupt Enable & Disable

- Interrupt-requests interrupt the execution of a program, and may **alter the intended sequence of events**:
 - Sometimes **such alterations may be undesirable**, and must not be allowed.
- Processors generally provide the ability to enable and disable such interruptions as desired.
- One simple way is to provide machine instructions such as ***Interrupt-enable*** and ***Interrupt-disable*** for this purpose.
- **Edge triggered**- processor will receive only one interrupt regardless of how long line is activated

Machine Instructions and Programs

Avoidance of successive interrupts

- The interrupt request signal will be active until it learns that the processor has responded to its request. **It is essential to ensure that this active signal does not lead successive interrupts.**
- Several mechanism are available to solve this problem:
- The interrupt be disabled/enabled in the interrupt-service routine.

- The **processor automatically disable interrupts** before starting the execution of the interrupt-service routine.
- The processor has a special interrupt request line through which it receives **only one interrupt request** regardless how long the line is activated.(Edge Triggered)

The sequence of events involved in handling an interrupt request from a single device :

1. The device raises an IR
2. The processor interrupt the program currently being executed.
3. Interrupts are disabled by changing bit in PS
4. The device is informed that its request has been recognized & in response, it deactivates the IR signal
5. The action requested by the interrupt is performed by ISR
6. Interrupts are enabled & execution of the interrupted program is resumed.

Machine Instructions and Programs

Handling Multiple Devices

- **Multiple I/O** devices may be connected to the processor and the memory via a bus. Some or all of these devices may be capable of generating interrupt requests.
 - Each device operates independently, and hence no definite order can be imposed on how the devices generate interrupt requests?
- How does the processor know **which device** has generated an interrupt?
- How does the processor know **which interrupt service routine** needs to be executed?
- When the processor is executing an interrupt service routine for one device, can **other device interrupt** the processor?
- If two interrupt-requests are received **simultaneously**, then how to break the tie?

Handling Multiple Devices which device has generated an interrupt?

- When a request is received over the common interrupt-request line, **additional information** is needed to identify the particular device that activated the line.
- The additional information is **available in its status register**. KIRQ & DIRQ.
- Then ISR **polls** all the I/O devices connected to the bus.
- The first device with its **IRQ bit set** is the device that should be serviced.

Machine Instructions and Programs

Handling Multiple Devices

- An appropriate subroutine is called to provide the request service.
- **The polling scheme is easy to implement , but it spends time in interrogating the IRQ bits of all the devices that may not be requesting any service.**
- An alternative approach is to use vectored interrupts.

Machine Instructions and Programs

Vectored Interrupts which interrupt service routine

- To reduce the time involved in polling process, **a device requesting an interrupt may identify itself directly to the processor.**
- Then the processor can immediately start executing the corresponding ISR.
- A device requesting an interrupt can identify itself by sending **a special code (4 to 8 bits)** to the processor over the bus.
- The code supplied by the device may represent **the starting address of the ISR for that device.**

Machine Instructions and Programs

Vectored Interrupts

- This arrangement implies that the **ISR for a given device must always start at the same location.**
- Flexibility can be provided by storing the starting address of the ISR in the location pointed to by the interrupting device
- The processor reads this address, called the **interrupt vector** and loads it into the PC.



Interrupt source	Interrupt vector
Main program	0x0000_0000
Timer 0	0x0000_0001
Timer 1	0x0000_0002
Multiplier	0x0000_0003
Clock manager	0x0000_0004
UART 0 TX	0x0000_0008
UART 0 RX	0x0000_0009
UART 1 TX	0x0000_000A
UART 1 RX	0x0000_000B
I2C	0x0000_000B
SPI	0x0000_000C
Parallel port	0x0000_000D
External interrupt 1	0x0000_0010
External interrupt 0	0x0000_0018

- The interrupt vector may also include a new value for the processor status register.
- When a device generates an interrupt request, the processor may not **be ready to receive the interrupt_vector code immediately.**
- **Ex. It must first complete the execution of current instruction which may require the use of bus**

- **The interrupting device must wait** to put data on the bus only when the processor is ready to receive it
- When the processor is ready to receive the interrupt_vector code, it activates the interrupt_acknowledgement line, **INTA**.
- The I/O device responds by sending the interrupt_vector code and **deactivating the INTR signal**.

Machine Instructions and Programs

Interrupt Nesting(other device raise interrupt)



- The interrupts should be disabled during the execution of an ISR, to ensure that a request from one device will not cause more than one interruption.
- This may **delay** in responding to an interrupt request from **other devices**.
- For some devices, a long delay may lead to erroneous operation.

Ex. Processor Clock.

- This example suggests that I/O devices should be organized in a priority structure.
- **An interrupt request from a high priority device should be accepted while the processor is servicing another request from a lower priority device.**
- To implement this scheme , we can assign a priority level to the processor that can be changed under program control.

- The priority level of the processor is the priority of the program that is currently being executed.
- **This action disables interrupt from devices at the same level of priority or lower.**
- **However, interrupt requests from higher priority devices will continue to be accepted.**

- **The processor's priority is usually encoded in a few bits of the processor status word.**
- This can be changed by privileged instructions, which are executed only while the processor is serving in the supervisor mode.

Machine Instructions and Programs

Interrupt Nesting

- The processor is in the **supervisor mode** only when executing operating system routines.
- It switches to the **user mode** before beginning to execute application programs.

- If a user program accidentally or intentionally, changes the priority of the processor and disrupt the system operation, will lead to a special type of interrupt called a **privilege exception**.
- Implementation of interrupt priority using individual interrupt-request & acknowledgement lines is as shown

Machine Instructions and Programs

Interrupt Nesting

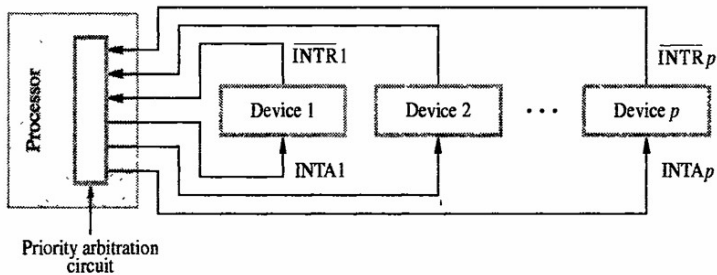


Figure 4.7 Implementation of interrupt priority using individual interrupt-request and acknowledge lines.

Machine Instructions and Programs

Simultaneous Requests

- Consider the problem of simultaneous arrivals of interrupt requests from **two or more devices**.
- The processor must have some means of deciding which request to service first.
- If all the devices have **individual interrupt-request lines then using a priority scheme**, the processor accepts the request having the highest priority.

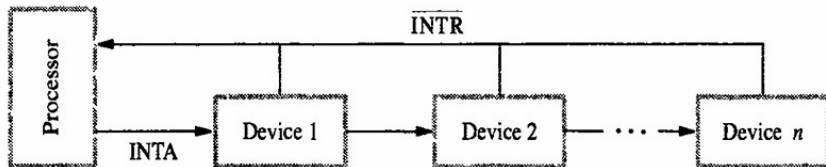
If all the devices share a common interrupt-request line, some other mechanism is needed.

1. **Polling the status registers** of the I/O devices, in which priority is determined by the order in which the devices are polled.
2. In vectored interrupts, **we must ensure that only one device is selected to read its vectored interrupt code.**

Machine Instructions and Programs

Simultaneous Requests

- For this a widely used scheme is to connect the devices to form a *daisy chain* as shown



(a) Daisy chain

Machine Instructions and Programs

Simultaneous Requests

- the interrupt request line INTR is common to all devices
- the interrupt acknowledge line INTA is connected in a daisy chain fashion , such that the INTA signal propagates serially through the devices
- When INTR is active the processor responds by setting INTA to 1

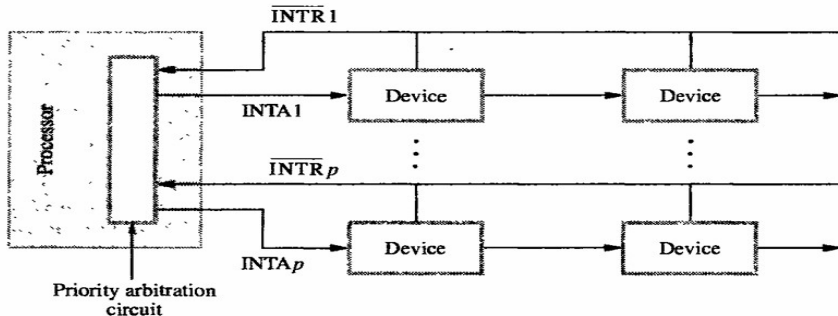
- This signal is received by device1 , and passes it to device2 only if it does not require any service.
- If the Device 1 has pending request for interrupt, it blocks the INTA signal and proceeds to put its identifying code on the data lines
- **The device closest to cpu has higher priority**

- The interrupt priority scheme is combined with daisy chain scheme as shown below
- Devices are organized in groups and each group is connected at a different priority level.
- Within a group devices are inter-connected in a daisy chain

Machine Instructions and Programs

Simultaneous Requests

- Arrangement of priority groups as shown



(b) Arrangement of priority groups

- Interrupts caused by interrupt-requests sent by I/O devices.
- Interrupts could be used in many other situations where the execution of one program needs to be suspended and execution of another program needs to be started.
- In general, **the term exception is used to refer to any event that causes an interruption.**
 - **Interrupt-requests from I/O devices is one type of an exception.**
- Other types of exceptions are:
 - Recovery from errors
 - Debugging
 - Privilege exception

- Many sources of errors in a processor. For example:
 - Error in the data stored.
 - Error during the execution of an instruction.
- When such errors are detected, exception processing is initiated.
 - Processor takes the same steps as in the case of I/O interrupt-request.
 - It suspends the execution of the current program, and starts executing an exception-service routine.

- Difference between handling I/O interrupt-request and handling exceptions due to errors:
 - In case of I/O interrupt-request, the processor usually completes the execution of an instruction in progress before branching to the interrupt-service routine.
 - In case of exception processing however, **the execution of an instruction in progress usually cannot be completed.**

Machine Instructions and Programs

Exceptions-helps programmer find errors in prog

- Debugger uses exceptions to provide important features:
 - Trace,
 - Breakpoints.
- Trace mode:
 - Exception **occurs after the execution of every instruction.**
 - Debugging program is used as the exception-service routine.
- Breakpoints(Trap/software interrupt)
 - Exception occurs only at **specific points** selected by the user.
 - Debugging program is used as the exception-service routine.

- Certain instructions can be executed only when the processor is in the **supervisor mode**. These are called **privileged instructions**.
- If an attempt is made to execute a privileged instruction in the user mode, a **privilege exception** occurs.
- Privilege exception causes:
 - Processor to switch to the supervisor mode,
 - Execution of an appropriate exception-servicing routine.

In a computer system, if the **I/O devices and memory share the same address space**, the system is said to use:

- a) Memory-mapped I/O
- b) Isolated I/O
- c) Program-controlled I/O
- d) Direct I/O

The CPU waits for a device to complete its operation by repeatedly checking its status register. This is called:

- Interrupt-driven I/O
- b) Direct Memory Access (DMA)
- c) Polling
- d) Channel I/O

In a computer system, if the **I/O devices and memory share the same address space**, the system is said to use:

- a) Memory-mapped I/O
- b) Isolated I/O
- c) Program-controlled I/O
- d) Direct I/O

Answer: a) Memory-mapped I/O

The CPU waits for a device to complete its operation by repeatedly checking its status register. This is called:

- Interrupt-driven I/O
- b) Direct Memory Access (DMA)
- c) Polling
- d) Channel I/O

Answer: c) Polling

The daisy chain priority scheme for handling multiple interrupts has the disadvantage that:

- a) It requires more hardware lines
- b) The device closest to the CPU has the lowest priority
- c) The device farthest from the CPU has the lowest priority
- d) All devices must generate interrupts simultaneously

The delay between the generation of an interrupt request and the execution of the corresponding ISR is known as

- e) Cycle time
- f) Instruction delay
- g) Interrupt latency
- h) Response time

The daisy chain priority scheme for handling multiple interrupts has the disadvantage that:

- a) It requires more hardware lines
- b) The device closest to the CPU has the lowest priority
- c) The device farthest from the CPU has the lowest priority
- d) All devices must generate interrupts simultaneously

Answer: c) The device farthest from the CPU has the lowest priority

The delay between the generation of an interrupt request and the execution of the corresponding ISR is known as

- e) Cycle time
- f) Instruction delay
- g) Interrupt latency
- h) Response time

Answer: c) Interrupt latency



THANK YOU

Team DDCO
Department of Computer Science



DIGITAL DESIGN AND COMPUTER ORGANIZATION

Direct Memory Access T2 Chapter 4: 4.4

Team DDCO

Department of Computer Science and Engineering

- **Processor–I/O Data Transfer**
- Data between processor and I/O devices is transferred using instructions like **MOVE DATAIN, R0**
- **Processor checks device readiness** before transfer:
 - By **polling** the device's status flag, or
 - Waiting for an **interrupt request** from the device.
- Each data word transfer involves **multiple instructions**, causing **high overhead**:
- - Polling the status register
 - Incrementing memory addresses
 - Keeping track of word counts
- **Interrupt-based I/O** adds extra overhead for saving/restoring:
 - Program counter** and **processor state information**
- For **large data transfers**, a **faster alternative** is used:
 - A **special control unit** (e.g., **DMA – Direct Memory Access**) handles block transfers directly between I/O device and memory.

Direct Memory Access (DMA) allows data transfer **directly between an I/O device and main memory** without continuous processor intervention.

- **DMA transfer** is managed by a special control circuit in the I/O interface — the **DMA Controller**.
- The **DMA controller** performs tasks usually handled by the processor when accessing memory.
- For each word transferred, it:
 - ❖ Provides the **memory address**.
 - ❖ Generates all **bus control signals** for data transfer.

Since data is transferred in **blocks**, the DMA controller:

- ❖ **Increments the memory address** for each successive word.
- ❖ **Keeps count** of the number of transfers completed.

- To initiate a DMA transfer, the processor sends the starting address, word count, and direction of transfer to the DMA controller.
- Once started, the DMA controller transfers the entire block of data autonomously.
- When the transfer completes, the controller raises an interrupt to inform the processor.
- During the DMA transfer, the processor can execute another program, improving system efficiency.

- The DMA controller contains registers for:
 - Starting address
 - Word count
 - Status and control flags
- The **R/W bit** determines the direction (read from memory or write to memory).
- The **Interrupt Enable (IE) bit**, when set, causes the controller to generate an interrupt after the transfer.
- The **Done flag** indicates completion, and the **IRQ bit** shows an interrupt request has been sent.

DMA Controller

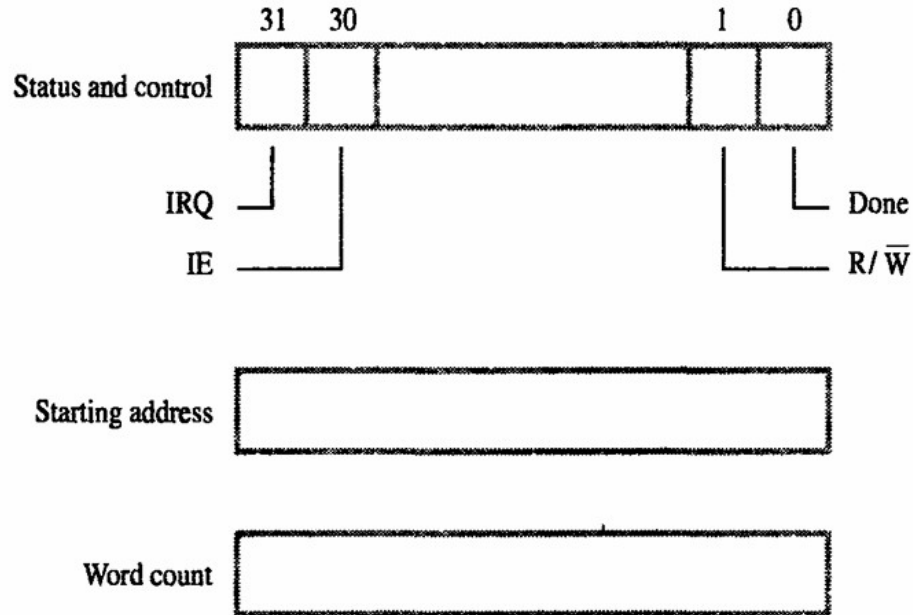


Figure 4.18 Registers in a DMA interface.

DMA Controller

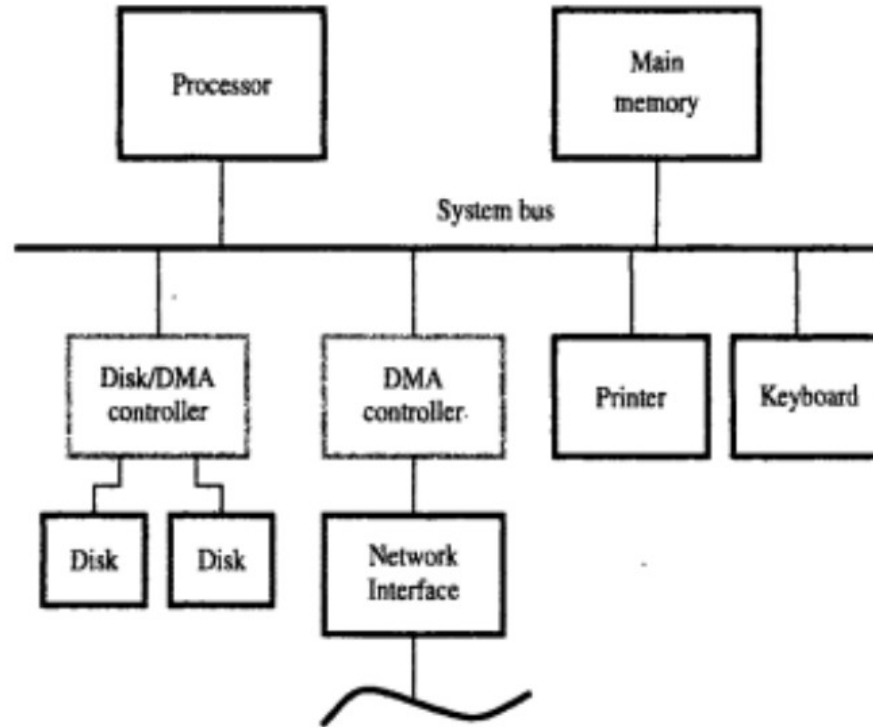


Figure 4.19 Use of DMA controllers in a computer system.

- A **DMA controller** connects high-speed devices (like disks or networks) to the computer bus.
- A **disk controller** may handle **multiple disks**, each with its own **DMA channel** — enabling **independent transfers**.
- Each channel has its own **registers** to store:
 - Memory address
 - Word count
 - Control information

Steps in DMA Operation

- **Program initialization:**

CPU writes memory address and word count into DMA channel registers.

Disk controller also receives identifying info about the data to transfer.

- **Independent operation:**

DMA controller handles data transfer autonomously, without CPU intervention.

- **Completion & status update:**

On completion, the **Done bit** in the control register is set.

If interrupts are enabled (**IE bit**), an **IRQ** signal is sent to notify the processor.

- **Status register** logs transfer success or errors.

- DMA controllers can be part of devices like disk controllers or network interfaces, each with independent DMA channels.
- Memory access by the processor and DMA controllers is interleaved, with DMA given higher priority for bus access.
- This process of sharing memory access is called *cycle stealing*.
- In **burst (block) mode**, the DMA controller gets exclusive access to memory for faster transfer.

Why do we need DMA?



The assumption about the I/O machines like keyboards, mouse, and printer etc. are genuinely very slow when compared with the central processing unit (CPU).

To overcome this issue an interrupt handler was used and the I/O machine produces all the signals that the CPU produce, then the I/O machine can bypass the information alienates to central processing unit and hence increases the speed.

Why do we need DMA?

- Cycle Stealing Process: Traditionally it is a method of accessing RAM or bus without interfering with the CPU.
- DMA allows I/O controllers to read or write RAM without CPU intervention.
- Clever exploitation of specific CPU or bus timings can permit the CPU to run at full speed without any delay if external devices access memory not actively participating in the CPU's current activity and complete the operations before any possible CPU conflict.
- Such systems are nearly dual-port RAM without the expense of high speed RAM.
- Most systems halt the CPU during the steal, essentially making it a form of DMA by another name.

- DMA **reduces processor overhead** by eliminating the need to execute multiple I/O instructions per word transferred.
- An **arbitration mechanism** ensures that bus conflicts between the processor and multiple DMA devices are resolved efficiently.

Bus Master & Bus Arbitration:

The device that controls data transfer on the bus is called the **Bus Master**. When the current master finishes, **control passes** to another device.

Bus Arbitration is the process of selecting the next bus master. Arbitration ensures **orderly and prioritized access** to the bus among devices.

Types of Bus Arbitration

- **Centralized Arbitration**

A **single bus arbiter** manages and grants bus access. Simpler control and predictable priority system.

- **Distributed Arbitration**

All devices participate in choosing the next bus master. Removes single-point dependency; supports more flexibility.

Bus Arbitration- Central Arbitration

- Bus Arbiter:**

May be the **processor** or a **separate control unit** that manages access to the system bus.

- Bus Master:**

The device currently controlling the bus (CPU or DMA controller).

- Bus Request (BR):**

DMA controller requests bus control by asserting the **BR** line.

- Bus Grant (BG):**

Processor responds with a **BG signal** → indicates permission to use the bus.

- Daisy-chain connection:**

- BG1 output of one DMA connects to BG2 input of the next

- If DMA 1 is requesting, it **blocks BG signal** to lower-priority devices.

- Ensures a **priority-based sequence** of control.

- Bus Busy (BBSY):**

When DMA gets control, it asserts **BBSY** to inform others that the bus is occupied.

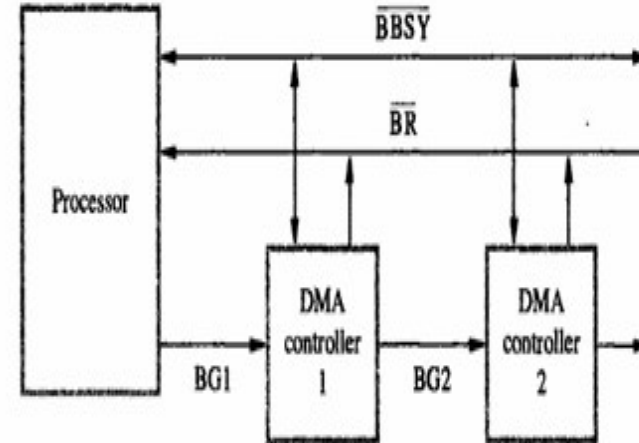


Figure 4.20 A simple arrangement for bus arbitration using a daisy chain.

Bus Arbitration- Central Arbitration

Sequence of Bus Mastership (from timing diagram):

- 1 Processor releases the bus.
- 2 DMA 2 requests (BR asserted).
- 3 Processor issues **BG**; DMA 2 waits for BBSY = 0.
- 4 DMA 2 assumes bus mastership → sets **BBSY = 1**.
- 5 Performs data transfer (cycle stealing / block mode).
- 6 After completion, DMA 2 releases bus → **BBSY = 0** → processor regains control.

•Priority Control Methods:

•Fixed Priority:

e.g., BR1 > BR2 > BR3 > BR4 (BR1 highest).

•Rotating Priority:

After a device is served, its priority moves to lowest (next order: 2, 3, 4, 1).

•Provides **flexibility** and **fairness** in servicing multiple DMA requests.

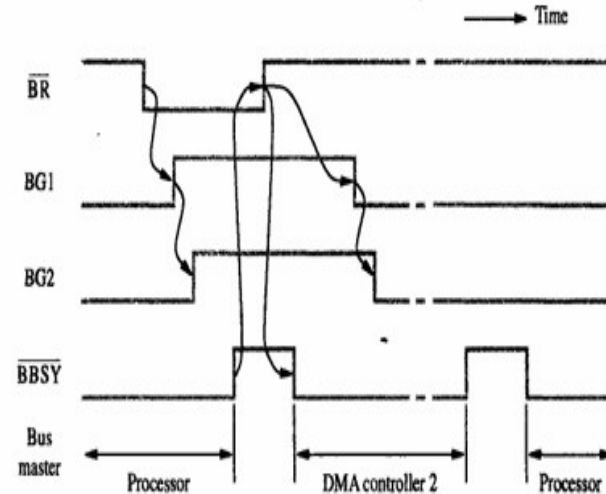


Figure 4.21 Sequence of signals during transfer of bus mastership for the devices in Figure 4.20.

Bus Arbitration- Distributed Arbitration



- Definition:**

All devices share equal responsibility for deciding who becomes the **bus master** — no central arbiter is required.

- How it works:**

- Each device has a **unique 4-bit ID number**.

- When requesting the bus, devices:

 - Assert a **Start-Arbitration** signal.

 - Place their **ID number** on 4 arbitration lines (ARB0–ARB3).

- The bus logic compares all IDs simultaneously.

- The device with the **highest ID** wins control of the bus.

- Open-Collector Lines:**

- Drivers use **open-collector (O.C.) outputs**, allowing **wired-OR logic**.

- If any device drives a '0' on a line, the bus line reads '0'.

- Ensures that **logic 1 loses to logic 0** — effectively implementing priority.

Bus Arbitration- Distributed Arbitration

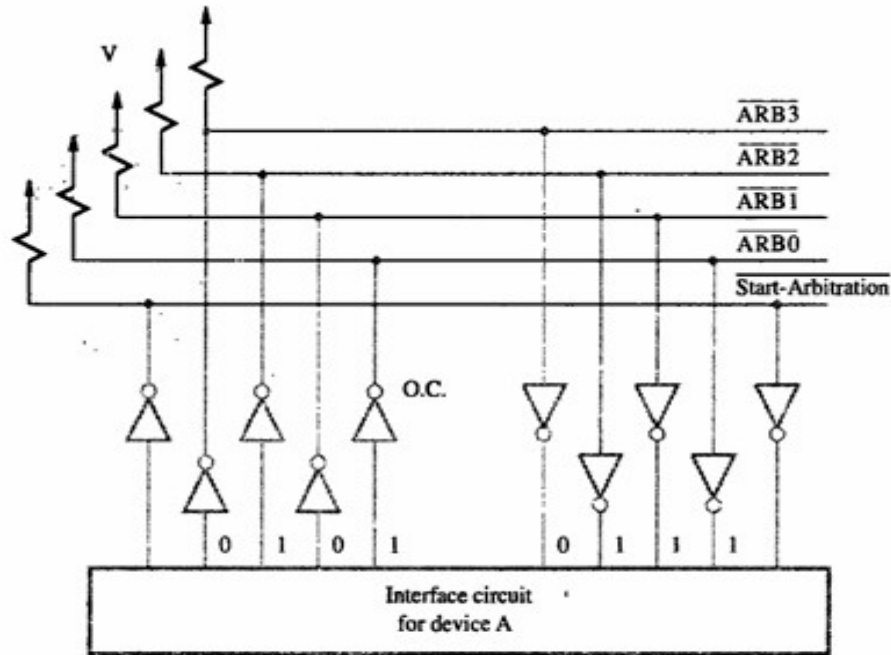


Figure 4.22 A distributed arbitration scheme.

Bus Arbitration- Distributed Arbitration



Distributed Arbitration – Example & Advantages

- **Example (Device A: ID=0101, Device B: ID=0110):**
 - Both request the bus → IDs placed on lines.
 - Combined pattern seen = 0111.
 - Devices compare bit-by-bit (MSB first):
 - At bit 1, A sees mismatch → **disables its lower bits (ARB1, ARB0).**
 - B continues → **wins arbitration** (higher ID = 6).
 - Temporary disables are re-enabled after bus control resolves.

Advantages:

- ✓ **No single-point failure** — no central arbiter needed.
- ✓ **Higher reliability** and **scalability** for multiple devices.
- ✓ Used in **SCSI bus** and other high-reliability systems.

In a DMA transfer, the role of the CPU is to:

- a) Transfer data between I/O and memory directly
- b) Provide the starting address and control to the DMA controller, then get interrupted after transfer
- c) Continuously poll the device for data readiness
- d) Perform all I/O operations itself

In a system using **cycle stealing DMA**, which statement is correct?

- a) CPU is completely halted during DMA transfer.
- b) CPU and DMA share bus cycles — DMA steals one memory cycle at a time.
- c) CPU executes only I/O instructions during DMA transfer.
- d) DMA controller uses dual-port RAM.

In a DMA transfer, the role of the CPU is to:

- a) Transfer data between I/O and memory directly
- b) Provide the starting address and control to the DMA controller, then get interrupted after transfer
- c) Continuously poll the device for data readiness
- d) Perform all I/O operations itself

Answer: (b)

Explanation: CPU initializes DMA by giving starting address, word count, and direction. After completion, the DMA controller interrupts the CPU.

In a system using **cycle stealing DMA**, which statement is correct?

- a) CPU is completely halted during DMA transfer.
- b) CPU and DMA share bus cycles — DMA steals one memory cycle at a time.
- c) CPU executes only I/O instructions during DMA transfer.
- d) DMA controller uses dual-port RAM.

Answer: (b)

Explanation: In cycle stealing, DMA “steals” one memory cycle from the CPU, allowing CPU execution in the remaining cycles.

In a **daisy chain** priority scheme used for bus arbitration, which statement is true?

- a) Highest priority device is closest to the processor.
- b) Lowest priority device is closest to the processor.
- c) All devices have equal priority.
- d) Bus grant signal propagates from lowest to highest priority device.

In **distributed arbitration**,

- a) A single bus arbiter assigns bus mastership.
- b) Each device compares its ID on arbitration lines and highest ID wins.
- c) Lowest ID device wins control.
- d) Arbitration depends on interrupt priority levels.

In a **daisy chain** priority scheme used for bus arbitration, which statement is true?

- a) Highest priority device is closest to the processor.
- b) Lowest priority device is closest to the processor.
- c) All devices have equal priority.
- d) Bus grant signal propagates from lowest to highest priority device.

Answer: (a)

Explanation: Bus grant passes sequentially through devices; the nearest device has highest priority.

In **distributed arbitration**,

- a) A single bus arbiter assigns bus mastership.
- b) Each device compares its ID on arbitration lines and highest ID wins.
- c) Lowest ID device wins control.
- d) Arbitration depends on interrupt priority levels.

Answer: (b)

Explanation: All devices participate by putting their 4-bit IDs on shared arbitration lines. Highest ID wins the bus.



THANK YOU

Team DDCO

Department of Computer Science and Engineering



DIGITAL DESIGN AND COMPUTER ORGANIZATION

Standard Input and Output

Department of Computer Science and Engineering

DIGITAL DESIGN AND COMPUTER ORGANIZATION

Standard Input and Output

Department of Computer Science and Engineering

Basic Structure of computers

Outline

- **Standard I/O Interfaces**
 - **Peripheral Component Interconnect Bus**
 - **SCSI Bus**
 - **Universal Serial Bus**

DIGITAL DESIGN AND COMPUTER

ORGANIZATION Standard I/O Interfaces T2:Ch4 4.7

Department of Computer Science and Engineering

Standard Input and Output

Standard I/O Interfaces-T2 section 4.7

- I/O device is connected to a computer using an interface circuit.
- Do we have to design a different interface for every combination of an I/O device and a computer?
- A practical approach is to develop standard interfaces and protocols.
- A personal computer has:
 - A **motherboard** which houses the processor chip, main memory and some I/O interfaces.
 - A few connectors into which additional interfaces can be plugged.
- Processor bus is defined by the signals on the processor chip.
 - Devices which require high-speed connection to the processor are connected directly to this bus.

Standard Input and Output

Standard I/O Interfaces

- Because of electrical reasons only a few devices can be connected directly to the processor bus.
- Motherboard usually provides another bus that can support more devices.
 - Processor bus and the other bus (called as **expansion bus**) are interconnected by a circuit called “**bridge**”.
 - Devices connected to the expansion bus experience a **small delay** in data transfers.
- Design of a processor bus is closely tied to the architecture of the processor.
 - No uniform standard can be defined.
- Expansion bus however can have uniform standard defined.

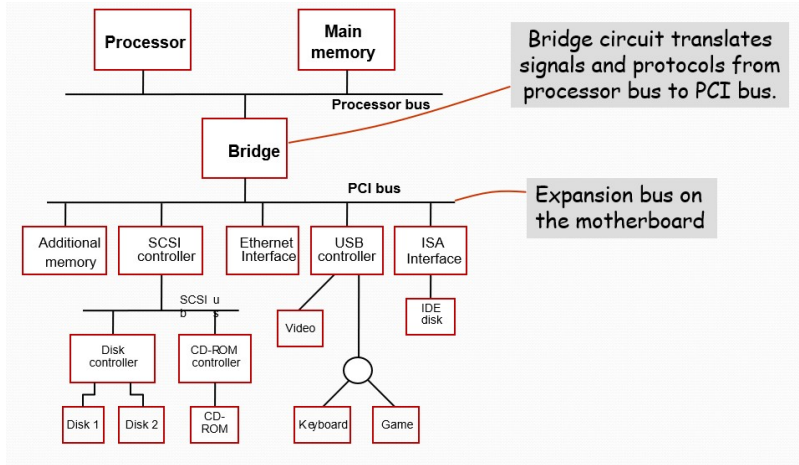
Standard Input and Output

Standard I/O Interfaces

- A number of standards have been developed for the expansion bus.
 - Some have evolved by default.
 - For example, IBM's Industry Standard Architecture.
- Three widely used bus standards:
 - PCI (Peripheral Component Interconnect)
 - SCSI (Small Computer System Interface)
 - USB (Universal Serial Bus)

Standard Input and Output

Standard I/O Interfaces



Standard Input and Output

PCI BUS

Introduction to PCI Bus

PCI (Peripheral Component Interconnect) – standardized bus system connecting I/O devices to the processor.

Enables **plug-and-play** device connection independent of CPU architecture.
Devices appear as if connected directly to processor memory space.

Supports high-speed peripherals (disk, graphics, network, etc.). Widely used due to **low cost**, **processor independence**, and **high bandwidth**.

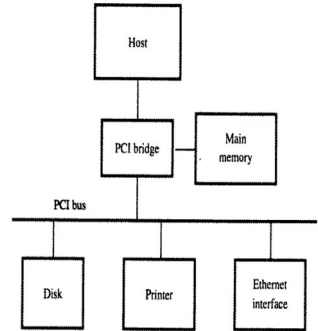


Figure 4.39 Use of a PCI bus in a computer system.

PCI Bus Features and Organization

- **Bus Components:**
- **Host / CPU** connects through a **PCI Bridge** to I/O devices.
- **Devices:** Disk, Printer, Ethernet Interface, etc.
- **Parallel Shared Bus** — supports multiple devices simultaneously.
- PCI allows **32-bit or 64-bit data width** and **33/66 MHz** clock speeds.
- **Memory-mapped addressing** used for uniform data access.
- **Bus Mastering** — any device can initiate data transfer.
- **Burst Mode Transfer:** Sends a block (burst) of words sequentially.

Standard Input and Output

PCI BUS

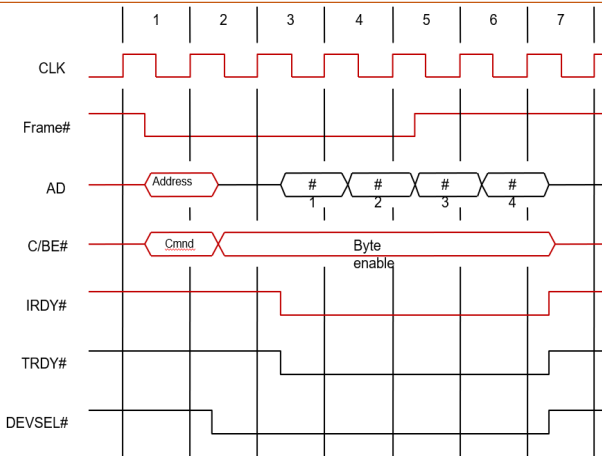
Table 4.3 Data transfer signals on the PCI bus.

Name	Function
CLK	A 33-MHz or 66-MHz clock.
FRAME#	Sent by the initiator to indicate the duration of a transaction.
AD	32 address/data lines, which may be optionally increased to 64.
C/BE#	4 command/byte-enable lines (8 for a 64-bit bus).
IRDY#, TRDY#	Initiator-ready and Target-ready signals.
DEVSEL#	A response from the device indicating that it has recognized its address and is ready for a data transfer transaction.
IDSEL#	Initialization Device Select.

- Each transfer is **synchronized by CLK** and organized into **phases** (Address → Data).
- **Initiator (Master):** CPU or DMA controller.
- **Target:** Memory or I/O device.

Standard Input and Output

PCI Bus



A read operation on the PCI bus

- A **transaction** begins when CPU asserts **FRAME#**.
- **Clock Cycle 1:**
 - Address placed on **AD** lines.
 - Command (e.g., Read) issued on **C/BE#** lines.
- **Clock Cycle 2:**
 - CPU releases AD lines for data phase.
 - Target asserts **DEVSEL#** to acknowledge.
- **Clock Cycles 3–6:**
 - Target sends data words (**#1**, **#2**, **#3**, **#4**).
 - Data transfer synchronized by **IRDY#** (**Initiator Ready**) and **TRDY#** (**Target Ready**).
- **Clock Cycle 7:**
 - CPU negates **FRAME#** (end of burst).
 - Target deasserts **DEVSEL#**; transfer completed.

Standard Input and Output

PCI Bus

Handshake Control (IRDY# / TRDY# Timing)

IRDY# (Initiator Ready): Indicates CPU ready to receive or send data.

TRDY# (Target Ready): Indicates target ready to supply or accept data.

Data transfer occurs when both signals are low (active).

If initiator or target is not ready → **wait states** are inserted automatically.

Example:

Target delays data → TRDY# held high.

Initiator waits → IRDY# high.

This **asynchronous handshaking** ensures reliable data synchronization.

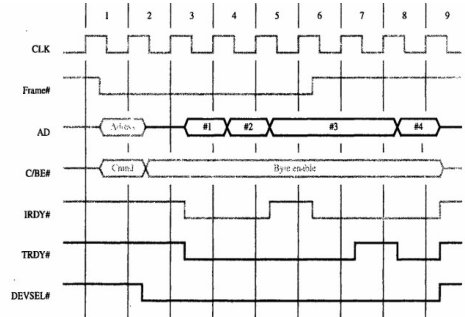


Figure 4.41 A read operation showing the role of IRDY#/TRDY#.

Standard Input and Output

PCI Bus-PCI Transaction Phases Summary

Address Phase:

Initiator drives address and command on AD + C/BE# lines.

Data Phase:

Target returns or accepts data (1 or multiple bursts).

Handshake:

IRDY# and TRDY# coordinate data readiness.

Completion:

FRAME# negated → end of burst.

DEVSEL# deasserted → bus released.

Additional Features:

Burst Transfers: Multiple data words in one transaction.

Low wire count design → cost-effective bus.

Plug-and-Play Initialization: Devices detected automatically using IDSEL line.

- The acronym SCSI stands for **Small Computer System Interface**.
- It refers to a standard bus defined by the American National Standards Institute (ANSI) under the designation X3.131 .
- In the original specifications of the standard, devices such as disks are connected to a computer via a 50-wire cable, which can be up to 25 meters in length and can transfer data at rates up to 5 megabytes/s.
- The SCSI bus standard has undergone many revisions, and **its data transfer capability has increased very rapidly, almost doubling every two years**.
- SCSI-2 and SCSI-3 have been defined, and each has several options.
- Because of various options SCSI connector may have 50, 68 or 80 pins.

- SCSI stands for Small Computer System Interface.
- Standard bus defined by ANSI (X3.131) for connecting disks, tapes, printers, etc.
- Supports multiple devices connected via 50–80 pin connectors.
- Uses DMA for high-speed data transfer between memory and I/O devices.
- Devices communicate through a SCSI controller interfacing the CPU bus.
- Evolved through multiple generations: SCSI-1, SCSI-2, SCSI-3.

Standard Input and Output

SCSI Bus

- Narrow SCSI: 8-bit data bus; Wide SCSI: 16-bit data bus.
- Data rate ranges from 5 MB/s up to 640 MB/s (anticipated for newer versions).
- Devices are connected to the SCSI bus via a controller.
- Supports up to 8 devices on a narrow bus and 16 on a wide bus.
- Each device identified by a unique SCSI ID (0–7 or 0–15).
- Data transferred in bursts using DMA.
- SCSI bus not part of CPU address space – acts as an external subsystem

Standard Input and Output

SCSI Bus

- **Two main controller roles:**
- **Initiator:** Starts communication, selects the target, and sends commands.
- **Target:** Executes commands and manages data transfer.
- The **initiator establishes a logical connection** with the target before data transfer.
- Once connected, **data transfer commands and bursts** are exchanged over the SCSI bus.
- Connections can be **suspended and restored** as needed to handle multiple operations.
- When one connection is suspended, **other devices can use the bus** for their transfers — enabling **overlapping I/O operations**.
- This overlapping of transactions is a **key feature** of SCSI that leads to **high system performance**.
- The **disk controller** usually acts as the **target**, carrying out commands issued by the initiator.
- The initiator can reestablish the suspended connection later to **resume data exchange**.

Assume that processor needs to read block of data from a disk drive and that data are stored in disk sectors that are not contiguous.

The processor sends a command to the SCSI controller, which causes the following sequence of events to take place:

1. **The SCSI controller**, acting as an **initiator**, contends for control of the bus.
2. When the initiator wins the **arbitration** process, it selects the **target controller** and hands over control of the bus to it.
3. The target starts an output operation (**from initiator to target**); in response to this, the initiator sends a command specifying the required read operation.

4. The target, realizing that it first needs to perform a disk seek operation, sends a message to the initiator indicating that it will temporarily suspend the connection between them. Then it releases the **bus**.
5. **The target controller sends a command to the disk drive to move the read head to the first sector involved in the requested read operation.** Then, it reads the data stored in that sector and stores them in a data buffer. When it is ready to begin transferring data to the initiator, the target requests control of the bus. After it wins arbitration, it reselects the initiator controller, thus restoring the suspended connection.

6. **The target transfers the contents of the data buffer to the initiator and then suspends the connection again**
7. The target controller sends a command to the disk drive to perform another seek operation. Then, it transfers the contents of the second disk sector to the initiator as before. At the end of this transfers, the logical connection between the two controllers is terminated.
8. As the initiator controller receives the data, it stores them into the main memory using the DMA approach.
9. **The SCSI controller sends as interrupt to the processor to inform it that the requested operation has been completed**

Standard Input and Output

SCSI Bus

Category	Name	Function
Data	-DB(0) to -DB(7)	Data lines: Carry one byte of information during the information transfer phase and identify device during arbitration, selection, and reselection phases.
	-DB(P)	Parity bit for the data bus.
Phase	-BSY	Busy: Asserted when the bus is not free.
	-SEL	Selection: Asserted during selection and reselection.
Information Type	-C/D	Control/Data: Asserted during transfer of control information (command, status, or message).
	-MSG	Message: Indicates that the information being transferred is a message.
Handshake	-REQ	Request: Asserted by a target to request a data transfer cycle.
	-ACK	Acknowledge: Asserted by the initiator when it has completed a data transfer operation.
Direction of Transfer	-I/O	Input/Output: Asserted to indicate an input operation (relative to the initiator).
Other	-ATN	Attention: Asserted by an initiator when it wishes to send a message to a target.
	-RST	Reset: Causes all device controls to disconnect from the bus and assume their start-up state.

Standard Input and Output

SCSI Bus

SCSI Bus Arbitration and Selection

- Bus free when $\overline{\text{BSY}}$ inactive (high voltage).
- Multiple controllers may request bus access simultaneously.
- Each controller has fixed priority (ID 7 = highest).
- Controller with highest ID wins arbitration.
- After winning, initiator asserts $\overline{\text{SEL}}$ and $\overline{\text{BSY}}$ to select target.
- Target acknowledges by asserting $\overline{\text{BSY}}$ and begins data transfer phase.

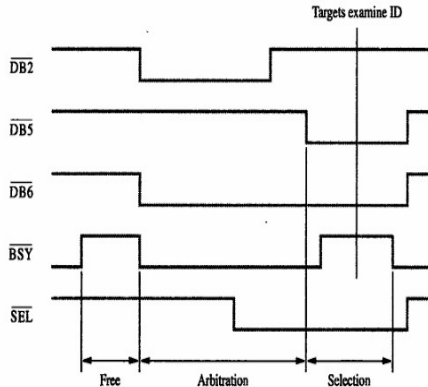


Figure 4.42 Arbitration and selection on the SCSI bus. Device 6 wins arbitration and selects device 2.

Information Transfer and Reselection

- Data transfer may include commands, status, or data packets.
- Uses -REQ and -ACK handshake similar to PCI transfer.
- Target acts as bus master during data transfer.
- After transfer, bus released for use by others.
- Reselection: Target can reconnect to initiator when ready to continue transfer.

- Universal Serial Bus (USB) is an industry standard developed through a collaborative effort of several computer and communication companies, including Compaq, Hewlett-Packard, Intel, Lucent, Microsoft, Nortel Networks, and Philips.
- Speed
 - Low-speed(1.5 Mb/s)
 - Full-speed(12 Mb/s)
 - High-speed(480 Mb/s)
- Port Limitation
- Device Characteristics
- Plug-and-play

Standard Input and Output

USB

- **Main Components:**

- **Host Controller:** Manages all communication and bus access.

- **Hub:** Expands a single port into multiple downstream ports.

- **Function (Device):** Endpoint device like keyboard, printer, camera, etc.

- Communication **always initiated by the host.**

- Devices **cannot directly** communicate with each other.

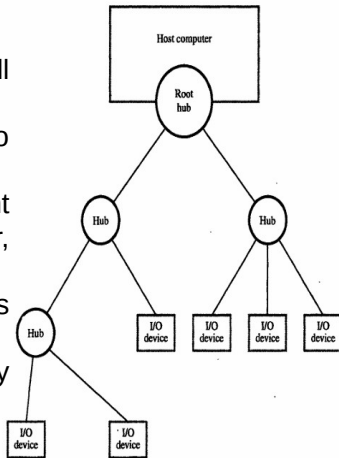


Figure 4.43 Universal Serial Bus tree structure.

- **USB = Universal Serial Bus.**

- Developed to simplify connection between computers and peripherals.

- Supports **hot-plugging** and **automatic device configuration.**

- Uses **tiered star topology** – a host connected to multiple devices via hubs.

- Allows **up to 127 devices** on a single bus.

Standard Input and Output

USB

- To accommodate a large number of devices that can be added or removed at any time, the **USB has the tree structure** as shown in the figure.
- **Each node of the tree has a device called a hub**, which acts as an intermediate control point between the host and the I/O devices. At the root of the tree, a root hub connects the entire tree to the host computer. **The leaves of the tree are the I/O devices** being served (for example, keyboard, Internet connection, speaker, or digital TV)
- In normal operation, **a hub copies a message that it receives from its upstream connection to all its downstream ports**. As a result, a message sent by the **host computer** is broadcast to all **I/O devices**, but only the addressed device will respond to that message. **However, a message from an I/O device is sent only upstream towards the root of the tree and is not seen by other devices**. Hence, the **USB enables the host to communicate with the I/O devices**, but it does not enable these devices to communicate with each other.

- When a USB is connected to a host computer, its root hub is attached to the processor bus, where it appears as a single device. The host software communicates with individual devices attached to the USB by sending packets of information, which the root hub forwards to the appropriate device in the USB tree.
- **Each device on the USB, whether it is a hub or an I/O device, is assigned a 7-bit address. This address is local to the USB tree and is not related in any way to the addresses used on the processor bus.**

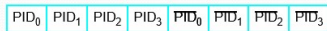
- A hub may have any number of devices or other hubs connected to it, and **addresses are assigned arbitrarily. When a device is first connected to a hub, or when it is powered on, it has the address 0.** The hardware of the hub to which this device is connected is capable of detecting that the device has been connected, and it records this fact as part of its own status information. Periodically, the **host polls each hub to collect status information and learn about new devices that may have been added or disconnected.**
- **When the host is informed that a new device has been connected, it uses a sequence of commands to send a reset signal on the corresponding hub port, read information from the device about its capabilities, send configuration information to the device, and assign the device a unique USB address.** Once this sequence is completed the device begins normal operation and responds only to the new address.

- All information transferred over the USB is organized in **packets**, where a packet consists of one or more bytes of information. There are many types of packets that perform a variety of control functions.
- The information transferred on the USB can be divided into two broad categories: **control and data**.
- Control packets perform such tasks as addressing a device to initiate data transfer, acknowledging that data have been received correctly, or indicating an error.
- Data packets carry information that is delivered to a device.

- A packet consists of one or more fields containing different kinds of information. **The first field of any packet is called the packet identifier, PID, which identifies the type of that packet.**
- **They are transmitted twice. The first time they are sent with their true values, and the second time with each bit complemented**
- The four PID bits identify one of 16 different packet types. Some control packets, such as ACK (Acknowledge), consist only of the PID byte.

Standard Input and Output

USB Protocols



(a) Packet identifier field



(b) Token packet, IN or OUT

Control packets used for controlling data transfer operations are called token packets.



(c) Data packet

Figure 45. USB packet format.

- Data transfer occurs through packets:
- Token packets: Identify type and direction of transfer.
- Data packets: Contain payload data (Data0, Data1).
- Handshake packets: ACK, NAK, STALL signals for flow control.
- Frame-based communication: Each frame = 1 ms (for full-speed).
- Endpoints: Logical channels in each device for communication.

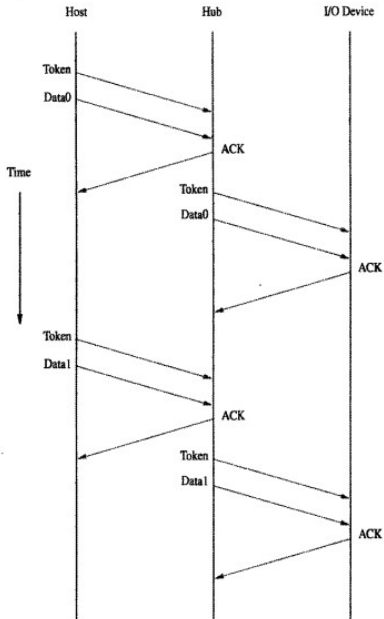


Figure 4.46 An output transfer.

•**Token Packet:**

•The host begins by sending a *Token* to indicate the start of an output transaction.

•**Data Packet (Data0 / Data1):**

•The host sends the data packet immediately after the token.

•*Data0* and *Data1* alternate to ensure data integrity and synchronization.

•**Acknowledgement (ACK):**

•The I/O device sends an ACK signal back through the hub to confirm correct reception of data.

•**Repeat Cycle:**

•The process repeats for subsequent data packets (e.g., another *Token* → *Data1* → *ACK* cycle).

•**Direction of Communication:**

•Always **from Host** → **Device** (for output transfer).

•**Timing:**

•Events progress vertically in time, ensuring synchronized packet exchange.

Standard Input and Output

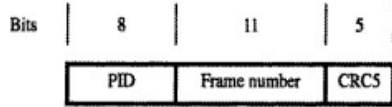
Isochronous Traffic on USB

- One of the key objectives of the USB is to support the transfer of isochronous data.
- Devices that generates or receives isochronous **data require a time reference to control the sampling process.**
- To provide this reference. **Transmission over the USB is divided into frames of equal length.**
- A frame is 1ms long for low-and full-speed data.
- The root hub generates a **Start of Frame control packet (SOF) precisely once every 1 ms to mark the beginning of a new frame.**

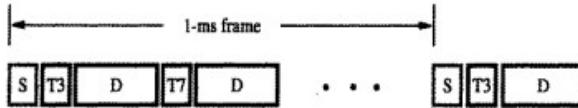
Standard Input and Output

Inochronous Traffic on USB

- The **arrival of an SOF packet at any device constitutes a regular clock signal** that the device can use for its own purposes.
- To assist devices that may need longer periods of time, the SOF packet carries **an 11-bit frame number**.
- Following each SOF packet, the host carries out **input and output transfers** for isochronous devices.
- This means that each device will have an opportunity for an input or output transfer once every 1 ms.



(a) SOF Packet



S — Start-of-frame packet

T_n — Token packet, address = n

D — Data packet

A — ACK packet

(b) Frame example

Figure 4.47 USB frames.

Standard Input and Output

Electrical Characteristics of USB

- The cables used for USB connections consist of four wires.
- Two are used to carry power, +5V and Ground.
- Thus, a hub or an I/O device may be powered directly from the bus, or it may have its own external power connection.
- The other two wires are used to carry data.
- Different signaling schemes are used for different speeds of transmission.
- At low speed, 1s and 0s are transmitted by sending a high voltage state (5V) on one or the other of the two signal wires. For high-speed links, differential transmission is used.

Standard Input and Output

MCQ

In a **PCI transaction**, data transfer takes place when:

- a) Both IRDY# and TRDY# signals are low.
- b) Either IRDY# or TRDY# is high.
- c) FRAME# and DEVSEL# are high.
- d) DEVSEL# is low and TRDY# is high.

Which of the following represents the correct sequence of phases during a PCI bus read operation?

- a) Data → Address → Handshake → Completion
- b) Address → Data → Handshake → Completion
- c) Handshake → Address → Data → Completion
- d) Address → Handshake → Data → Completion

In a **PCI transaction**, data transfer takes place when:

- a) Both IRDY# and TRDY# signals are low.
- b) Either IRDY# or TRDY# is high.
- c) FRAME# and DEVSEL# are high.
- d) DEVSEL# is low and TRDY# is high.

Answer: (a)

Explanation: Data is transferred only when both Initiator Ready (IRDY#) and Target Ready (TRDY#) are asserted low — ensuring handshake completion

Which of the following represents the correct sequence of phases during a PCI bus read operation?

- a) Data → Address → Handshake → Completion
- b) Address → Data → Handshake → Completion
- c) Handshake → Address → Data → Completion
- d) Address → Handshake → Data → Completion

Answer: (b)

Explanation: Address phase comes first, followed by one or more data phases controlled by IRDY#/TRDY#, then handshake and bus release (FRAME# deasserted)

Standard Input and Output

MCQ

In USB communication:

- a) Devices can communicate directly with each other.
- b) Communication is always initiated by the host.
- c) Hubs can transfer data without host intervention.
- d) Devices use interrupt lines to alert the host.

SCSI supports high-speed transfers by:

- a) Using interrupts for each byte.
- b) Direct memory access (DMA) block transfers.
- c) Polling I/O ports continuously.
- d) Bit-serial CPU register copy.

In USB communication:

- a) Devices can communicate directly with each other.
- b) Communication is always initiated by the host.
- c) Hubs can transfer data without host intervention.
- d) Devices use interrupt lines to alert the host.

Answer: (b)

Explanation: USB uses a **host-controlled polling** mechanism — all transactions are host-initiated

SCSI supports high-speed transfers by:

- a) Using interrupts for each byte.
- b) Direct memory access (DMA) block transfers.
- c) Polling I/O ports continuously.
- d) Bit-serial CPU register copy.

Answer: (b)

Explanation: SCSI bus uses DMA for direct block transfers between I/O and memory without CPU intervention



THANK YOU

Team DDCO
Department of Computer Science



DIGITAL DESIGN AND COMPUTER ORGANIZATION

Semiconductor RAM Memories

T2 Chapter 5: 5.1 and 5.2

Team DDCO

Department of Computer Science and Engineering

Introduction(T2- section 5.1 and 5.2)



Role of Memory

- Programs and data are stored in the **main memory** of the computer.
- Execution speed depends on **how fast data and instructions** are transferred between processor and memory.
- Larger memory is needed for executing complex programs that process huge data sets.

The Memory Design Challenge

- Ideally, memory should be **fast, large, and inexpensive**.
- However, achieving all three simultaneously is **impossible** due to cost and technology limitations.
- Increasing memory **speed or size** results in **higher cost**.
- Designers use **hierarchical structures** to balance speed, capacity, and cost.

Some Basic Concepts (T2- section 5.1 and 5.2)



- The **maximum size of memory** depends on the **addressing scheme**.
 - A computer with k -bit addresses can access 2^k **memory locations**.
 - Example:
 - 16-bit → 64 K locations
 - 32-bit → 4 G locations
 - 40-bit → 1 T locations
 - This defines the **address space** of the computer.
 - Each address corresponds to one **memory location** (typically 1 byte).
-
- Most modern systems are **byte-addressable** — every byte has its own address.
 - Big-endian**: Most significant byte stored at the lowest address (e.g., Motorola 68000).
 - Little-endian**: Least significant byte stored at the lowest address (e.g., Intel x86).
 - ARM** processors can operate in either mode.
 - Endianness affects data interpretation but **not memory capacity**.

Some Basic Concepts(T2- section 5.1 and 5.2)

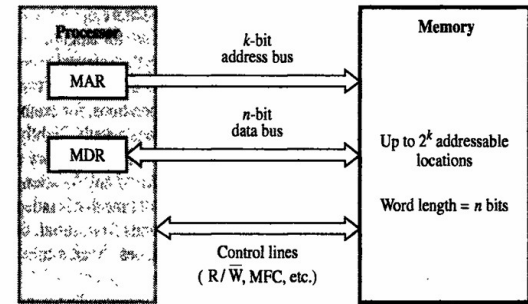
Memory stores and retrieves data in **word-length units**.

A computer's **word length** = number of bits transferred in one access.

In a 32-bit system, the address bus (high 30 bits) selects the word; the low 2 bits identify the byte.

Read Operation: Other bytes of the same word may be fetched but ignored by CPU.

Write Operation: Control logic ensures only the targeted bytes are updated.



• **Figure 5.1** Connection of the memory to the processor.

- **MAR (Memory Address Register):** Holds k-bit address sent to memory.
- **MDR (Memory Data Register):** Holds n-bit data to/from memory.
- **Address Bus:** k bits → selects one of 2^k locations.
- **Data Bus:** n bits → transfers word data.
- **Control Lines:** Read/Write, Memory-Function-Complete (MFC), etc.

Some Basic Concepts(T2- section 5.1 and 5.2)



- The **memory unit** is viewed as a **black box** that communicates with the processor.
- Data transfer between memory and processor is handled through two key registers:
- **MAR (Memory Address Register)** – holds the memory location address.
- **MDR (Memory Data Register)** – holds the data being read or written.
- If MAR is k bits long, the memory can have **2^k addressable locations**.
- If MDR is n bits long, **n bits** of data are transferred in each memory cycle.

The **processor bus** has:

- **k address lines** (from MAR)
- **n data lines** (to/from MDR)
- **Control lines** for operations:
 - **R/W** → Read (1) or Write (0)
 - **MFC (Memory Function Complete)** → indicates operation completion
- Other control lines may indicate the number of bytes to transfer.
- Communication is **synchronized** using these control signals.

Some Basic Concepts(T2- section 5.1 and 5.2)



- **Read Operation:**

Processor loads the address into **MAR**.

Sets **R/W = 1** (read).

Memory places data on data lines and asserts **MFC**.

Processor transfers data into **MDR**.

- **Write Operation:**

Processor loads address into **MAR** and data into **MDR**.

Sets **R/W = 0** (write).

Memory stores MDR contents into the addressed location.

- **Block Transfer:**

- Used for consecutive memory addresses.
- Only the first address is sent; remaining data are transferred sequentially.

- **Synchronization:**

- Transfers may be **clocked** (synchronous) or **signal-controlled** (asynchronous).
- Bus signaling schemes manage timing and coordination.
- Read and write transfers correspond to **input and output bus operations**, respectively.

Some Basic Concepts(T2- section 5.1 and 5.2)



- **Memory Access Time:** Time between the start and completion of a read/write (e.g., Read → MFC).
- **Memory Cycle Time:** Minimum time between two memory operations; slightly longer than access time.
- **Random-Access Memory (RAM):** Any location can be accessed in a fixed time, unlike serial memories (e.g., tapes or disks).
- Implemented using **semiconductor ICs** for fast, reliable access.
- The processor executes faster than memory → **speed mismatch bottleneck**.
- Solution → **Memory Hierarchy:** Use multiple levels (cache + main memory + secondary storage) to balance speed, size, and cost.

Some Basic Concepts(T2- section 5.1 and 5.2)



Cache Memory:

Small, high-speed buffer between CPU and main memory.

Holds frequently used data/instructions.

Reduces average access time; exploits **temporal & spatial locality**.

Virtual Memory:

Expands apparent size of physical RAM.

Uses **logical addresses** mapped to **physical addresses** by the **Memory Management Unit (MMU)**.

Only active portions of a program kept in RAM; inactive pages stored on disk.

Page replacement dynamically loads required pages, maintaining program continuity.

Summary: Cache enhances speed; Virtual Memory expands capacity → together they provide the illusion of large, fast, and cost-efficient memory.

Semiconductor RAM Memories

(T2- section 5.2)



- Semiconductor memories are available in a **wide range of speeds**, with **cycle times ranging from 100 ns to less than 10 ns**.
- When first introduced in the **late 1960s**, they were **more expensive** than the magnetic-core memories they replaced.
- Due to **rapid advances in Very Large Scale Integration (VLSI)** technology, their cost has **dropped dramatically**.
- As a result, **semiconductor memories are now used almost exclusively** in implementing main memories.

Internal Organization of Memory Chips

- Memory cells are usually organized in the form of an array, where each cell stores one bit of information.
- Each row of cells constitutes a memory word, and all cells in that row are connected to a common word line driven by the address decoder.
- The cells in each column are connected to a Sense/Write circuit by two bit lines.
- The Sense/Write circuits are connected to the data input/output lines of the chip.
- The **address decoder** activates one *word line* ($W_0 - W_{15}$) based on the input address ($A_0 - A_3$).

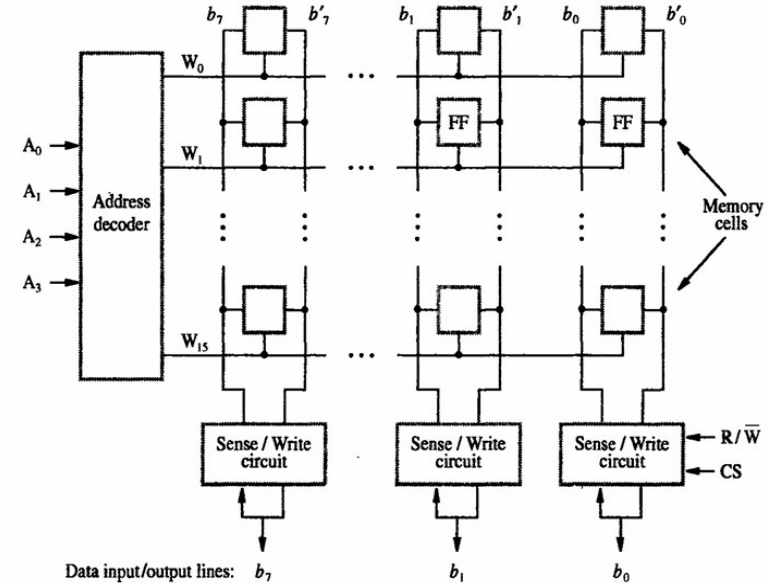


Figure 5.2 Organization of bit cells in a memory chip.

Read and Write Operations

- During a Read operation, the Sense/Write circuits sense or read the information stored in the selected cells and transmit it to the output data lines.
- During a Write operation, the Sense/Write circuits receive input information and store it in the selected cells.
- The data input and data output of each Sense/Write circuit are connected to a single bidirectional data line linked to the system data bus.

Example – 16×8 Memory Organization



- A small memory chip example consists of 16 words of 8 bits each, referred to as a 16×8 organization.
- Two control lines, R/W (Read/Write) and CS (Chip Select), are provided in addition to address and data lines.
- The R/W input specifies the required operation, while the CS input selects a given chip in a multichip memory system.
- This memory circuit stores 128 bits and requires external connections for address, data, and control lines.

Larger Memory Org. (1K Memory Ex.)



- A larger chip (e.g., **1 K × 1 = 1024 bits**) uses a **10-bit address**.
- The 10-bit address is divided into two 5-bit parts:
- **Row Address** → selects 1 of 32 rows via a decoder.
- **Column Address** → selects 1 bit of the 32 via a multiplexer/demultiplexer.
- The **32 × 32 cell array** organization provides parallel access within a row.
- Uses a **Sense/Write circuit**, **32-to-1 output multiplexer**, and **input demultiplexer** to manage data flow.

Larger Memory Org. (1K Memory Ex.)

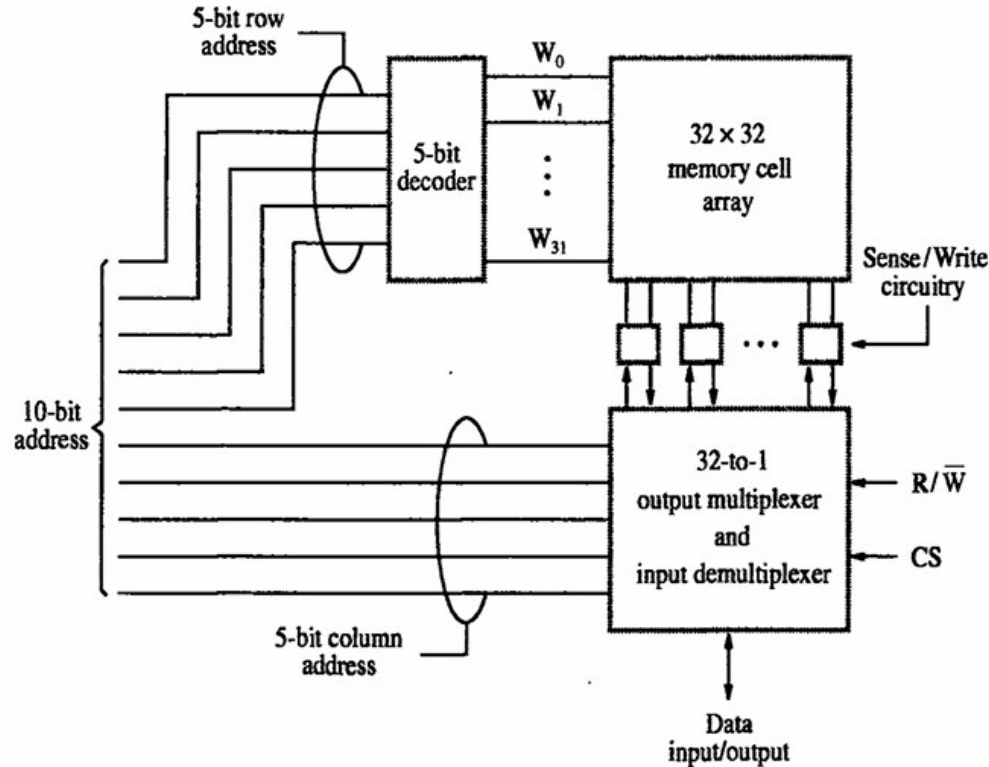


Figure 5.3 Organization of a $1K \times 1$ memory chip.

- Semiconductor memories are classified into **Static RAM (SRAM)** and **Dynamic RAM (DRAM)**.
- Both store information in binary form but differ in **circuit structure, speed, cost, and refresh requirements**.
- SRAM and DRAM form the **primary storage components** of computer systems.
- **SRAM** → used for *cache memory*; **DRAM** → used for *main memory*.

Static Memories

- **Static RAM (SRAM)**
- Each cell is made of a **bistable latch (flip-flop)** that stores one bit.
- **No refresh** is required as data is maintained as long as power is supplied.
- **Read operation:** Word line activates cell, output drives bit lines.
- **Write operation:** Data from bit lines overwrites stored latch value.
- **Advantages:** Very fast access time, stable, simple operation.
- **Disadvantages:** Large cell area, low density, expensive.
- **Applications:** CPU cache, register files, buffering circuits.

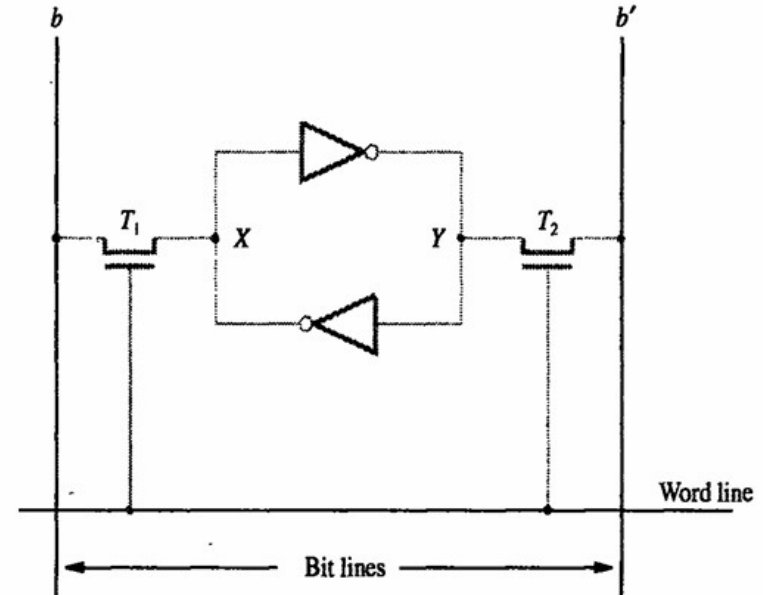


Figure 5.4 A static RAM cell.

Static Memories

Dynamic RAM (DRAM)

Each memory cell uses a **capacitor** to store charge (1 bit) and a **transistor** to control access.

Charge leaks over time → requires **periodic refresh cycles**.

Read operation: Word line enables transistor; charge on capacitor sensed on bit line, then rewritten.

Write operation: Bit line drives charge into capacitor.

Advantages: High density, low cost per bit.

Disadvantages: Slower access, refresh overhead, destructive read.

Applications: Main memory in PCs, laptops, and servers.

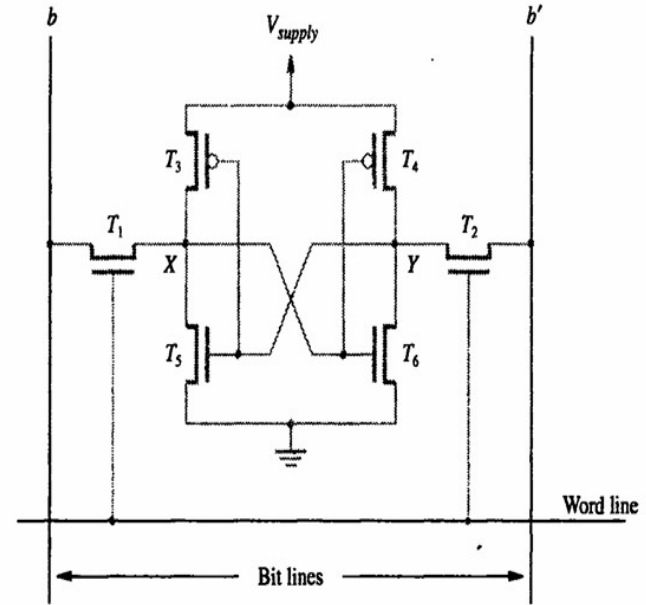


Figure 5.5 An example of a CMOS memory cell.

Static Memories

Feature	SRAM	DRAM
Storage element	Flip-flop (6T)	Capacitor + 1T
Refresh required	No	Yes
Access speed	Very fast	Slower
Density	Low	High
Cost	High	Low
Data retention	While powered	Requires refresh
Use	Cache memory	Main memory

DRAM V/S SRAM

- **DRAM vs SRAM**

- SRAM: fast but expensive (6-transistor cell).
- DRAM: slower but cheaper (1-transistor + 1-capacitor cell).

- **Data storage principle**

- Information stored as charge on a capacitor (C).
- Charge leaks in milliseconds → requires **periodic refresh**.

- **Cell structure** (Fig. 5.6)

- 1 Transistor (T) + 1 Capacitor (C).
- Word line controls transistor gate; bit line connects to I/O.

- **Read/Write operation**

- *Write*: Apply voltage → charge or discharge capacitor.
- *Read*: Sense small voltage on bit line → refresh automatically after read.

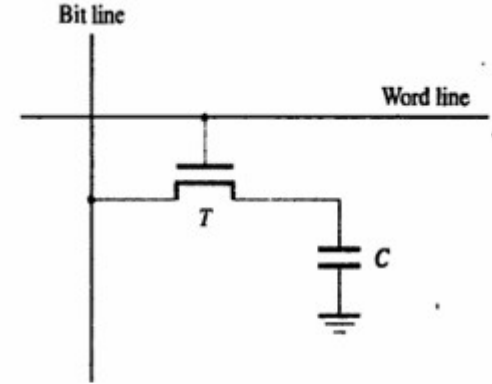


Figure 5.6 A single-transistor dynamic memory cell.

Asynchronous DRAM

Internal Organization of DRAM

Example: 16-Mbit chip.

Array structure

4K × 4K cell array

Control signals

RAS (Row Address Strobe) – latches row address.

CAS (Column Address Strobe) – latches column address

R/ W line – specifies read or write.

CS (Chip Select) – enables the chip.

Read/Write sequence

Apply row address → RAS asserted.

Apply column address → CAS asserted.

Data transferred to/from bit lines (D_0 – D_7).

Automatic refresh

Reading a row refreshes all cells in that row.

Dedicated **refresh circuit** handles periodic updates.

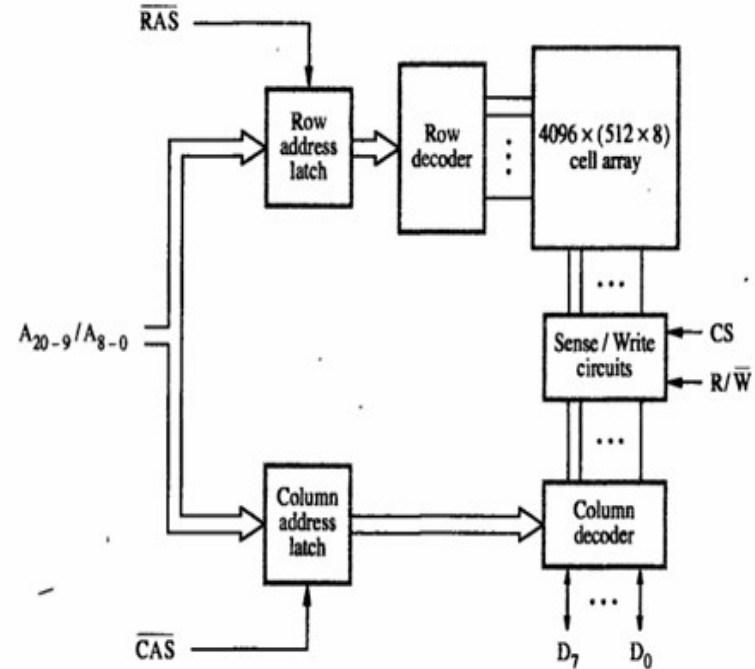


Figure 5.7 Internal organization of a 2M × 8 dynamic memory chip.

Synchronous DRAM

SDRAM is a type of DRAM whose **operations are synchronized with the system clock**. All commands—read, write, refresh—are triggered on the **rising edge of the clock**.

Structure

- Similar cell array as asynchronous DRAM.

- **Row/column address latch** and **data registers** align transfers with the clock.
- Data passes through **input/output registers**, enabling pipelined operation.

•Operation:

- **Mode register** defines timing parameters and burst length.
- **Clock + control signals (RAS, CAS, CS, R/W)** determine data flow.
- Burst transfers supported — data placed on output pins each clock cycle.
- Read data appears after fixed **latency**, then continues sequentially (burst mode).

•Advantages:

- Faster than asynchronous DRAM (no waiting for timing signals).
- Supports **pipelining and block (burst) transfers**.

Synchronous DRAM

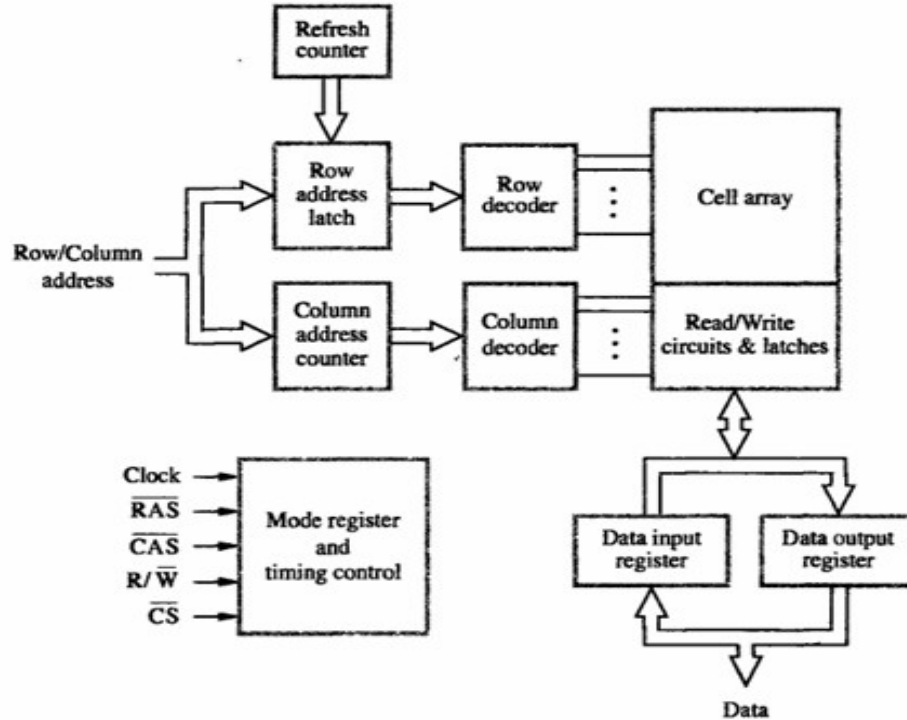


Figure 5.8 Synchronous DRAM.

Rambus Memory (RDRAM)



Purpose & Background

A **RAM Bus** is a **specialized data bus architecture** designed by **Rambus Inc.** to connect the processor (or memory controller) with **RDRAM chips**.

It allows **very high data transfer rates** through **fast signaling** on a **narrow bus**.

Purpose

Traditional SDRAM uses wide buses (64 bits or more) but operates at moderate speeds.

Rambus Memory aimed to **increase bandwidth** without making the bus wider — instead, it made the **bus faster**.

Rambus Channel Communication Flow

Processor (Master) --> Request Packet --> RAM (Slave)

RAM (Slave) --> Acknowledge Packet --> Processor (Master)

RAM (Slave) --> Data Packet --> Processor (Master)

A $1K \times 1$ memory chip requires how many address lines?

- a) 8
- b) 10
- c) 12
- d) 16

Answer: (b)

Explanation:

Number of address lines = $\log_2(1024) = 10$.

Each address accesses one bit ($1K \times 1 = 1024 \times 1$ -bit array)

In **SDRAM**, the term “**burst mode**” refers to:

- a) Writing multiple rows simultaneously.
- b) Reading or writing a sequence of data words in consecutive clock cycles after a single address.
- c) Refreshing multiple rows in one clock cycle.
- d) Transferring data on both clock edges.

Answer: (b)

Explanation: SDRAM burst mode allows sequential access to a series of columns within a row in consecutive cycles, improving data throughput

Which combination of memory technologies provides the best compromise between **speed, cost, and capacity** in a typical computer?

- a) SRAM + DRAM + Magnetic Disk
- b) SRAM + Flash + Optical Disk
- c) DRAM + PROM + Cache
- d) Flash + EEPROM + Tape

Answer: (a)

Explanation:

- SRAM** → Cache (fast)
- DRAM** → Main memory (moderate speed, cost)
- Magnetic Disk** → Secondary storage (large capacity, cheap)

This combination forms the standard memory hierarchy



THANK YOU

Team DDCO

Department of Computer Science and Engineering